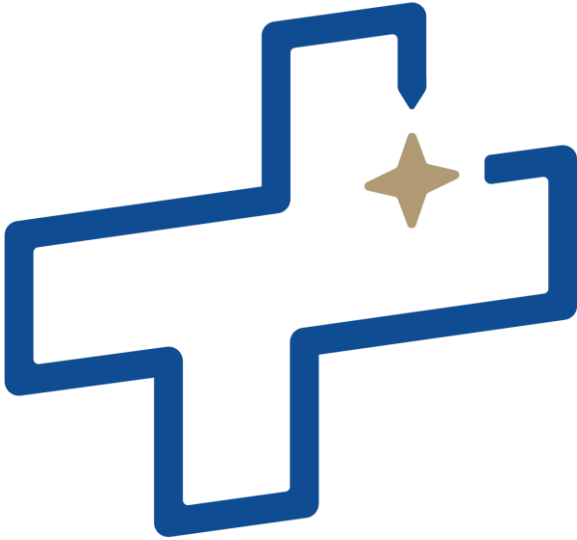




CUBESPACE

CubeSupport

HMI User Manual

DOCUMENT NUMBER
VERSION
DATE
PREPARED BY
REVIEWED BY
APPROVED BY
DISTRIBUTION LIST

CS-DEV.UM.CU-01
1.01
18/09/2023
L. VISAGIE
KM, OS, OG, JM, RvW, CL
W. Morgan
External



Revision History

VERSION	AUTHORS	DATE	DESCRIPTION
A	L. Visagie	05/04/2022	Initial draft
B	L. Visagie	01/08/2023	Version for review
C	L. Visagie	01/08/2023	Update after SW review
D	L. Visagie	01/09/2023	Update after PO review
1.00	L. Visagie	14/09/2023	First version released
1.01	L. Visagie	18/09/2023	Table formatting corrected and figure references added

Reference Documents

The following documents are referenced in this document.

[1]	N/A	(Gen1) CubeADCS ICD, ver 7.4
[2]	N/A	(Gen1) CubeADCS User Manual, ver 7.1
[3]	N/A	(Gen1) CubeADCS Commissioning Manual, ver. 7.2
[4]	CS-DEV.AMNL.BL-01	Bootloader Application Manual, Ver.4.00, or later
[5]	CS-DEV.GD.SWB-01	Software Guide, Ver.1.00, or later
[6]	CS-DEV.FRM.CA-01	CubeProduct Firmware Reference Manual, Ver.7.00, or later
[7]	CS-DEV.UM.CA-01	CubeADCS User Manual, Ver.1.03, or later
[8]	N/A	(Gen1) CubeADCS ICD, ver 7.4
[9]	N/A	(Gen1) CubeADCS User Manual, ver 7.1
[10]	CS-DEV.CMNL.CA-01	CubeADCS Commissioning Manual, Ver.1.00, or later
[11]	N/A	(Gen1) CubeADCS Commissioning Manual, ver. 7.2
[12]	N/A	SW release package/libcubeobc/api



List of Acronyms/Abbreviations

ACP	ADCS Control Program
ADCS	Attitude Determination and Control System
CAN	Controller Area Network
COTS	Commercial Off The Shelf
CSS	Coarse Sun Sensor
CVCM	Collected Volatile Condensable Materials
DUT	Device Under Test
EDAC	Error Detection and Correction
EHS	Earth Horizon Sensor
EM	Engineering Model
EMC	Electromagnetic Compatibility
EMI	Electromagnetic Interference
ESD	Electrostatic Discharge
FDIR	Fault Detection, Isolation, and Recovery
FM	Flight Model
FSS	Fine Sun Sensor
GID	Global Identification
GNSS	Global Navigation Satellite System
GPS	Global Positioning System
GYR	Gyroscope
I2C	Inter-Integrated Circuit
ID	Identification
LTDN	Local Time of Descending Node
LEO	Low Earth Orbit
MCU	Microcontroller Unit
MEMS	Microelectromechanical System
MTM	Magnetometer
MTQ	Magnetorquer
NDA	Non-Disclosure Agreement
OBC	On-board Computer
PCB	Printed Circuit Board



RTC	Real-Time Clock
RWA	Reaction Wheel Assembly
RW	Reaction Wheel
SBC	Satellite Body Coordinate
SOFIA	Software Framework for Integrated ADCS
SPI	Serial Peripheral Interface
SRAM	Static Random-Access Memory
SSP	Sub-Satellite Point
STR	Star Tracker
TC	Telecommand
TCTLM	Telecommand and Telemetry (protocol)
TID	Total Ionizing Dose
TLM	Telemetry
TML	Total Mass Loss
UART	Universal Asynchronous Receiver/Transmitter



Table of Contents

1	Introduction	12
1.1	Software and Hardware Version Applicability	12
1.2	Content Summary	13
2	Environment	14
3	Getting Started	15
3.1	Running CubeSupport from standalone release	15
3.2	Running CubeSupport from software bundles	15
4	User Interface	16
4.1	Nomenclature	16
4.2	Initial (start-up) HMI	18
4.3	Main menu bar	19
4.3.1	Tools	19
4.3.2	Windows	20
4.4	Connection toolbar	20
4.5	User controls interface	20
4.6	Connection Settings UI region	21
4.7	Logging and plot UI region	22
4.7.1	Communication log files	23
4.8	Advanced Features and Offline Functionality	23
4.8.1	Magnetometer Calibration	24
4.8.2	Telemetry log decoder	26
4.8.3	Event log decoder	28
4.8.4	ADCS Configuration Editor	29
4.8.5	Message Encoder	30
4.8.6	Message Decoder	31
4.9	Error Handling	32
5	Connecting to Device	33
5.1	Creating a new connection	33
5.2	Connection Setup	34
5.2.1	CAN setup	34
5.2.1.1	CSP vs CubeSpace Protocol	34
5.2.2	UART	35
5.2.3	RS485 setup	35



5.2.4	I2C setup	35
5.3	Establishing a Connection	35
6	CubeProduct-specific User Interface Elements and Features	38
6.1	General telecommand and telemetry actions	38
6.1.1	Requesting telemetry	38
6.1.2	Sending a telecommand	38
6.1.3	Combined telemetry and telecommand box	39
6.1.4	Special user interface elements for commands or telemetry	39
6.1.4.1	Unix time set/get	39
6.1.4.2	Orbit parameters	39
6.1.5	TCTLM groups	40
6.2	High-level UI tabs vs. automatically generated TCTLM tabs	40
6.2.1	Gen2 Common-Framework auto-generated UI elements	41
6.3	Gen1 Integrated CubeADCS	41
6.3.1	Bootloader	41
6.3.1.1	Uploading new control program	42
6.3.1.2	File downloads	43
6.3.2	Control Program	43
6.3.2.1	Mode selection	45
6.3.2.2	Sub-system power selection	45
6.3.2.3	ADCS Commands	45
6.3.2.4	Configuration	46
6.3.2.5	Images and Logs	48
6.3.2.6	SD card file management	49
6.4	Gen2 Bootloader	51
6.4.1	Firmware/Config Upload Tab	51
6.4.1.1	Local files	52
6.4.1.2	Remote Files	52
6.4.1.3	Jump to App	52
6.4.2	Raw Memory Access tab	52
6.4.2.1	Read from memory	53
6.4.2.2	Write to memory	53
6.4.3	Advanced Tab	53
6.5	Gen2 Integrated ADCS Control Program	53



6.5.1	Port Map Tab	54
6.5.2	CubeComputer tab.....	56
6.5.2.1	Unsolicited telemetry and event log output.....	56
6.5.3	Power State.....	59
6.5.4	ADCS functionality.....	60
6.5.5	ADCS Configuration	60
6.5.6	CubeComputer and Node Health.....	61
6.5.7	File Transfers/Upgrades Tab	61
6.5.7.1	Local files.....	61
6.5.7.2	Remote files.....	62
6.5.8	Telemetry Log interface	63
6.5.9	Event Log interface	65
6.5.10	Image Log interface	66
6.5.10.1	Transfer image from optical sensor to CubeComputer image log.....	66
6.5.10.2	Transfer image directly from sensor.....	67
6.5.11	Pass-Through telecommands and telemetry.....	67
6.6	CubeMag Control Program	68
6.6.1	CubeMag-Application.....	68
6.6.2	CubeMag-Calibration.....	68
6.6.3	CubeMag-Configuration.....	69
6.6.4	CubeMag-Housekeeping	69
6.6.5	CubeMag-Deployable.....	69
6.6.5.1	CubeMag-4-Control-Program-Deploy-1 - Application	69
6.6.5.2	CubeMag-4Control-Program-Deploy-1 - Calibration.....	69
6.6.5.3	PNI Magnetometer Config	69
6.6.5.4	Cube-mag-4-control-program-deploy-1 – Deployment.....	69
6.6.5.5	Cube-mag-4-control-program-deploy-1 – Housekeeping.....	70
6.6.5.6	Cube-mag-4-control-program-deploy-1- Housekeeping telemetry	70
6.7	CubeWheel Control Program	71
6.7.1	Home tab.....	74
6.7.1.1	Creating a telemetry log.....	74
6.7.2	Configuration tab.....	76
6.7.3	Advanced tab.....	76
6.8	CubeSense Sun Control Program	77



6.8.1	CubeSense Sun Home tab functionality	77
6.8.2	CubeSense Sun Advanced tab	78
6.9	CubeSense Earth Control Program	79
6.9.1	Horizon detection	79
6.9.2	Image downloads	80
6.9.3	Sensor Health	81
6.9.4	Dead-pixel masking	82
6.9.5	Detection settings	84
6.10	CubeStar Control Program	86
7	Scripts	87
7.1	Built-in scripts	87
7.2	Loading scripts	88
7.3	Script Options	88
8	Known Issues	89

Table of Tables

Table 1: CubeProducts supported by CubeSupport	12
Table 2: Document content summary	13
Table 3: Minimum host system requirements	14
Table 4: Definitions and examples of UI elements	16
Table 5: Description of advanced features	23
Table 6: Configuration messages to read or set calibration in-flight parameters	24
Table 7: Types of message dialogs, what causes them to appear, and the resulting action	32
Table 8: Default UART settings	35
Table 9: Gen2 CubeComputer Control Program tabs	53
Table 10: Configuration Entries Related to the Detection algorithm	84
Table 11: Built-in scripts	87

Table of Figures

Figure 1: Contents of a standalone CubeSupport release package	15
Figure 2: Main components of the CubeSupport Home Screen	19
Figure 3: CubeSupport menu bar	19
Figure 4: Tools Menu	19



Figure 5: Window menu	20
Figure 6: CubeSupport main window showing additional UI elements when a CubeProduct is connected.	21
Figure 7: Connection Settings UI region.....	22
Figure 8: Logging and plot UI region.....	22
Figure 9 Communication log showing transmitted and received bytes.....	23
Figure 10: Magnetometer calibration utility.....	25
Figure 11: Telemetry log type drop-down selection in Telemetry Log Decoder.....	26
Figure 12: Telemetry Log Decoder Options window.....	27
Figure 13: Decoded telemetry log example.....	28
Figure 14: Event Log Decoder window	29
Figure 15: ADCS configuration editor.....	30
Figure 16: Message Encoder window.....	31
Figure 17: Message Decoder window	32
Figure 18: Creating a new Connection in CubeSupport.....	33
Figure 19: Default connection settings for PCAN (left) and UART (right)	34
Figure 20: Communications ID scan range controls.....	36
Figure 21: Busy Form while attempting to connect to CubeADCS or CubeProducts.....	36
Figure 22: Connection settings UI region after successful PCAN connection.	37
Figure 23: Communications Log	37
Figure 24 Telecommand message UI elements	38
Figure 25: Example of UI elements for a telemetry request.....	38
Figure 26: Telecommand UI elements (with unsent parameter changes)	38
Figure 27: Example of combined telecommand and telemetry UI elements	38
Figure 28: Unix time command/telemetry user interface element	39
Figure 29: Orbit Parameters message with TLE formatting	40
Figure 30: Collapsed ADCS tab group headings, with the Collapse icon indicated.....	40
Figure 31: Tab with auto-generated TCTLM UI elements (left) vs. high-level functionality (right)	41
Figure 32: Gen1 CubeComputer bootloader user interface.....	42
Figure 33: CubeComputer Reset telecommand.....	42
Figure 34: Set Boot Index telecommand.....	43
Figure 35: Gen1 CubeComputer control program UI.....	44
Figure 36: Setting of ADCS run mode.	45



Figure 37: Setting of attitude control and estimation mode.....	45
Figure 38: Power selection of sub-systems.....	45
Figure 39: Actuator commands.....	46
Figure 40: Setting of the reference attitude.....	46
Figure 41: Deployment of magnetometer command.....	46
Figure 42: Open button highlighted, to load XML configuration files provided by CubeSpace.....	47
Figure 43: Save Configuration command, to persist configuration changes to flash memory	48
Figure 44: Orbit configuration.....	48
Figure 45: Saving orbit parameters in flash.	48
Figure 46: Image capture and save command.....	49
Figure 47: Save status.....	49
Figure 48: Initiate logging command.....	49
Figure 49: File download from SD card.....	50
Figure 50: Bootloader tabs and Firmware/Config upload UI elements	51
Figure 51: Bootloader Raw Memory Access tab	53
Figure 52: CubeSupport Port Map Successful Auto-Discovery.....	55
Figure 53: CubeSupport Unsuccessful Auto-Discovery.....	55
Figure 54: Unsolicited telemetry log setup	57
Figure 55: Example of unsolicited telemetry log window in CubeSupport.....	58
Figure 56: Unsolicited event output setup	58
Figure 57: Example of unsolicited events log output in CubeSupport.....	59
Figure 58: Power state TCTLM UI element.....	60
Figure 59: ADCS tab control group headings (collapsed).....	60
Figure 60: CubeComputer File Transfer / Upgrades Tab.....	61
Figure 61: File Load Option	61
Figure 62: Telemetry Log display for CubeComputer	63
Figure 63: Telemetry Log request form.....	64
Figure 64: Event Log request form.....	65
Figure 65: Image Log interface.....	66
Figure 66: Pass-through interface.....	67
Figure 67: CubeMag Control Program Home Screen	68
Figure 68: CubeSupport successful connection.....	71
Figure 69: Home tab for CubeWheel.....	71



Figure 70: Configuration tab for CubeWheel	72
Figure 71: Advanced tab for CubeWheel	72
Figure 72: The UI regions of the Home tab	74
Figure 73: CubeWheel telemetry logs created by CubeSupport.....	75
Figure 74: CubeWheel telemetry log example.....	76
Figure 75: CubeSense Sun Home tab contents	77
Figure 76: Home tab for CubeSense Earth Control Application	79
Figure 77: Detection Results Subsection	79
Figure 78: Detection result - Successful Detection	79
Figure 79: Detection Metadata - No Horizon in View	80
Figure 80: Detection Metadata - Horizon in View	80
Figure 81: Downloading and displaying images	81
Figure 82: Home Tab - Health Overview	82
Figure 83: Home Tab – Profile.....	82
Figure 84: Advanced Tab - DeadPixels.....	82
Figure 85: Advanced Tab – Configuration	82
Figure 86: Advanced Tab - RequestDeadPixels.....	82
Figure 87: Bit-mask Values For Surrounding Pixels	83
Figure 88: Dead Pixels Pre-Masking	84
Figure 89: Dead Pixels After Masking	84
Figure 90: Mask Entry For First Dead Pixel	84
Figure 91: Mask Entry For Second Dead Pixel	84
Figure 92: Scripting window.	87



1 Introduction

CubeSupport is an all-inclusive software application consisting of tools and utilities used to connect to and interface with the CubeSpace ADCS (CubeADCS) and other CubeSpace products. The application supports both CubeSpace's Generation 1- and Generation 2 integrated CubeADCS as well as standalone CubeProducts. The CubeSupport application supports multiple hardware interfaces such as USB-CAN, USB-UART, USB-I2C and USB-RS485.

Note that I2C and RS-485 are available for internal CubeSpace use but not offered as standard options for client interfacing to CubeProducts.

This manual provides an overview of the CubeSupport Human-Machine-Interface (HMI) features and how to use the application to interface with CubeSpace Products.

1.1 Software and Hardware Version Applicability

All communication with CubeSpace CubeProducts rely on that component implementing a defined set of commands and telemetry request responses. This definition is also called the "Node Definition" or NodeDef. Node Definitions are specified by a node type identifier (a combination of CubeProduct type and program type), and an interface version. All such node definitions and APIs can be found in the latest software release package [12].

Table 1 details the list of programs and their respective Node Definitions to which CubeSupport can interface.

Table 1: CubeProducts supported by CubeSupport

CUBEPRODUCT	PROGRAM	INTERFACE VERSION(S)
GEN1 CUBEPRODUCTS		
CubeComputer Gen1	Flash bootloader	3
	Control program (CubeACP)	3,5,6,7
CubeStar Gen1	Control program	4,5
CubeWheel Gen1	Control program	2
GEN2 CUBEPRODUCTS		
CubeComputer Gen2	Flash bootloader	4
	Control program	8
CubeMag Gen2	Common-node bootloader	4
	Control program deploy	1
	Control program compact	1
CubeSense Sun Gen2	Common-node bootloader	4
	Control program	5
CubeSense Earth Gen2	Common-node bootloader	4
	Control program	1
CubeStar Gen2	Common-node bootloader	4
	Control program	6



CUBEPRODUCT	PROGRAM	INTERFACE VERSION(S)
CubeWheel Gen2	Common-node bootloader	4
	Control program	3
CubeNode Gen2	Common-node bootloader	4
	Control program NSS-RWL	1
	Control program PST3S	1

1.2 Content Summary

The remainder of this User Manual is organized in the following sections.

Table 2: Document content summary

CHAPTER	TITLE	DESCRIPTION
2	Environment	The minimum system requirements and environment required to run CubeSupport.
3	Getting Started	Steps for setting up and running the application for the first time.
4	User Interface	An overview of the application, main features, interfaces, and advanced controls, including functionality that does not require a hardware connection (Telemetry and event log decoder interfaces, magnetometer calibration utility, ADCS configuration editor and communications message encoder and decoders).
5	Connecting to Device	Detailed instructions on creating and modifying connections, communication protocols and connecting to CubeSpace hardware.
6	CubeProduct-specific User Interface Elements and Features	Description of how to interact with connected CubeProducts through their respective UI elements.
7	Scripts	Detailed description on loading and running scripts in CubeSupport



2 Environment

To run CubeSupport, the host computer must meet the minimum requirements in Table 3.

Table 3: Minimum host system requirements

ITEM	VERSION
Operating System	Windows version 7 and higher
Architecture	x86/x64
.Net Framework	V4.8 or higher
RAM	4 GB
Disk space	100MB
Hardware connectivity	1 x USB 2.00 or higher



3 Getting Started

This Chapter describes how to set up and run CubeSupport for the first time. CubeSupport is distributed either as part of a software release bundle or as a standalone application. Instructions are provided for installing the software for both scenarios.

3.1 Running CubeSupport from standalone release

The standalone software is distributed as a compressed file with the following name:

sw-cubesupport-release-vX.Y.Z.tar.gz

Where X, Y and Z denote the major, minor and patch versions respectively.

Unzip the package to a known location and navigate to the resulting folder. If successful, the following files should be visible in the folder:

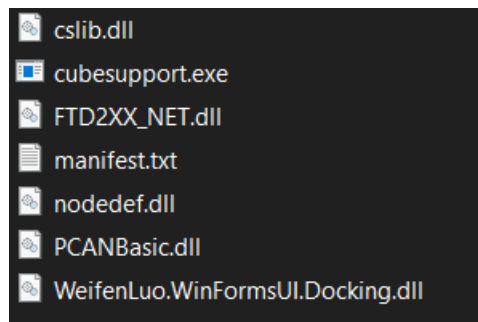


Figure 1: Contents of a standalone CubeSupport release package

Double Click on the **cubesupport.exe** file to run the program. There is no need to run an installation program.

3.2 Running CubeSupport from software bundles

Information relating to the software release bundle can be found in “Folder Structure” in the Software Guide [5]. Once the files have been unzipped, open the folder, and navigate to <folder path>\tools\cube-support. the same files shown in Figure 1 should be visible.



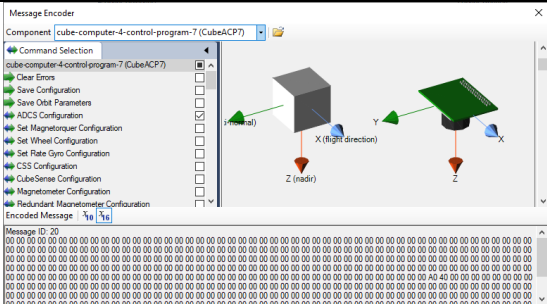
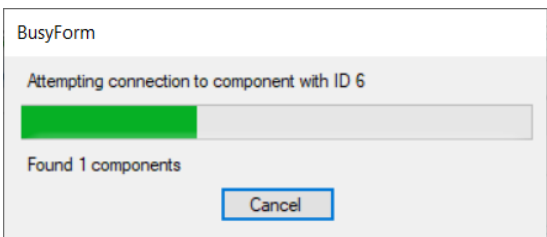
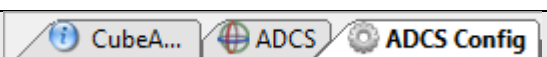
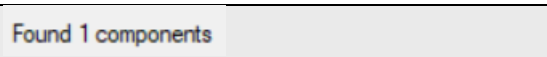
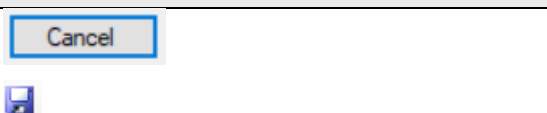
4 User Interface

This chapter describes the CubeSupport main HMI and functionality.

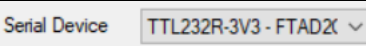
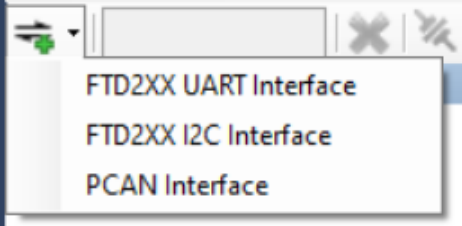
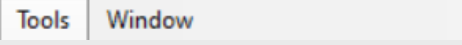
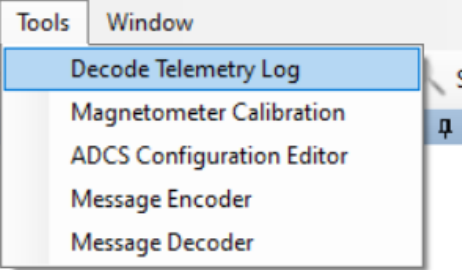
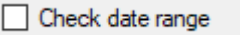
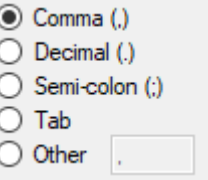
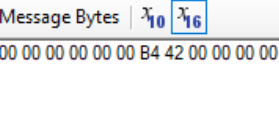
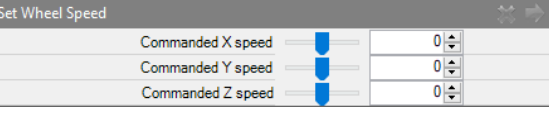
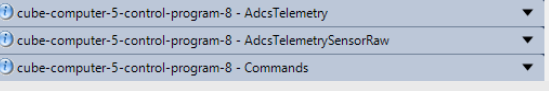
4.1 Nomenclature

Table 4 defines terms for various user interface elements that are referred to throughout the document.

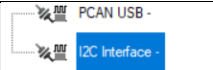
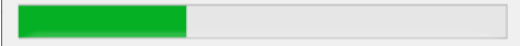
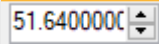
Table 4 Definitions and examples of UI elements

TERM	DEFINITION	EXAMPLE(S)
UI element	A generic term to refer to all possible parts of the HMI that a user may interact with. All of the items listed in this table are UI elements.	
UI region	A rectangular area with any number of UI elements, dedicated to a particular function.	
Window	A rectangular, bordered area on the screen dedicated to a particular function. The main CubeSupport HMI is displayed in a window. Windows have a title bar and optional buttons on the title bar to close, minimize or maximize the window. Dialogue windows that display error message and information are also windows.	 
Tab (bar)	Mouse-selectable buttons which causes a different set of UI elements to be displayed in the region below the tab bar.	
Toolbar	A strip of buttons, labels and other UI elements that are grouped in a horizontal row, and whose function applies to the region just below it.	
Label	Text that is displayed on the HMI that does not have any interaction options, other than the user reading it.	
Button	Any area on the UI that exercises an action when the button has captured the mouse cursor and the user presses a mouse button. Buttons can have simple text labels indicating the action, or icons (pictures). The latter are prevalent on toolbars.	



TERM	DEFINITION	EXAMPLE(S)
Drop-down list	A UI element with a text portion and a drop-down arrow that causes a vertical arrangement of menu options to appear if selected.	
Drop-down button	A button (usually on a toolbar) that causes a vertical arrangement of menu options to appear if selected.	
Menu bar	A horizontal strip of buttons with text only, of which the menu buttons cause a drop-down menu to be displayed if selected.	
Drop-down menu	A vertical arrangement of text buttons with different menu options, that appear if an element on a Menu bar is selected.	
Checkbox	A square box with label next to it. Selecting the square box (by pressing mouse button while mouse cursor is in the box, or pressing the ENTER key if the checkbox has focus) enables or disables a setting that may either be on or off (Boolean).	
Radio button(s)		
Text box	A white rectangular area in which the user can type text. The text input may be limited by the type of data – any text vs. only numerical data. The type of input is indicated by a label above or to the left of the text box.	
TCTLM UI element(s)	A combined group of UI elements to edit telecommand parameters and/or display telemetry values. TCTLM UI elements have a header label to display the communications (API) message to which the UI elements apply.	
TCTLM group header	A coloured label with drop-down arrow on the right. These headers are used to group TCTLM UI elements together based on particular function (i.e. TCTLM for ADCS Telemetry, Raw sensor telemetry, commands etc.)	



TERM	DEFINITION	EXAMPLE(S)																																																
Table	A grid or table arrangement of text values. The table has a header row indicating the type or source of content in that column.	<table><tr><th>Abstract Type</th><th>Node Type</th><th>Serial Integer</th><th>Slot</th></tr><tr><td>Str0</td><td>CubeNode1</td><td>21005</td><td>6</td></tr><tr><td>Fss0</td><td>CubeSense4</td><td>21223</td><td>4</td></tr><tr><td>Fss1</td><td>CubeSense4</td><td>21224</td><td>5</td></tr><tr><td>Hss0</td><td>CubeIrr1</td><td>2108</td><td>9</td></tr><tr><td>Hss1</td><td>CubeIrr1</td><td>2109</td><td>10</td></tr><tr><td>Mag0</td><td>CubeMagDeploy4</td><td>21002</td><td>7</td></tr><tr><td>Mag1</td><td>CubeMagCompact4</td><td>21005</td><td>8</td></tr><tr><td>Rwl0</td><td>CubeWheel2</td><td>2405</td><td>0</td></tr><tr><td>Rwl1</td><td>CubeWheel2</td><td>2410</td><td>1</td></tr><tr><td>Rwl2</td><td>CubeWheel2</td><td>2415</td><td>2</td></tr><tr><td>Rwl3</td><td>CubeWheel2</td><td>2419</td><td>3</td></tr></table>	Abstract Type	Node Type	Serial Integer	Slot	Str0	CubeNode1	21005	6	Fss0	CubeSense4	21223	4	Fss1	CubeSense4	21224	5	Hss0	CubeIrr1	2108	9	Hss1	CubeIrr1	2109	10	Mag0	CubeMagDeploy4	21002	7	Mag1	CubeMagCompact4	21005	8	Rwl0	CubeWheel2	2405	0	Rwl1	CubeWheel2	2410	1	Rwl2	CubeWheel2	2415	2	Rwl3	CubeWheel2	2419	3
Abstract Type	Node Type	Serial Integer	Slot																																															
Str0	CubeNode1	21005	6																																															
Fss0	CubeSense4	21223	4																																															
Fss1	CubeSense4	21224	5																																															
Hss0	CubeIrr1	2108	9																																															
Hss1	CubeIrr1	2109	10																																															
Mag0	CubeMagDeploy4	21002	7																																															
Mag1	CubeMagCompact4	21005	8																																															
Rwl0	CubeWheel2	2405	0																																															
Rwl1	CubeWheel2	2410	1																																															
Rwl2	CubeWheel2	2415	2																																															
Rwl3	CubeWheel2	2419	3																																															
List	A vertical arrangement of items that are displayed in a dedicated region. Items in the list may optionally have icons to their left, and may be selected.																																																	
Progress bar	A horizontal bar with shading to indicate progress or completion status of the currently executing task.																																																	
Numeric up-down editor	Text box with up and down buttons to the right. The text box allows editing values using the keyboard and the up and down buttons can be used to make positive or negative increments to the value.																																																	
Slider	A UI element that can be interacted with using the mouse. Holding the mouse button down while the mouse cursor is positioned over the slider allows the value to be changed by dragging the mouse.																																																	

Note Throughout the rest of this document, **this style** is used to indicate UI elements on the CubeSupport user interface which may be interacted with.

Note The term “interface” when used on its own in this document without preceding “user” is used to refer to a communications interface – a means of interfacing from the application to the CubeADCS or CubeProduct through a hardware communications link.

4.2 Initial (start-up) HMI

When running for the first time, the program HMI will appear. The main window, as well as the UI regions in it are shown in Figure 2 and consist of:

- Main menu bar
- Connection toolbar
- Connected functions UI region
- Connection settings UI region
- Logging and plot UI region

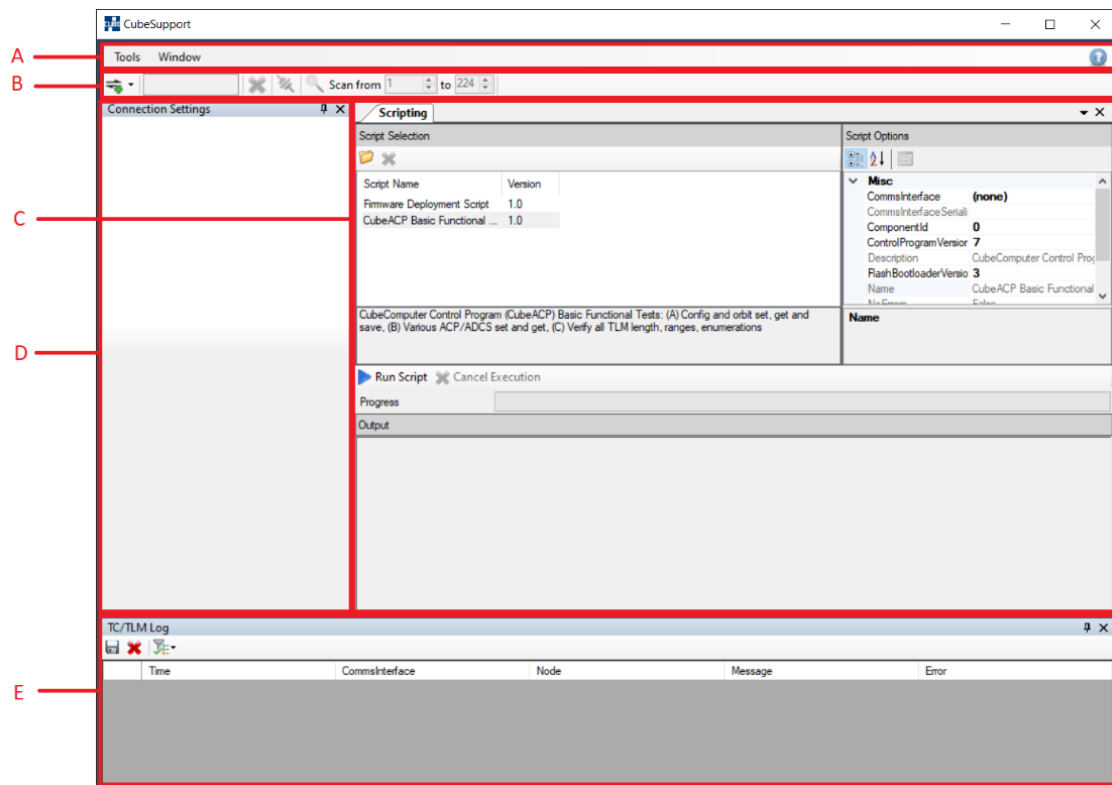


Figure 2: Main components of the CubeSupport Home Screen

4.3 Main menu bar

The menu bar provides quick access to CubeSupport features and interface options. The menu bar shown in Figure 3 has two menu options i.e. **Tools** and **Window**.

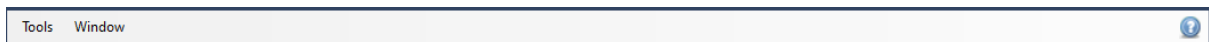


Figure 3: CubeSupport menu bar

4.3.1 Tools

The **Tools** menu contains a list of features that can be used without a connection to hardware, as shown in Figure 4. Detail on these features are given in section 4.8.

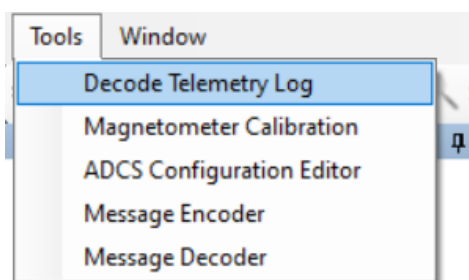


Figure 4: Tools Menu



4.3.2 Windows

The **Window** menu contains options to modify the main screen by closing and hiding the various UI regions. These options are shown in Figure 5 below.

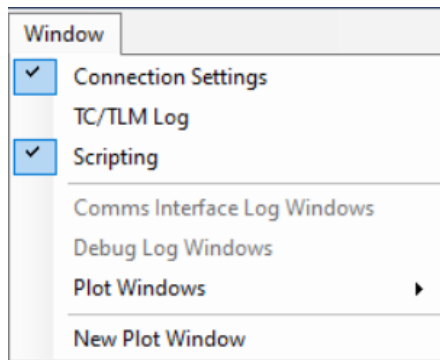


Figure 5: Window menu

UI regions with a “✓” are enabled and will appear in the main window. Clicking on an entry in the menu list will toggle the view of the selected UI region. By default, the **Connection Settings** and **Scripting** are enabled. The **Comms Interface Log Windows** and **Debug Log Windows** will be disabled (greyed out) until a valid connection to a CubeProduct is established (see Section 5). Finally, the **New Plot Window** item will add a plot tab to the Logging and plot UI region (see section 4.7). This will allow for the plotting of any Telemetry incoming from the Cube ADCS or Cube Products. The user can enable or disable the plot tabs by selecting **Plot Window**→<plot window name>.

4.4 Connection toolbar

This toolbar is used to create connections to the desired CubeProduct or CubeADCS. The toolbar provides options to select a communications interface, clear current interfaces, connect to or disconnect from a device, and select the node IDs to scan through when searching for a device. A more detailed description for creating connections can be found in Chapter 5.

4.5 User controls interface

On successful connection to a CubeProduct, additional UI elements will appear as a tab in the Connected functions UI region. These tabs contain functions that are applicable to the connected CubeProduct. An example is shown in Figure 6.

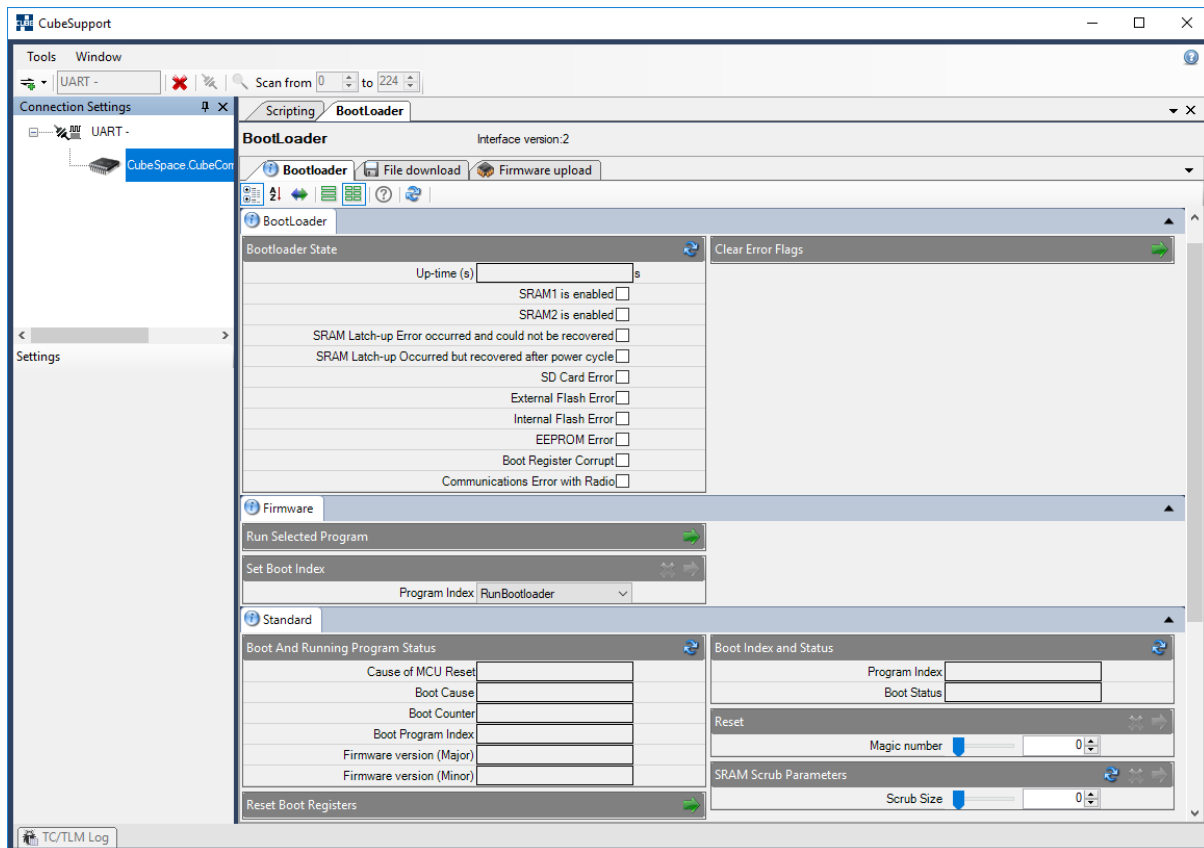


Figure 6: CubeSupport main window showing additional UI elements when a CubeProduct is connected.

By default, the **Scripting** tab appears even when the program is run for the first time with no devices connected. This forms part of the advanced features which are discussed in section 7.

4.6 Connection Settings UI region

When a new connection is created, the corresponding properties are displayed in the **Connection Settings** UI region as shown in Figure 7.

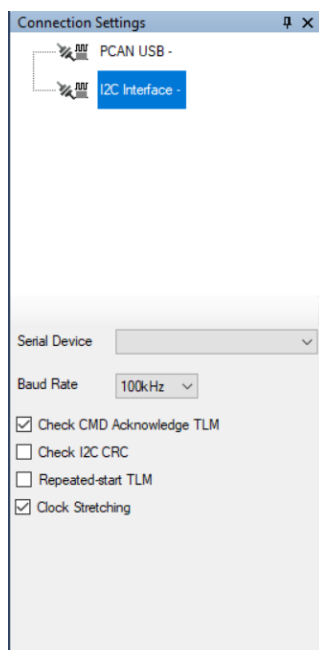


Figure 7: Connection Settings UI region

Multiple simultaneous hardware connections are supported. By default, the connection list appears empty and is populated with connections as they are created. Connections can be selected by clicking on the list which displays the setting options for the selected communications interface on the bottom part of the Connection settings UI region. Further details on how to change connection settings for communication interfaces can be found in Chapter 5.

4.7 Logging and plot UI region

The **Logging and plot** UI region is shown in Figure 8. This region of the user interface displays UI elements to view and log data incoming and outgoing to the device. Communication with CubeSpace hardware is performed using telecommands and telemetry requests (see Communication Protocols in [6]).

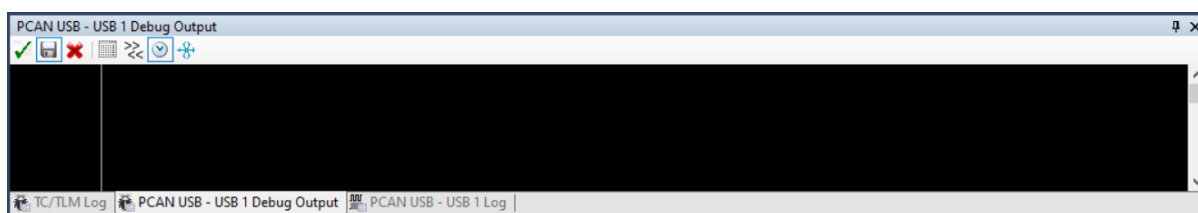


Figure 8: Logging and plot UI region

By default, the UI region displays only the **TC/TLM Log**. However, when a connection is created and used, two tabs appear with the connection name – the one with a header name in the form **<connection> - Debug Output**, while the other with a header name in the form **<connection> - Log**. The **<connection> - Log** tab shows data packets being sent and received as a hexadecimal string with each hex value representing a byte, as in Figure 9.

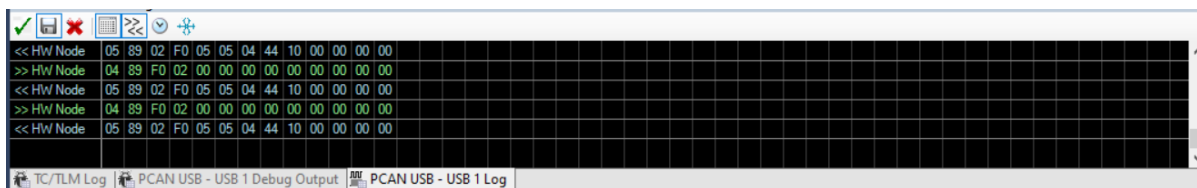


Figure 9 Communication log showing transmitted and received bytes

The `<connection>` - `Debug Output` tab is intended for CubeSpace internal use and will not display any information when interfacing to CubeProducts with release firmware. The user can switch which log is displayed by selecting the desired tab displayed at the bottom of the interface.



The `TC/TLM Log` tab will be deprecated in future versions of CubeSupport and does not display any information.

4.7.1 Communication log files

All communication that takes place with hardware will automatically be logged to hard disk. The log files for each hardware interface, per session, are stored in a sub-folder called *output*, under the CubeSupport root folder.

4.8 Advanced Features and Offline Functionality

In addition to the above-mentioned main features, CubeSupport contains a suite of tools to assist with calibration, validation, and verification of CubeSpace hardware and systems that does not require nor makes use of a hardware connection. The features available are shown in Table 5 with links to the relevant chapter and or section.

Table 5: Description of advanced features.

FEATURE	DESCRIPTION	CHAPTER / SECTION
Scripting	Run scripts to test and verify CubeProduct functionality.	7
Decode Telemetry Log	Decode a binary Telemetry log (*.TLM) file from the CubeADCS CubeComputer and display the data in a readable format, or export to CSV file. (Not applicable to standalone CubeProducts)	4.8.2
Decode Event Log	Decode a binary Event log file from CubeADCS and display the data in a readable format, or export to CSV file	4.8.3
Magnetometer Calibration	Calculates updated calibration coefficients for a CubeMag used on an integrated CubeADCS	4.8.1
ADCS Configuration Editor	Provides a convenient UI to load, edit and save configuration settings to/from XML file (supports Gen1 CubeADCS only)	4.8.4



FEATURE	DESCRIPTION	CHAPTER / SECTION
Message Encoder	Displays the byte encoding for a communication message to any CubeProduct	4.8.5
Message Decoder	Displays the decoded communication message fields from a provided byte array	4.8.5

4.8.1 Magnetometer Calibration

The magnetometer calibration utility is available from the **Tools** menu of CubeSupport. This utility is typically used to determine in-orbit calibration parameters for the magnetometer in an integrated CubeADCS.

The calibration parameters in question are configuration settings for the CubeComputer control program. Specifically, the TCTLM messages in Table 6 can be used to read back or alter the magnetometer in-flight calibration values.

Table 6: Configuration messages to read or set calibration in-flight parameters

	GEN1 CUBEADCS (CUBECOMPUTER4 CONTROL-PROGRAM-7)	GEN2 CUBEADCS (CUBECOMPUTER5 CONTROL-PROGRAM-8)
Message name(s)	MagConfig (Magnetometer Configuration) RedMagConfig (Redundant Magnetometer Configuration)	ConfigMag0OrbitCal (Mag0 magnetometer in-orbit calibration configuration) ConfigMag1OrbitCal (Mag1 magnetometer in-orbit calibration configuration)
TLM Get ID(s)	204, 205	191, 194
TC Set ID(s)	26, 36	63, 66

Each magnetometer used in a CubeADCS must be calibrated separately.

In an integrated CubeADCS, magnetometer measurements pass through two steps of calibration corrections, where the raw magnetic field sensing element measurements are converted to calibrated values for use in the estimation and control algorithms. In the first step (so-called pre-flight or “on-ground” calibration correction), the raw magnetometer sensing element output is converted to a 3-axis measured B-field vector in μT units.

The coefficients that are used to perform this first step correction, are determined by measurement in a Helmholtz-coil setup, as part of the production process at CubeSpace. These coefficients are stored as configuration values in non-volatile memory. In Generation 1 CubeADCS, the coefficients are stored in the CubeComputer control-program (CubeACP) system configuration. In generation 2 CubeADCS, the coefficients are stored in the CubeMag Deploy and Compact themselves and the calibration correction is performed by the CubeMag.



The on-ground calibration coefficients for the magnetometer are determined during production through careful measurement and should not be changed.



In the second calibration correction step, a “delta” correction is applied to correct for any offsets or non-orthogonality that occurs as a result of the magnetometer mounting in the satellite and / or the disturbances it experiences. It is the coefficients (sensitivity matrix values and offset vector) of this latter in-flight correction that are calculated using the calibration process described in this section.

To perform this calibration, it is required to have a log of already calibration-corrected magnetometer measurements, in μT units, taken at intervals of 10s. The measurements that are to be used, would have thus already passed through both sets of calibration corrections. The new coefficients that are produced will have to be incorporated with existing in-flight calibration settings. The current in-flight calibrated coefficients (obtained using telemetry request as in Table 6) must be known.

For details on obtaining the required telemetry log, please consult the ADCS Commissioning Manual [10][3].

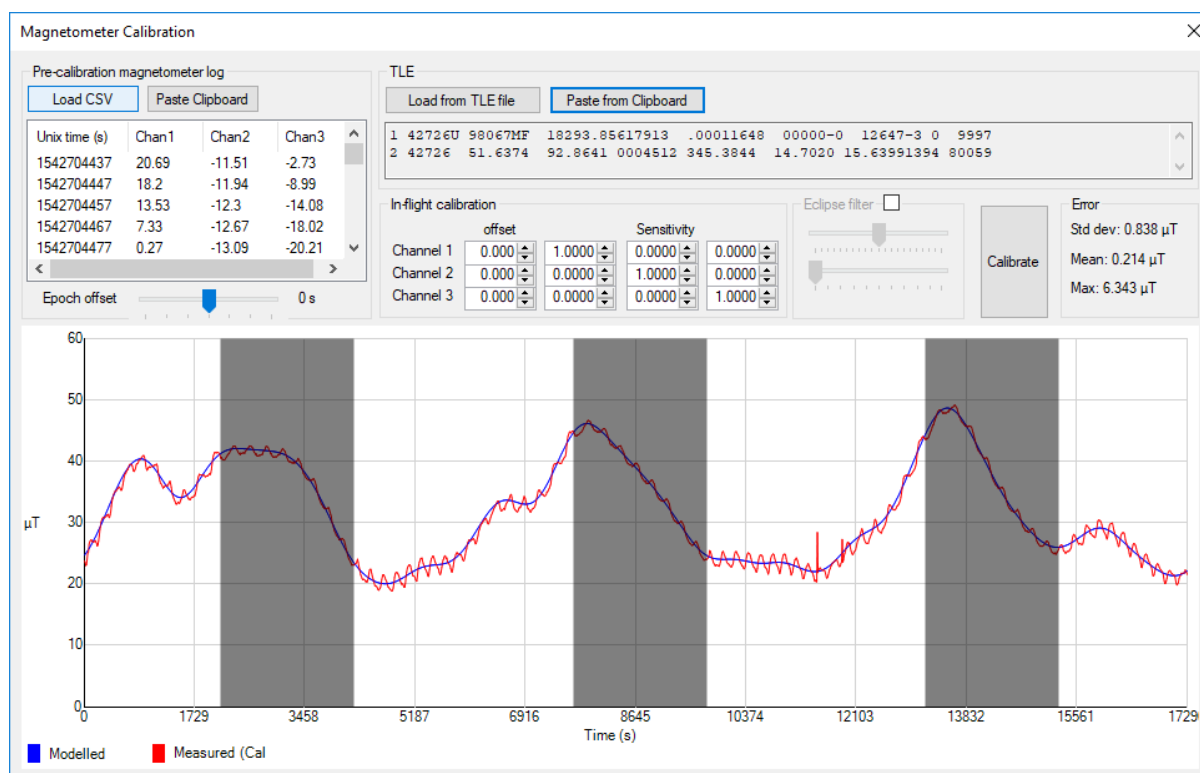


Figure 10: Magnetometer calibration utility

The log file in CSV format is loaded by pressing the **Load CSV** button. Alternatively, the log data can be pasted from the clipboard, after copying it from a spreadsheet program.

Secondly, the Two-Line Elements (TLEs) are required for the same time that the log was sampled. This can be loaded from file using the **Load From TLE file** button or pasted from the clipboard.

After loading the relevant bits of information, the plot UI region will automatically display two line graphs with ideal B-field magnitude, and measured B-field magnitude over time, using the current calibration coefficients as seen in Figure 10.

Press the **Calibrate** button to calculate updated offset and sensitivity values. The values in the offset and sensitivity controls will update automatically, and the graph trace will update to show the error after the improved calibration correction. The sensitivity matrix elements and offset vector values represent the



“delta” calibration that should be applied to the calibration settings that were used when the log was sampled.

If the log file was obtained with the in-flight magnetometer calibration offset set to zero, and identity transform for the sensitivity matrix (typical for the first time the magnetometer is calibrated in flight), then the new offset and sensitivity matrix is simply the values provided by the calibration utility. But if the in-flight calibration was already set to some other calibration, it is necessary to compute a new offset and sensitivity matrix from:

$$\mathbf{d}_{new} = \mathbf{d}_{\Delta} + \mathbf{S}_{\Delta} \mathbf{d}_0$$
$$\mathbf{S}_{new} = \mathbf{S}_{\Delta} \mathbf{S}_0$$

Where $\mathbf{d}_0$ and $\mathbf{S}_0$ is the in-flight offset vector and sensitivity matrix that was used when the log file was sampled. $\mathbf{d}_{\Delta}$ and $\mathbf{S}_{\Delta}$ is the offset vector and sensitivity matrix that CubeSupport calculates, and $\mathbf{d}_{new}$ and $\mathbf{S}_{new}$ is the new offset vector and sensitivity matrix to use (for the in-flight calibration settings).

To complete the process, update the magnetometer configuration sections on the actual CubeADCS by sending the telecommands listed in Table 6.

4.8.2 Telemetry log decoder

The telemetry log decoder is accessed from the **Tools** menu in CubeSupport, by selecting the **Decode Telemetry Log** menu option.

It is required to inform the utility which CubeComputer program created the telemetry log. This is achieved using the **Telemetry Log Type** drop-down list, as in Figure 11.

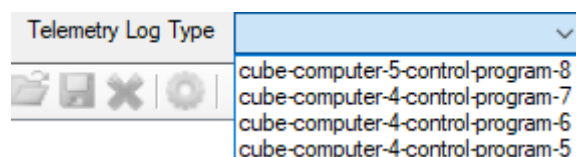


Figure 11: Telemetry log type drop-down selection in Telemetry Log Decoder

For Gen1 CubeADCS, use the **cube-computer-4-control-program-7** option. For Gen2 CubeADCS, use the **cube-computer-5-control-program-8** option. Other control program options have since been deprecated but are still available in the user interface for backwards compatibility.

Once the telemetry log type has been selected, the **Load**, **Save**, **Clear Log** and **Options** toolbar buttons become available.

The Options toolbar button causes a window to appear from where the decoding and export options can be adjusted. The **Export delimiter** sets the character that is used to delimit fields in the exported telemetry log file. The date range in the **Import Settings** group of UI elements can be used to restrict the decoded telemetry entries to fall in a certain range. This option is useful in cases where parts of the binary log file is corrupt. Corrupt entries will typically have unix time stamp very far into the future or past, and such entries will be discarded if the **Check date range** checkbox is checked.



Options

Export delimiter

☒ Comma (,)

☐ Decimal (.)

☐ Semi-colon (;)

☐ Tab

☐ Other

Import Settings

☐ Check date range

between

and

OK

Figure 12: Telemetry Log Decoder Options window

Use the **Load** button to display a file browser dialog window and navigate to the binary telemetry log file. After selecting **OK**, the Telemetry Decoder decodes the log and display the entries in a grid, as in Figure 15. For large telemetry files, the grid display takes too long to render, and the CubeSupport application will display a message indicating such. The decoded data is still available in memory and can be exported to CSV file by pressing the **Save** button.



Telemetry Decoder								
Telemetry Log Type cube-computer-4-control-prograr								
Nadir S...	Rate Se...	Wheel ...	Coarse ...	StarTra...	Star Tra...	Estimat...	Estimat...	Estimat...
False	False	False	True	False	False	-4.11	-5.49	-5.99
False	False	False	True	False	False	0.66	0.61	-7.69
False	False	False	True	False	False	5.05	2.83	-8.97
False	False	False	True	False	False	3.87	1.67	-4.56
False	False	False	True	False	False	1.29	0.36	-1.15
False	False	False	True	False	False	0.51	-0.04	-0.15
False	False	False	True	False	False	0.27	-0.1	0.08
False	False	False	True	False	False	54.53	15.33	21.12
False	False	False	True	False	False	40.92	6.37	8.12
False	False	False	True	False	False	11.76	1.22	2.15
False	False	False	True	False	False	2.16	0.05	0.69
False	False	False	True	False	False	0.23	-0.14	0.29
False	False	False	True	False	False	0.13	-0.11	0.21
False	False	False	True	False	False	43.96	-8.32	12.33
False	False	False	True	False	False	17.54	-4.55	4.6
False	False	False	True	False	False	3.27	-1.32	1.31

Processing log

Successfully decoded 1128 entries

Figure 13: Decoded telemetry log example

4.8.3 Event log decoder

The Event Log Decoder is accessed from the [Tools](#) menu in CubeSupport. The window that is displayed for this utility is shown in Figure 14.



Local Date/time	Remote date/tim...	Remote date/tim...	Uptime (s)	Class	Source	Details
-----------------	--------------------	--------------------	------------	-------	--------	---------

Event name		Class		Counter	
Description			Source		
Parameters					
	Parameter	Value			

Figure 14: Event Log Decoder window

The event log is intended to decode binary event data that was downloaded from the CubeComputer. To open such a log, press the **Open** toolbar button (), and navigate to the desired file. CubeSupport will display the decoded event file contents in a table view. This can be exported to a CSV file using the **Save** button (). The table can be cleared by pressing the **Clear log** button ().

The bottom UI region of the window in Figure 14 will display details about each event, when a row in the decoded event table is selected.

4.8.4 ADCS Configuration Editor

The User-adjustable ADCS Configuration settings for the Gen1 CubeADCS can be edited or viewed off-line (without an active connection to the ADCS hardware), by selecting the **ADCS Configuration Editor** option from the **Tools** menu. This allows for convenient changing and viewing of settings, and also import and export of the settings to XML file.

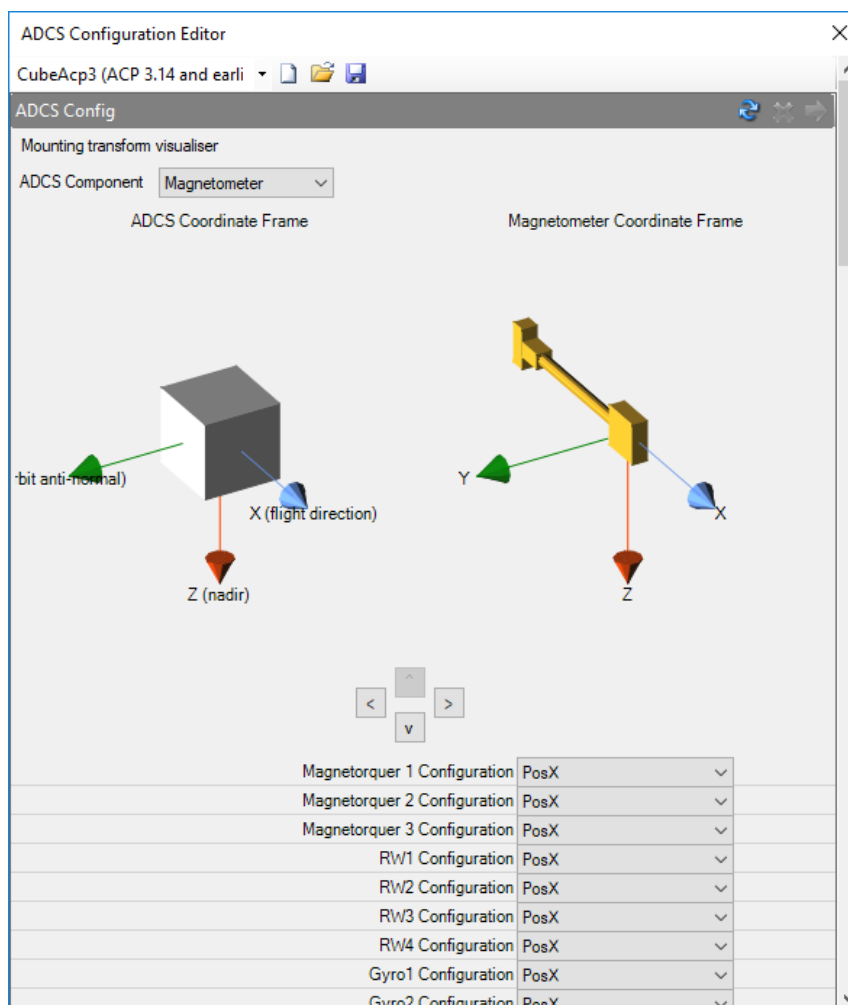


Figure 15: ADCS configuration editor

It is important to select the correct version of the ACP to which the configuration applies, from drop-down list at the top of the window.

The ADCS configuration editor also has a mounting angle visualisation included. This allows for mounting angles to be verified. To make use of this, first select the CubeProduct for which the mounting is to be checked from the **ADCS Component** drop-down list.

The visualisation displays the selected CubeProduct on the right, by making use of the currently specified mounting orientation. This should be checked against the cube on the left. The left-hand cube and coordinate system displays the ADCS coordinate system, relative to which all other mountings are specified.

4.8.5 Message Encoder

The Message Encoder utility allows the user to observe the bytes into which a communication message must be packed, in order to satisfy the interface definition for that message. The Message Encoder window is displayed in Figure 16.

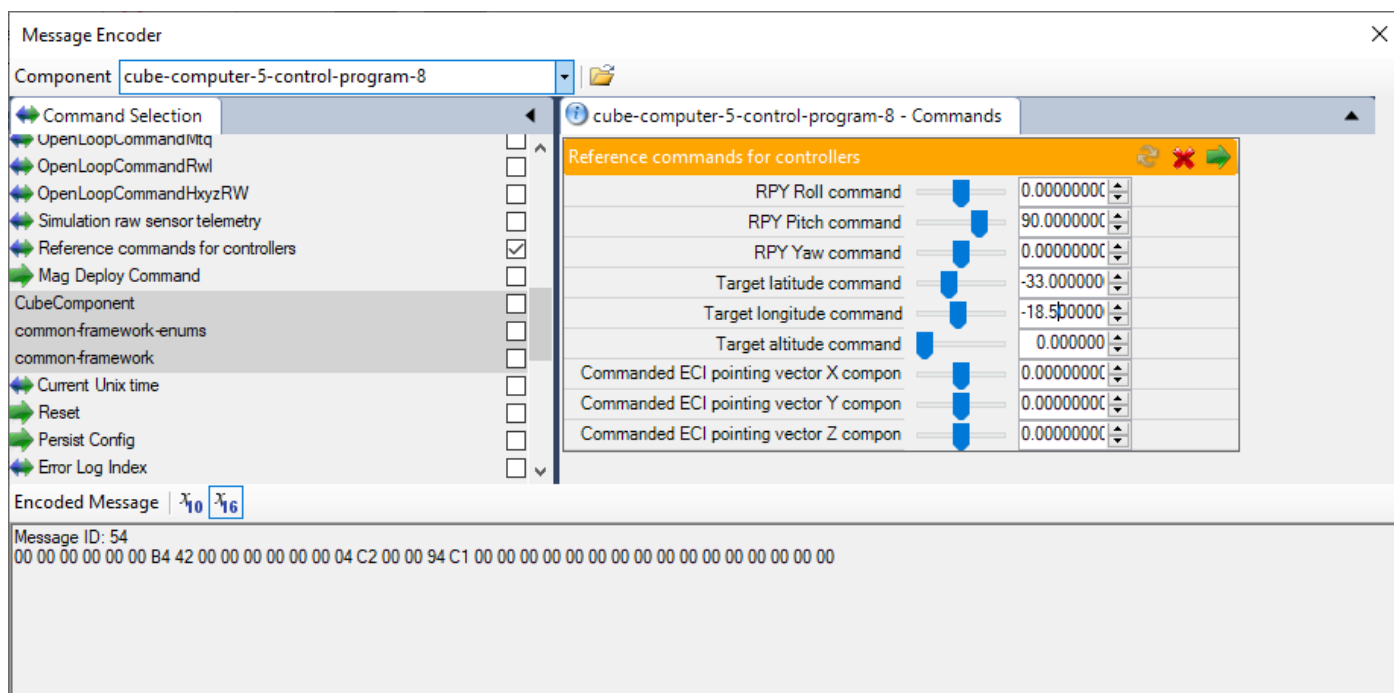


Figure 16: Message Encoder window

The user must first select the CubeProduct for which the communication message is intended for from the **Component** drop-down.

Upon doing this, the list of available commands for that CubeProduct is populated in the **Command Selection** UI region. The command for which the byte encoding is to be viewed must be selected from this list. Once this is done, the command parameters appear on the right-hand side of the window.

The command parameters can be adjusted in the usual way. The final byte array that comprises this command message will be displayed at the bottom of the window in Figure 16, along with the message ID. Note that this only displays the raw byte array for the message, and does not include any additional headers or escape bytes as needed for a particular communication interface.

It is also possible to load an XML file with command message definition and parameters using the **Open** button (). This automatically displays the message parameters as they were loaded from the file, along with the encoded byte array.

4.8.6 Message Decoder

The Message Decoder utility performs the opposite of the Message Encoder. It takes as input an array of bytes, with known message type, and unpacks the fields from it into user-friendly values. An example is shown in Figure 17.

The user must first select the CubeProduct from which the message bytes originate, from the **Component** drop-down list. Next, select the message type to which the bytes apply from the **Message Type** drop-down list. Finally, populate the message bytes in the **Message Bytes** text box. Bytes should be separated with a space. By default, the text box expects hexadecimal encoding, in which case byte values are supplied in hexadecimal notation (no leading characters or format specifiers such as '0x' should be used).



It is also possible to specify byte values in decimal, after selecting decimal notation input using the  button.

If the supplied bytes can successfully be decoded, the resultant user-friendly fields will be displayed in the **Decoded Message** text box. If there are errors in the input (invalid number of bytes specified, or data that cannot be converted) an error message is displayed in the **Decoded Message** text box.

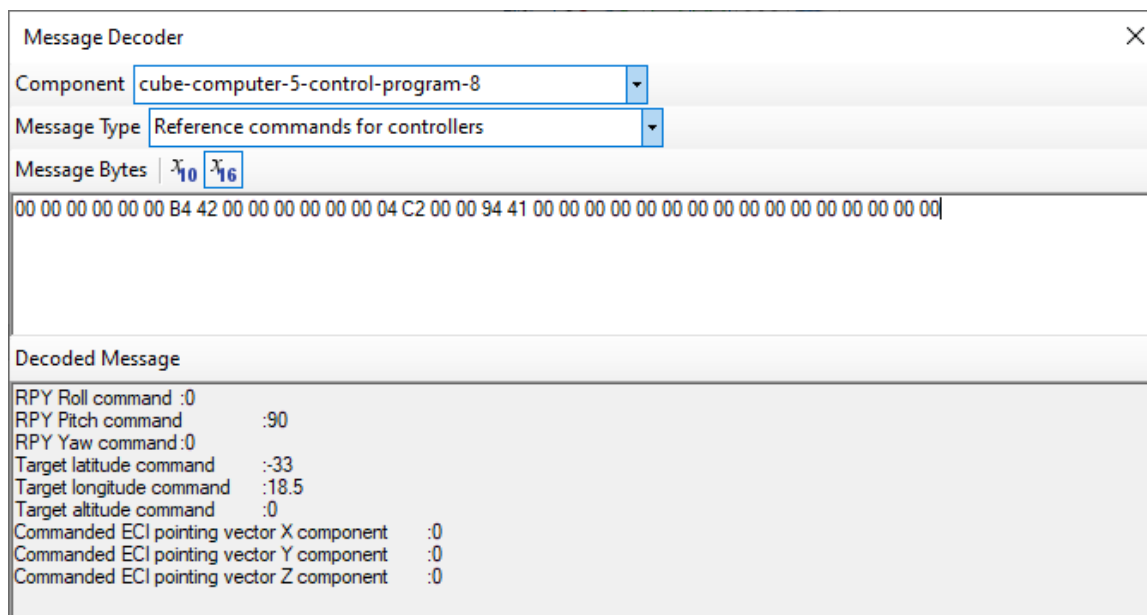


Figure 17: Message Decoder window

4.9 Error Handling



The CubeSupport application uses dialog messages to communicate alerts, errors or processes requiring user intervention. These types of messages are shown in Table 7.

Table 7: Types of message dialogs, what causes them to appear, and the resulting action.

MESSAGE TYPE	ICON	CAUSE	ACTION
Error		An exception occurred during a critical operation or process exited with undefined result.	Dialog displays issue and reason for aborting the current action. Program will resume running after user closes the message dialog, but the attempted action has to be re-attempted after corrective steps.
Warning		A function or operation requires user intervention or a current operation will significantly affect the outcome of a function or process.	Dialog displays, prompting the user for an action. Program will resume with user's command.
Information		A process has completed and the user is being notified of the result.	Dialog displays result with a prompt to close the message dialog.



5 Connecting to Device

Once the CubeSupport application is running, the program can be used to establish a connection to a CubeProduct or integrated CubeADCS. Ensure that the hardware has been set up correctly. More details can be found in the Getting Started section in the CubeADCS User Manual (see [7]), or relevant CubeProduct User Manual.

5.1 Creating a new connection

Create a new connection by clicking the  button as shown in Figure 18. Select the desired communications interface by clicking on the desired item from the drop-down menu.

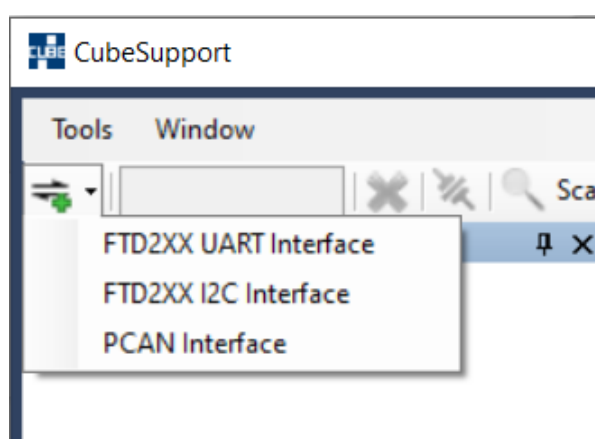


Figure 18: Creating a new Connection in CubeSupport

Once selected, the connection appears in the Connections settings UI region. CubeSupport enumerates each connected device for all the devices found. From the **Serial Device** or **PCAN device** drop-down list, select the appropriate device (in case there are multiple). It may be necessary to press the refresh button , to update the selection, in case a device is plugged in after CubeSupport was started.

Each connection type has a list of setting as shown in Figure 19. These settings can be used to modify connection and communication hardware options. The following sections provide detailed instructions for configuring the connection interfaces.

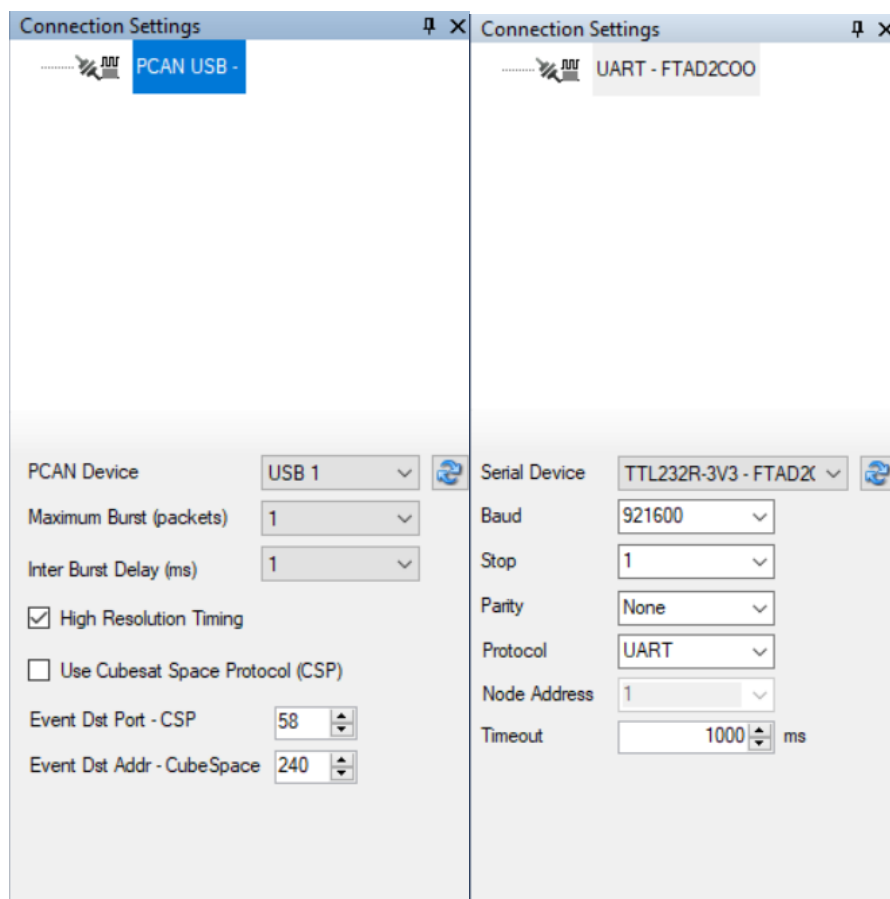


Figure 19: Default connection settings for PCAN (left) and UART (right)

5.2 Connection Setup

5.2.1 CAN setup

CubeSupport supports a PCAN-USB adapter from Peak Systems¹.

The CAN connection is made through the CubeSupport PCB. More information regarding the setup can be found in the Getting Started Section of the CubeADCS User Manual [7] or relevant CubeProduct User Manual.

Ensure that a device id appears in the drop-down menu next to **PCAN Device**. If not, reconnect the PCAN cable and press the refresh button.

5.2.1.1 CSP vs CubeSpace Protocol

By default, CubeProducts use the CubeSpace Protocol which is a transport layer based on CAN V2.0B hardware layers. The protocol allows for multiple CAN frames to be chained to form a single TCTLM message, up to a maximum payload length of 256 bytes. More information can be found in the CAN section of the CubeProduct Firmware Reference Manual [6].

CubeProducts also support the Cubesat Space Protocol (CSP), version 1.6, over CAN. This is a small, connectionless primary/secondary interface protocol which allows for easy communication between

¹ Please use Part no. IPEH-002021, available from <https://www.peak-system.com/>



distributed embedded systems. For the CSP connection option to be used, the CubeProduct must be configured to use CSP. The latter configuration option must be specified when ordering the CubeProduct. CubeSpace must be contacted to change the configuration for a CubeProduct to make use of CSP protocol.

If the CubeProduct that is being connected to is configured to make use of CSP over CAN, tick the **Use Cubesat Space Protocol (CSP)** check box before creating a connection. All other settings should remain at their defaults as shown in Figure 19.

5.2.2 UART

CubeSupport can interface with CubeProducts using UART - either through the CubeSupport PCB or through the CubeADCS directly (Gen1 CubeComputer).

The default connection settings are shown in Table 8. It is possible to configure the UART of the CubeProduct with different settings – this should be specified at time of ordering the CubeProduct. CubeSpace can be contacted to change settings from the defaults.

Table 8: Default UART settings.

PARAMETER	GEN1 CUBEPRODUCT DEFAULT	ALL OTHER CUBEPRODUCTS DEFAULT
Baud Rate (bit/s)	115200	921600
Parity	None	None
Stop Bits	1	1
Timeout (ms)	1000	1000

5.2.3 RS485 setup

CubeSupport supports RS485 connections through the CubeSupport PCB, making use of the protocol described in [6]. In order to connect using RS485, create a UART connection as shown in the previous section and under Protocol, select **RS485** from the drop-down list. The **Node Address** setting must match with the address that the CubeProduct is configured with.

5.2.4 I2C setup

CubeSupport supports I2C communication using an FTDI MPSSE cable. There are however known limitations and issues with this interface, and it is reserved for CubeSpace internal use only.

5.3 Establishing a Connection

CubeSupport establishes a connection by firstly connecting to the physical communications adapter, and then by sending an identification telemetry request to the connected CubeProduct. If the CubeProduct responds as expected, CubeSupport will display UI elements that are relevant to the connected CubeProduct(s).

In the case of I2C and CAN, it is possible to connect more than one CubeProduct on the same bus. CAN and I2C thus makes use of addressing to identify devices on the bus. Each CubeProduct is assigned an 8-bit CAN ID (for CAN interface) or a 7-bit I2C address (for I2C interface). When a connection is established using CAN or I2C, CubeSupport attempts to find multiple devices by sending identification telemetry requests to a range of addresses. The toolbar UI elements for setting the address scan range is shown in Figure 20 below.



Figure 20: Communications ID scan range controls

To initiate a connection (or connections to multiple devices), press the **Connect** button  on the connection toolbar bar. If a CAN or I2C interface was selected, this will initiate an address scan. During the scan, a window appears, as shown in Figure 21, which displays the status of the scan and lists the number of CubeProducts found.

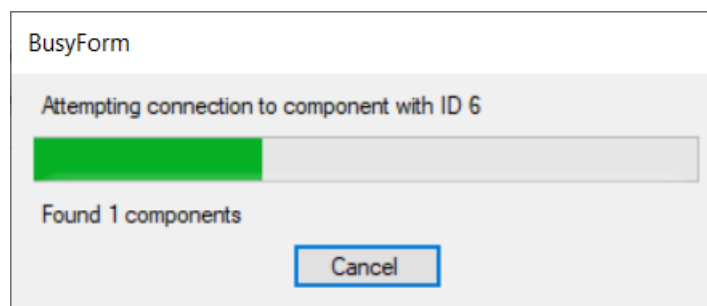


Figure 21: Busy Form while attempting to connect to CubeADCS or CubeProducts.

The Scan can be stopped by pressing the **Cancel** button or by waiting for CubeSupport to finish sweeping through the range.

If a connection was successful (at least one CubeProduct responded to the identification telemetry request), the found CubeProducts appears under the connection interface list in the connection settings UI region as shown in Figure 22.

The Connected functions UI region displays a tab for each CubeProduct found, with UI elements specific to the functionality of that CubeProduct. Section 6 provides further detail on the CubeProduct-specific user interface elements.

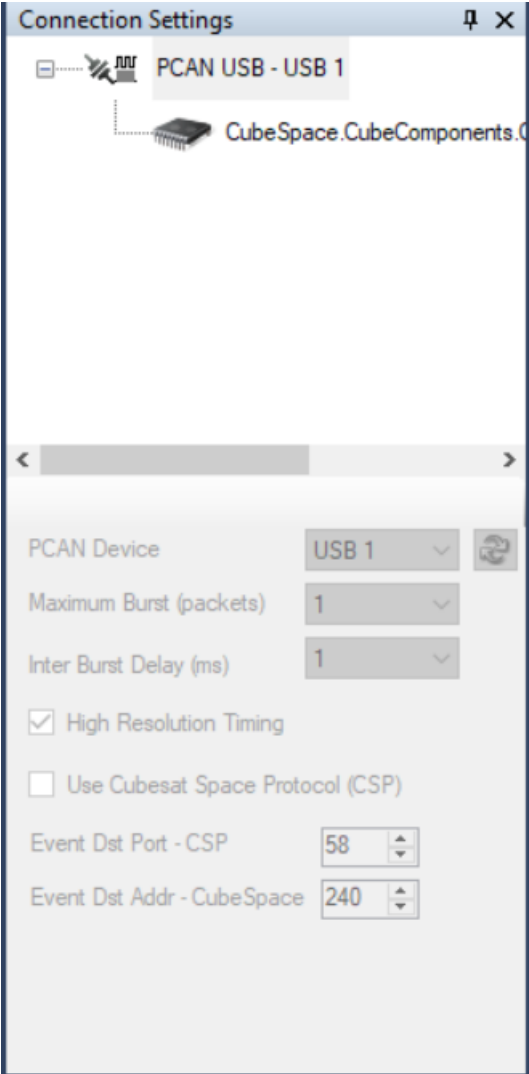


Figure 22: Connection settings UI region after successful PCAN connection.

The Logging and plot UI region towards the bottom of the CubeSupport window shows incoming and outgoing data on the communication link as shown in Figure 23.

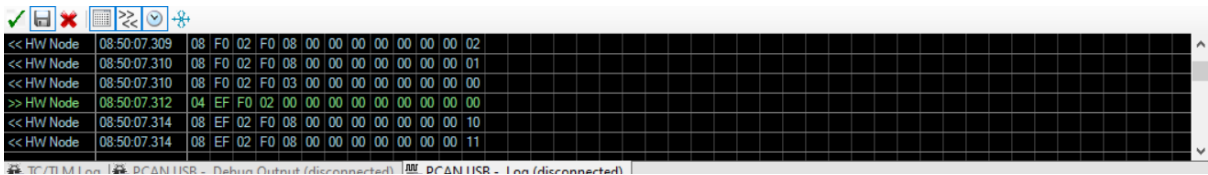


Figure 23: Communications Log



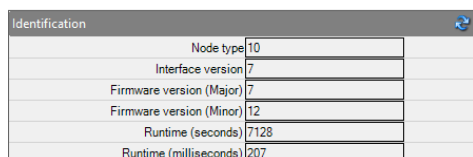
6 CubeProduct-specific User Interface Elements and Features

Each CubeProduct has a specific user interface elements that are presented to the user upon connecting to it, with functionality specific to that CubeProduct. Section 6.1 describes user interface elements that are present when connected to a CubeProduct, but of which the details are common across multiple or all CubeProducts. From section 6.2 onwards, the specific user interface elements per CubeProduct are described.

Note that the information in the following sections should be read in conjunction with the User Manual for the applicable CubeProduct. This CubeSupport HMI User Manual is not intended as a replacement for the CubeProduct's User Manual.

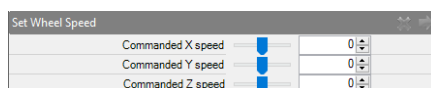
6.1 General telecommand and telemetry actions

CubeSupport will generally display UI elements with settings for telecommands (Figure 26 and Figure 26), and UI elements with return values for telemetry values (Figure 25). The header label for these groups are the display names for the particular telecommand or telemetry frame, with available messages listed in [12]. For communications messages with both set and get access, (telemetry request returns the same data that can be sent), this is shown in the same UI element group (Figure 27).



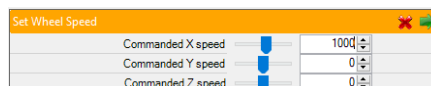
Identification	
Node type	10
Interface version	7
Firmware version (Major)	7
Firmware version (Minor)	12
Runtime (seconds)	7128
Runtime (milliseconds)	207

Figure 25: Example of UI elements for a telemetry request



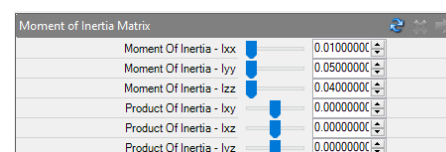
Set Wheel Speed	
Commanded X speed	0
Commanded Y speed	0
Commanded Z speed	0

Figure 24 Telecommand message UI elements



Set Wheel Speed	
Commanded X speed	1000
Commanded Y speed	0
Commanded Z speed	0

Figure 26: Telecommand UI elements (with unsent parameter changes)



Moment of Inertia Matrix	
Moment Of Inertia - box	0.01000000
Moment Of Inertia - lyy	0.05000000
Moment Of Inertia - lzz	0.04000000
Product Of Inertia - bxy	0.00000000
Product Of Inertia - bxz	0.00000000
Product Of Inertia - lyz	0.00000000

Figure 27: Example of combined telecommand and telemetry UI elements

6.1.1 Requesting telemetry

The request button in the header of the telemetry request header is denoted by the  symbol. Clicking on this button will send a single request to the CubeProduct for the relevant telemetry, and the fields in the box will be populated subsequently.

6.1.2 Sending a telecommand

Prior to sending a command, the parameters in the telecommand UI elements must be populated with the desired values. The header label above the UI elements will turn orange to show that changes were made and not yet transmitted, as in Figure 26.

The transmit button in the telecommand UI element message header is denoted by the  symbol. Clicking on this button will send the relevant telecommand to the CubeProduct once.



The UI elements for individual parameters are generated from the API [12]. Depending on the type of the parameter, the following UI element is used:

- a) Integer/FloatingPoint: Numeric up-down editor + Slider
- b) Enumeration: Drop down list
- c) Boolean: Check box

Note: fields of type “ByteArray” are not displayed. This is intended, as these TCTLM's require programmatic-based communication.

6.1.3 Combined telemetry and telecommand box

Communication messages with both get and set access will display both Refresh () and Send () buttons. The actions combine the behaviour of both telecommands and telemetry.

This type of message is often used for configuration settings. In this case, it is useful to first request the current settings, in order to modify only some of the parameters prior to sending the telecommand, to set the new configuration.

6.1.4 Special user interface elements for commands or telemetry

6.1.4.1 Unix time set/get

The UI element to retrieve Unix time from a CubeProduct displays the Unix time as a date and time of day (UTC) because it is more convenient to work with, and to interpret than the actual Unix seconds.



Figure 28: Unix time command/telemetry user interface element

The current Unix time is retrieved and adjusted in the same way as requesting telemetry or sending a normal telecommand respectively. An additional button is available on the message header, , to synchronise the CubeProduct's time to that of the PC running CubeSupport.

6.1.4.2 Orbit parameters

The messages to set or retrieve orbit parameters from the CubeComputer control program has a convenient text box at the bottom of the Orbit Parameters UI elements, that can be used to input or display the orbit parameters as a Two-Line Element set, as can be seen in Figure 29.



SGP4 Orbit Parameters			
Inclination		51.640000C	
Eccentricity		0.0001302C	
Right-ascension of the Ascending Node		125.24500C	
Argument of Perigee		154.14580C	
B-Star drag term		0.0011098C	
Mean Motion		15.4986013	
Mean Anomaly		11.515400C	
Epoch		23209.2159	
1 25544U 98067A 23209.21590000 .00063071 00000-0 11098-2 0 9991			
2 25544 051.6400 125.2450 0001302 154.1458 011.5154 15.49860133408156			

Figure 29: Orbit Parameters message with TLE formatting

If TLEs are pasted (TLEs for tracked objects can be obtained from celestrak.org, or space-track.org) into this text box, the orbital parameter fields are automatically updated to that of the supplied TLE. If the orbital element fields are modified, the TLE text box automatically updates to correspond to the provided orbital elements.

6.1.5 TCTLM groups

TCTLM message UI elements are grouped under TCTLM group headers – coloured labels with drop-down buttons to the right. Grouped TCTLM UI elements contain messages with related functionality. These groups can be collapsed for easier navigation to the target TCTLM UI elements, as shown below in Figure 30.

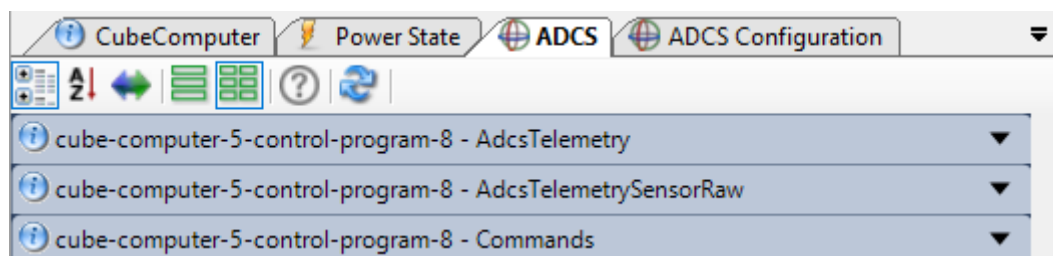


Figure 30: Collapsed ADCS tab group headings, with the Collapse icon indicated.

6.2 High-level UI tabs vs. automatically generated TCTLM tabs

The functionality that is made available through the CubeSupport HMI for a connected CubeProduct is organized into different tabs that appear when the connection to the CubeProduct is established. As stated earlier, these tabs are unique to the CubeProduct. A distinction can be made between tabs with more specific high-level function that applies to that CubeProduct, vs. tabs that contain “plain” UI elements for sending single telecommands and displaying telemetry.

The latter tabs contain UI elements that are automatically generated based on the communications interface definition (nodedef) for that CubeProduct [12]. Tabs with high-level functionality combine sequences of telecommands and telemetry with UI elements to provide a more intuitive or convenient UI experience.

An example of a auto-generated TCTLM UI element tab compared to a high-level function tab is shown in Figure 31.

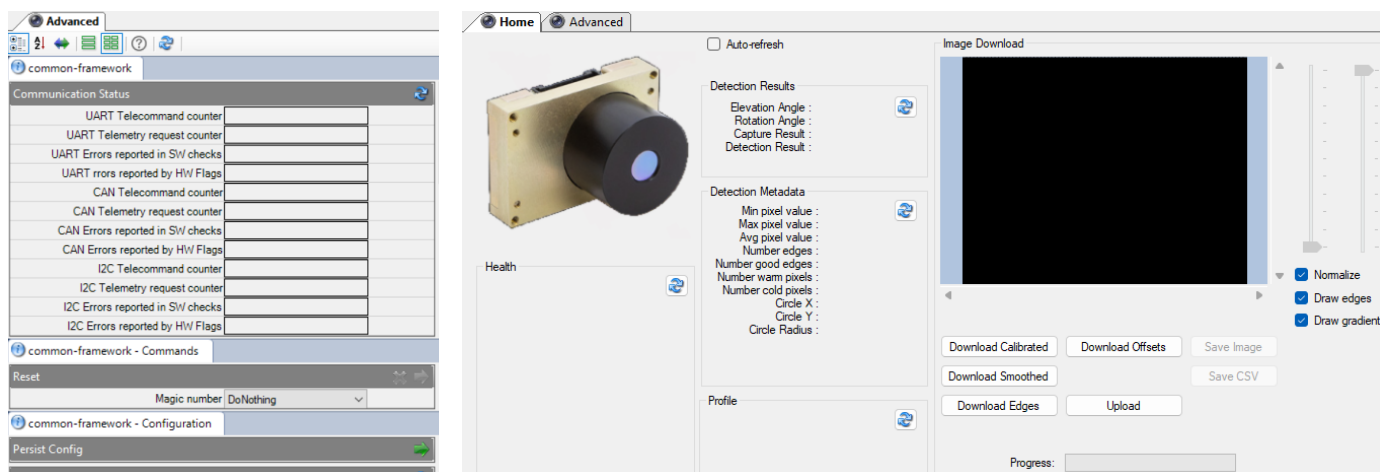


Figure 31: Tab with auto-generated TCTLM UI elements (left) vs. high-level functionality (right)

6.2.1 Gen2 Common-Framework auto-generated UI elements

All Gen2 CubeProducts implement the TCTLM in the common-framework node definition [12]. As a result, all CubeProducts will display automatically generated TCTLM UI elements for command-framework commands and telemetries.



Although all Generation 2 CubeProducts implement the common-framework TCTLM, the UI handling in CubeSupport is not consistent. In some cases, common-framework UI elements appear under an “Advanced” tab, while in other cases they are combined with other TCTLM unique to that CubeProduct

The common-framework UI elements exposes the following telemetry:

- Identification
- Version
- Serial number
- Boot Status
- Common Error Codes
- Communication Status

In addition, it also provides telecommands and telemetry to query the error log, persist configuration changes, and perform a reset.

The CubeProduct’s Unix time can be requested or adjusted using the common-framework TCTLM, in the same way as described in 6.1.4.1.

6.3 Gen1 Integrated CubeADCS

6.3.1 Bootloader

When connecting to a Gen1 CubeComputer while the bootloader is running, and the bootloader respond to the identification telemetry request, the user interface in Figure 32 is displayed.

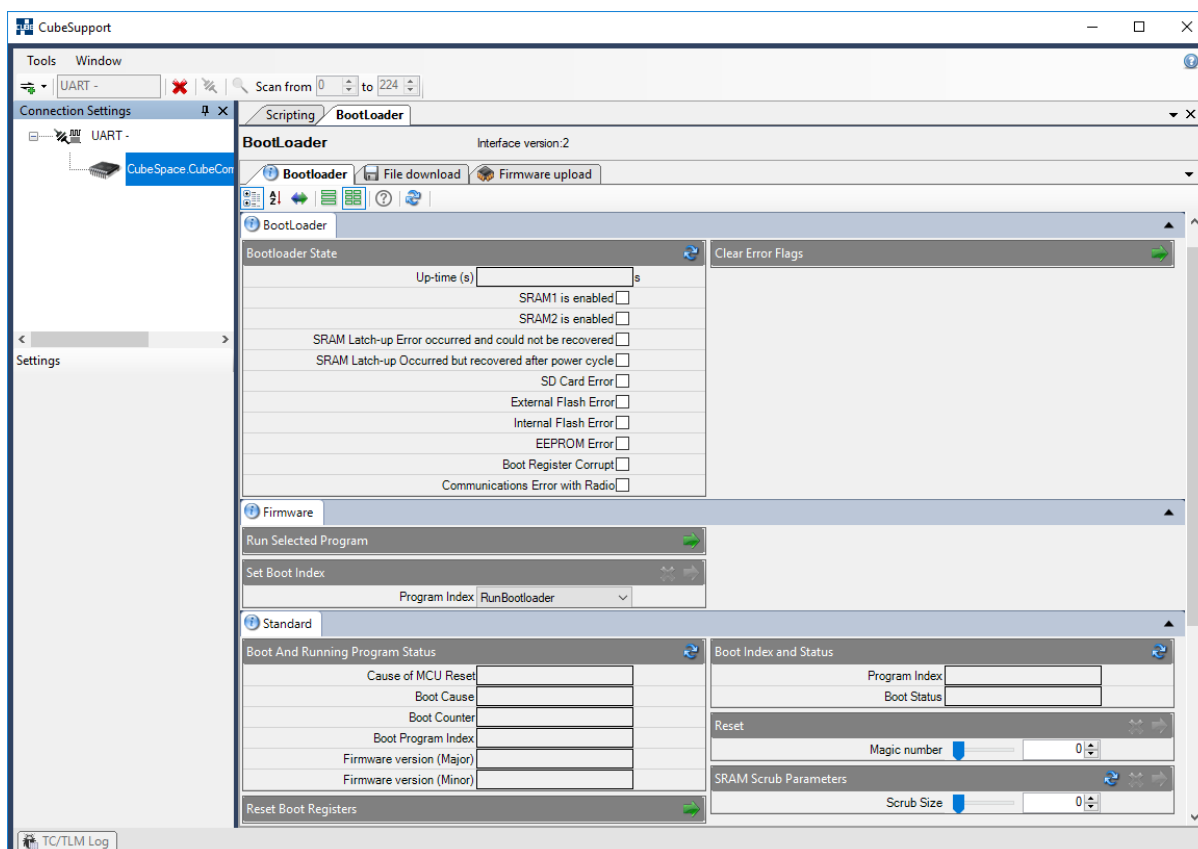


Figure 32: Gen1 CubeComputer bootloader user interface

The UI for the bootloader application has several tabs running along the top of the Connected functions UI region.

Bootloader	Houses UI elements to retrieve and set the bootloader settings
File Download	Allows the user to initiate file downloads from the SD card
Firmware Upload	Uploading new firmware

6.3.1.1 Uploading new control program

Using the CubeSupport, it is possible to program a new application (control program) (new version) to the CubeADCS CubeComputer. To place a new program into flash memory, perform the following steps, which are aided with illustrations:

1. Power cycle the CubeComputer or, if the control program is currently running (see section 6.3.2) perform a reset from the ADCS application by setting the **Magic number**, found in the **CubeACP** tab, to 90 (Figure 33). Click the transmit button (green arrow). Subsequently, the user will receive a timeout error message, indicating the ADCS has reset.



Figure 33: CubeComputer Reset telecommand

2. Wait for two to four seconds



3. Connect to the CubeComputer. The CubeSupport user interface should display the bootloader interface as shown in Figure 32.
4. Go to the **Firmware Upload** tab.
5. Press the **Refresh** button (↻). This retrieves the current applications available within flash memory.
6. Select the External Flash position where the new firmware needs to be placed and press the **erase file segment** button (✖) to remove the current content.
7. In the **Local File System** region browse to the location of the new binary. Select the binary along with the External Flash position. An **Upload** button (⏮ Upload) will appear.
8. Press the **Upload** button. This will initiate a file upload procedure. Wait until the **File Synchronization Status** indicates **Local and remote files match**.
9. Select the new file within the External Flash position and select the **Copy into Internal Flash** button (📄).
10. After this operation is finished, return to the **Bootloader** tab. In the **Set Boot Index** box select the **RunInternalFlashProgram** as the program index (Figure 34) and press the transmit button.

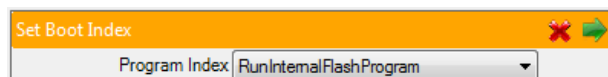


Figure 34: Set Boot Index telecommand

11. Power cycle the CubeComputer or perform a reset from the Bootloader application by setting the **Magic number** to 90 and transmitting.
12. Wait at least five seconds before connecting to the CubeComputer UART using CubeSupport. Verify that the CubeSupport user interface is set up for the ADCS application.

6.3.1.2 File downloads

File downloads through the Gen1 CubeComputer bootloader is handled in the same way as through the control program. Please see section 6.3.2.6 for details.

6.3.2 Control Program

If the control program is allowed to start (5 seconds or longer after reset or power-on) before attempting to connect using CubeSupport, the connected functions UI region will initialise to display the CubeComputer control program (also called CubeACP) UI elements.

The UI elements for the CubeACP application has several tabs running along the top of the Connected functions UI region. Figure 35 displays the CubeACP application specific UI elements.

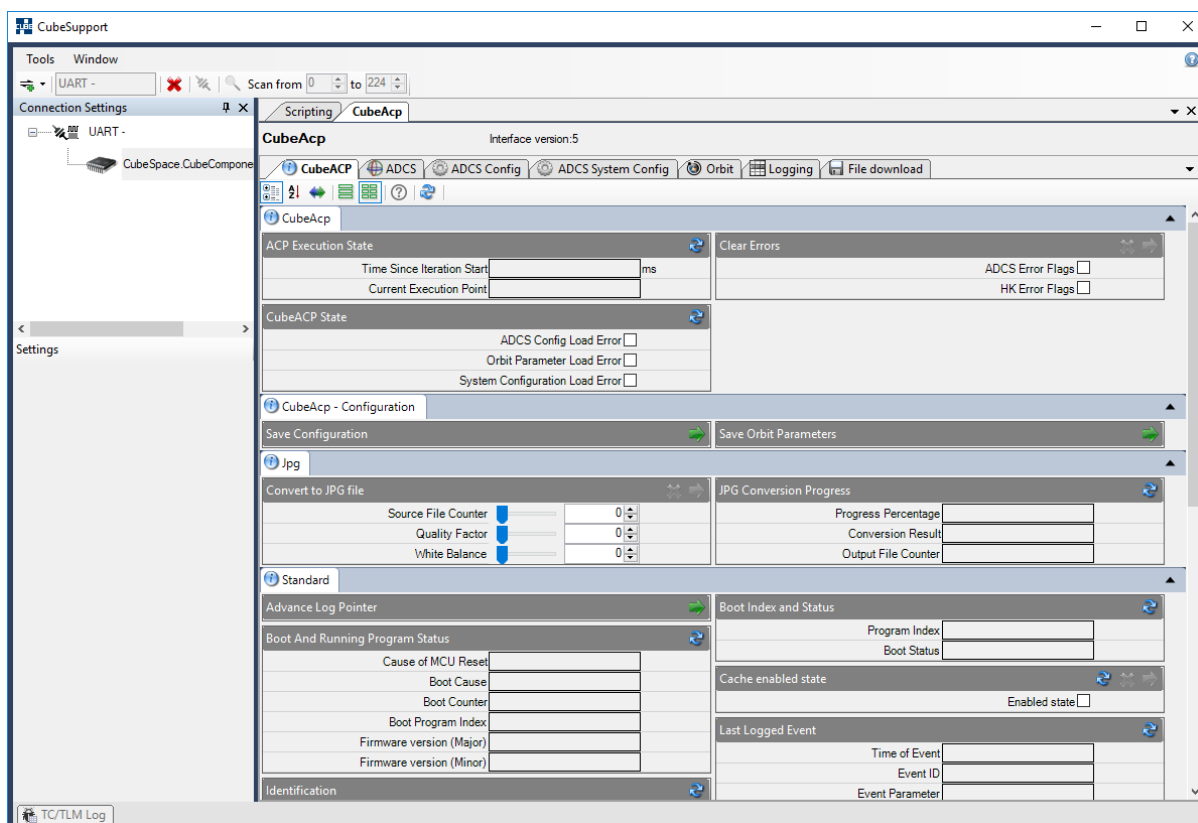


Figure 35: Gen1 CubeComputer control program UI.

The UI elements for the CubeACP application has several tabs running along the top.

CubeACP

Allows the user to obtain and set housekeeping and timing information.

ADCS

Contains all the UI elements to

- Change estimation and control modes
- Request estimated state, and actuator commands
- Request raw and calibrated sensor measurements
- Request current, temperature and power measurements

ADCS

Configuration

Allows the user to request and change the

- Mounting angles and orientations of all sensors and actuators
- Satellite moment of inertia
- Controller gains
- Estimator parameters
- Calibration values for CubeSense, CubeStar and rate sensor offsets

ADCS System Config

Allows changing the system configuration for the CubeADCS. These settings should normally not be changed.

Orbit

Allows the user to request and change the orbit parameters that the ADCS uses to determine its position

Logging

Allows the user to setup logging by the ADCS application on the on-board SD card or through the UART channel

File Download

Allows the user to initiate file downloads from the SD card



The ADCS specific functionality of the module is exercised through the **ADCS** tab. The Gen1 CubeADCS User Manual [1] and ICD [2] should be consulted for detailed information about the ADCS functions.

6.3.2.1 Mode selection

The **ADCS Mode** group of TCTLM UI elements allows the user to place the ADCS in a specific estimation and control state. It also allows the user to activate or de-activate the ADCS processing loop using the **ADCS Run Mode** setting displayed in Figure 36. For any ADCS actions to be performed or telemetry to be returned, the module has to be in the **AdcsEnabled** run mode.

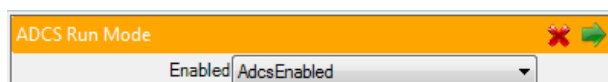


Figure 36: Setting of ADCS run mode.

Displayed in Figure 37 is the interface for the setting of attitude control and estimation mode. Certain transitions of estimation and control modes will not be allowed. See [2] for more detail.

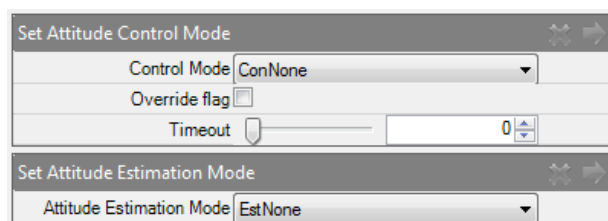


Figure 37: Setting of attitude control and estimation mode.

6.3.2.2 Sub-system power selection

The power to the sub-systems can be controlled by the **ADCS Power Control** TCTLM UI elements, as displayed in Figure 38.

ADCS Power Control		
CubeControl Signal Power Selection	PowOff	▼
CubeControl Motor Power Selection	PowOff	▼
CubeSense1 Power Selection	PowOff	▼
CubeSense2 Power Selection	PowOff	▼
CubeStarPower Power Selection	PowOff	▼
CubeWheel1Power Power Selection	PowOff	▼
CubeWheel2Power Power Selection	PowOff	▼
CubeWheel3Power Power Selection	PowOff	▼
Motor Power	PowOff	▼
GPS Power	PowOff	▼

Figure 38: Power selection of sub-systems.

6.3.2.3 ADCS Commands

6.3.2.3.1 Actuator commands

It is possible to send actuator commands to the ADCS through the CubeSupport UI, as portrayed in Figure 39. This is done from the **Actuator Commands** group.



Set Magnetorquer Output		
Commanded X Magnetorquer	<input type="range"/>	0.000
Commanded Y Magnetorquer	<input type="range"/>	0.000
Commanded Z Magnetorquer	<input type="range"/>	0.000

Set Wheel Speed		
Commanded X speed	<input type="range"/>	0
Commanded Y speed	<input type="range"/>	0
Commanded Z speed	<input type="range"/>	0

Figure 39: Actuator commands.

For the ADCS unit to act on these commands, the run mode has to be set to **AdcsEnabled**, and the control mode must be **None**. Power to the CubeControl MCUs and CubeWheels must be set to **PowOn**.

6.3.2.3.2 Reference attitude

The ADCS unit allows for the pitch, roll and yaw angles to be controlled to a reference value if the module is in the correct control mode. This reference value can be specified through the CubeSupport UI, as displayed in Figure 40.

Set Commanded Attitude Angles		
Commanded Roll Angle	<input type="range"/>	0.00
Commanded Pitch Angle	<input type="range"/>	0.00
Commanded Yaw Angle	<input type="range"/>	0.00

Figure 40: Setting of the reference attitude.

6.3.2.3.3 Magnetometer deployment



This function is provided to test the magnetometer deployment. Activating the magnetometer boom deployment while the magnetometer is connected to the ADCS unit will cause the magnetometer to deploy!

The magnetometer boom deployment can be activated through the CubeSupport UI, found in Figure 41, by sending the **Deploy Magnetometer Boom** command after selecting an appropriate time-out.

Deploy Magnetometer Boom	
Timeout	<input type="range"/>

Figure 41: Deployment of magnetometer command.

6.3.2.3.4 Other commands

Various other commands are also available and are mostly available at the top of the ADCS tab. Refer to [1] for other ADCS related telecommands.

6.3.2.4 Configuration

The configuration that the ACP program uses, is made up of three parts:

1. System Configuration
2. ADCS User-adjustable Configuration
3. Orbital Elements



6.3.2.4.1 System Configuration

The *System Configuration* is determined by CubeSpace and is used to configure the software based on hardware options (such as which components are present in the CubeADCS, enable line selection etc.). The System Configuration is typically programmed onto the CubeADCS before it is shipped. Only in exceptional circumstances (and in consultation with CubeSpace) should the System Configuration be reprogrammed.



Changes to the System Configuration should only be made in consultation with CubeSpace. Incorrect System Configuration settings will result in malfunction of the CubeADCS.

System Configuration settings are changed from the **ADCS System Config** tab in the connected functions UI region.

After setting the System Configuration, the CubeComputer has to be reset.

6.3.2.4.2 ADCS User-adjustable Configuration

The user-adjustable configuration settings are requested or changed from the **ADCS Config** tab.

These settings include:

- Sensor and actuator mounting angles and axes alignment,
- Satellite moment of inertia,
- Calibration values for CubeSense and CubeStar, and rate sensors,
- Magnetometer in-flight calibration values and
- Controller gains and estimation parameters.

These settings are determined prior to launch, and possibly adjusted during in-orbit commissioning. CubeSpace provides a service whereby the user's configuration is checked, based on inputs by the user, both prior to launch and during commissioning.

CubeSpace will typically provide the ADCS configuration as an XML file, that can be loaded from the CubeSupport User Interface by pressing the **Open** button on the toolbar in the **ADCS Config** tab.

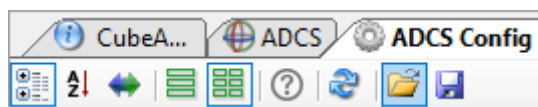


Figure 42: Open button highlighted, to load XML configuration files provided by CubeSpace

The ADCS configuration structure is large with numerous settings, but it is possible to get and set smaller groups of parameters from the CubeSupport interface by using the grouped UI elements.

Changes to the configuration are immediately acted on in the ACP program (without the need for a reset), but they are not automatically committed to flash memory. It is thus possible to try out new settings without changing the defaults stored in flash memory. In order to write the current configuration to flash memory, it is necessary to send the **Save Configuration** command (also available from the **ADCS Config** tab)



Figure 43: Save Configuration command, to persist configuration changes to flash memory

Further details about all configuration settings can be found in [1].

6.3.2.4.3 Orbit Elements

The orbital elements that are requested or changed from this tab, are used internal to the ADCS to determine the current position in the orbit, using a conventional SGP4 orbit propagator.

Set SGP4 Orbit Argument of Perigee	Set SGP4 Orbit B-Star Drag term
Argument of Perigee 0.000000000000	B-Star drag term 0.000000
Set SGP4 Orbit Eccentricity	Set SGP4 Orbit Epoch
Eccentricity 0.000000	Epoch 0.0
Set SGP4 Orbit Inclination	Set SGP4 Orbit Mean Anomaly
Inclination 0.000000000000	Mean Anomaly 0.000000000000
Set SGP4 Orbit Mean Motion	Set SGP4 Orbit RAAN
Mean Motion 0.000000	Right-ascension of the Asc 0.000000000000
Set SGP4 Orbit Parameters	
Inclination 0.000000000000	
Eccentricity 0.000000	
Right-ascension of the Asc 0.000000000000	
Argument of Perigee 0.000000000000	
B-Star drag term 0.000000	
Mean Motion 0.000000	
Mean Anomaly 0.000000000000	
Epoch 0.0	

Figure 44: Orbit configuration.

After setting the parameters, these values can be saved into flash memory by sending the **Save Orbit Parameters** telecommand in the **CubeACP** tab, as displayed in Figure 45.



Figure 45: Saving orbit parameters in flash.

6.3.2.5 Images and Logs

The integrated Generation1 CubeADCS has the ability to capture images from connected CubeSense and CubeStar sensors. The CubeSupport UI allows the user to initiate such image captures as well as log telemetry internally and save it to the SD card on the CubeComputer. Image files and TLM log files can then be downloaded to the PC.

6.3.2.5.1 Images

To initiate a new image capture, ensure that either the CubeStar or a CubeSense is switched on. Under the **ADCS** tab a new capture can be initiated by sending a **Save Image** command. Select the camera and the size of the image – these UI elements are displayed in Figure 46. The image size is a numbered list where **Size0** is the largest resolution image and **Size4** the smallest.



Save Image	
Camera Select	CamCam1
Image Size	Size0

Figure 46: Image capture and save command.

The save image process is broken down into a capture step and a download step to follow. The capture step will take a number of seconds before the downloading step starts. The progress of the downloading step can be obtained by requesting the **Status of Image Capture and Save Operation** telemetry, as seen in Figure 47.

Status of Image Capture and Save Operation	
Percentage Complete	s
Status	s

Figure 47: Save status.

After the percentage complete reaches 100, the image download is complete and the image is saved on the SD card of the CubeComputer.

6.3.2.5.2 TLM log files

The ADCS can log telemetry to the SD card for subsequent download, and to the UART for telemetry sampling by an external OBC. Logging can be initiated through the CubeSupport UI under the **Logging** tab. The selection is performed by a series of checkboxes, as seen in Figure 48. The **Log Period** is adjusted through the up-down selection. After the settings have been transmitted to the ADCS application, the logging will commence. To stop the logging process, simply transmit a logging option with a zero second **Log Period**.

If the SD card logging is selected, a log file is automatically created on the on-board SD card. In the case of UART logging the log file is also automatically created and available in the directory of the CubeSupport application.

SD Log1 Configuration	
Log Period	0
Log Destination	SdLogPrimary
Communication Status	☐
EDAC Error Counters	☐
Boot And Running Program	☐
Last Logged Event	☐
SRAM Latchup counters	☐
Current Unix Time	☐
Magnetorquer Command	☐
Wheel Speed Commands	☐
Magnetic Field Vector	☐
Coarse Sun Vector	☐
Fine Sun Vector	☐

Figure 48: Initiate logging command.

6.3.2.6 SD card file management

Files on the SD card are managed under the **File download** tab. To get the current list of files on the SD card, press the  button. The list will update to show all the available files. To perform an operation on a file, first select the row with the relevant file. To download the selected file to PC, select the root in the



Local File System UI region, as seen on the right-hand side of Figure 49. After pressing the **Download >>** button, the final path to where the file should be saved and the filename can be selected.

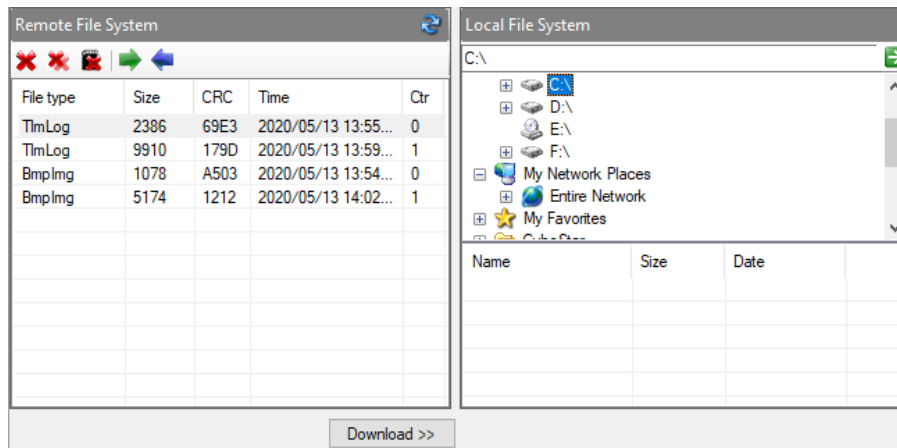


Figure 49: File download from SD card.

A progress UI element will be displayed below the Remote and Local File System UI regions, indicating the progress of the transfer. The download process is complete when the progress UI element is replaced by a message that the local and remote file are identical. In case of an error during the transfer, a window with appropriate message is displayed.

To delete a file from the SD card, press the  button. To delete all files from the SD card, press the  button. To format the SD card, press the  button.



Take care when deleting files or formatting the SD card, since the action cannot be undone – files are removed permanently from the SD card storage. CubeSupport will not display a prompt before deleting files.



6.4 Gen2 Bootloader

The user interface elements for the Gen2 bootloader – the bootloader that is present on all Gen2 CubeProducts – is the same.. Upon connection to the bootloader on any Generation2 CubeProduct, CubeSupport populates tabs for [Firmware/Config Upload](#), [Raw Memory Access](#) and [Advanced](#).

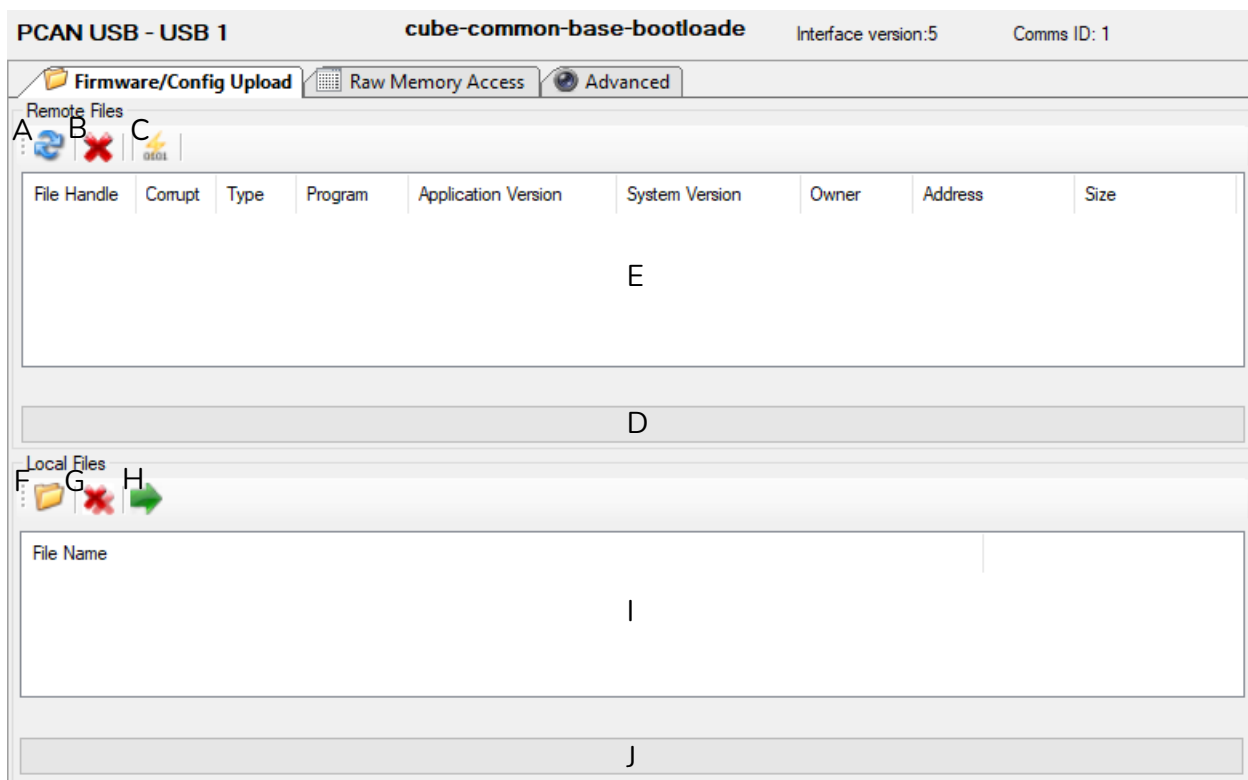


Figure 50: Bootloader tabs and Firmware/Config upload UI elements

6.4.1 Firmware/Config Upload Tab

The “Firmware/Config” tab is shown in Figure 50. The main capabilities are as follows:

Remote Files UI region:

- A) Refresh button: displays all the firmware files stored in the non-volatile storage memory of the CubeProduct.
- B) Erase button: erases the selected file from non-volatile storage memory (cannot be undone).
- C) Jump to App button: Instructs the CubeComputer bootloader to run the application program. If this operation is successful, the CubeSupport user interface will automatically reconfigure to show the connected functions for the CubeComputer control program.
- D) Progress bar: displays progress of ongoing transactions.
- E) Remote Filesystem table: shows information of files in the non-volatile storage memory on the connected CubeProduct.

Local Files UI region:

- F) Open File button: opens a binary (*.bin.cs) or configuration (*.cfg-bin.cs) file from the host PC.
- G) Clear Item button: removes the selected file from the local filesystem list.



- H) Upload button: uploads a file from the host PC to the non-volatile storage memory of the connected CubeProduct.
- I) Local Filesystem table.
- J) Local Filesystem progress bar.

6.4.1.1 Local files

The local files UI region is used as a staging area for files intended to be uploaded to the CubeProduct. The user can stage files from the local PC, view file information and clear unwanted files. Once staged, the files that the user wants to upload can be selected and then uploaded. The progress bar at the bottom of the group shows the progress of the upload of each individual file. When the batch of uploads is complete a message window appears indicating a successful upload of all files, or individual message window(s) for each failed upload.

The file list is automatically refreshed after every batch of file uploads.

6.4.1.2 Remote Files

The Remote Filesystem view shows the files on the connected CubeProduct. The user can use this UI region to see information about files in non-volatile storage memory and erase files.

6.4.1.2.1 Request file info

Click the Refresh button [A] to request file information from the remote filesystem. The progress bar will indicate a busy state until all file information is retrieved. The returned information will be displayed in the Remote Filesystem table [E].

6.4.1.2.2 Erase File

Select the desired files from the Remote Filesystem area [E]. Next, click the Erase button [B]. A message dialog then prompts the user for confirmation. The erase process is instantaneous and returns a success message on successful completion and the remote filesystem area will be updated via an additional file info request. If an error occurred during the operation, a window is displayed with text description of the problem, and optionally an error code for further diagnostic purposes.

6.4.1.3 Jump to App

After selecting an application from the Remove Files table, the user can click the Jump button [C] to start the selected application.

6.4.2 Raw Memory Access tab

The Raw Memory Access tab can be used to retrieve data from the CubeProduct flash memory directly, or to write to it. The tab contents are shown in Figure 51.

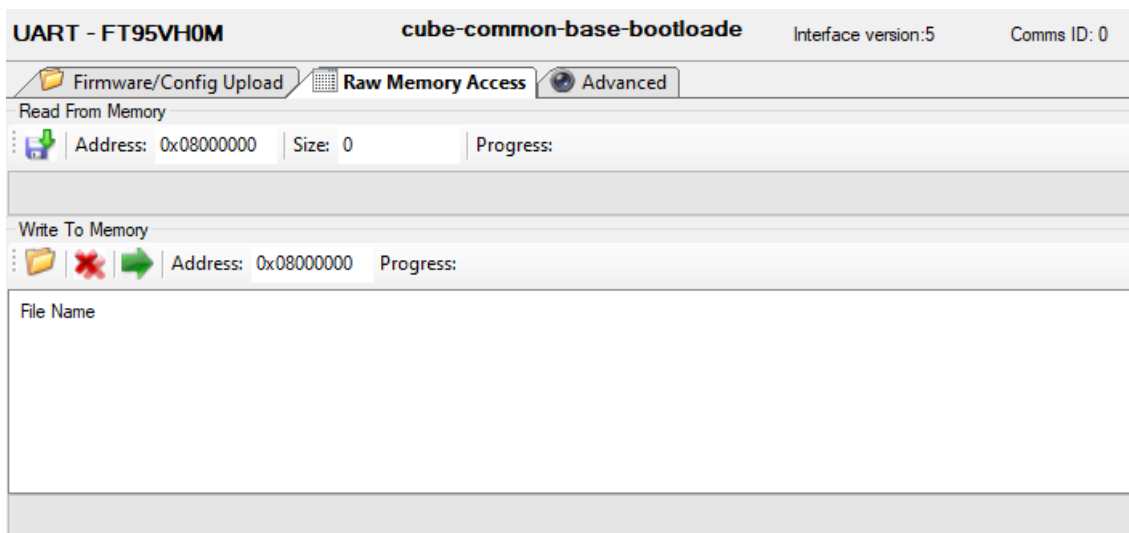


Figure 51: Bootloader Raw Memory Access tab

6.4.2.1 Read from memory

To read from flash memory directly, first specify the memory address and size in the Read from Memory toolbar, followed by the Save button (📁➡). The user is prompted for a destination file before the data is stored.

6.4.2.2 Write to memory

To write a binary file directly to flash memory, press the Open button (📁) and select the file from the file selection window. The file will appear in the Files list.

Specify the destination address in the Write to Memory toolbar, and press the Upload button (➡).

The file list can be cleared by pressing the Clear button (✖).

6.4.3 Advanced Tab

The “Advanced” tab is populated with automatically generated TCTLM UI elements for all telecommands and telemetry available to the Gen2 bootloader. Note that the Gen2 bootloader does not implement the common-framework TCTLM, however as far as possible commonality does exist (see [4] for detail).

6.5 Gen2 Integrated ADCS Control Program

The user interface for the Gen2 CubeADCS control program displays the following tabs:

Table 9: Gen2 CubeComputer Control Program tabs

TAB NAME	FUNCTIONS
Port Map	User interface elements to identify which CubeProducts (nodes) are connected to the CubeComputer, and initiate the auto-discovery process.
CubeComputer	User interface elements to send telecommands and request telemetry related to general CubeComputer housekeeping, and common infrastructure such as synchronising Unix time, version and identification telemetry and reset commands. This tab is also where unsolicited telemetry output is enabled



TAB NAME	FUNCTIONS
Power State	Houses commands to set power state of connected nodes, and request current power state
ADCS	User interface elements for full control of attitude estimation and control functions. Also reports sensor and actuator telemetry
ADCS Configuration	This tab contains UI elements for all command and telemetry pairs that can be used to change the control program configuration settings
Health	Telemetry UI elements for CubeComputer and connected nodes' health (temperature, current and voltage measurements)
File Transfer/Upgrades	
Telemetry Log	High-level interface to request logged telemetry, filtered by date, time and/or type
Event Log	High-level interface to request logged events, filtered by date, time and/or type
Image Log	High-level interface to request images from optical sensors connected to the CubeADCS
PassThrough	High-level interface to send telecommands or request telemetry direct to connected nodes, through the CubeComputer communications interface

The rest of the CubeComputer UI functionality is described in the following subsections.

6.5.1 Port Map Tab

The “Port Map” tab is shown in Figure 52. The layout is as follows:

- A) The left pane shows the discovered nodes on their respective ports.
- B) The Refresh button performs a new Auto-Discovery
- C) The port labels show the status of each port. Hovering the mouse over a label shows the information of the node connected to the port, as well as error information if applicable. The colour of the labels will change depending on the results of Auto-Discovery:
 - a. Green – Auto-Discovery completed successfully, and a node has successfully been discovered on this port.
 - b. Orange – Auto-Discovery was unsuccessful, however the node on this port was successfully discovered and was expected.
 - c. Red – Auto-Discovery was unsuccessful, or the node on this port has experienced an error.
 - d. Transparent – No communication was achieved on this port.

Regarding use of colours for the port labels: if any node could not be discovered, the auto-discovery process is deemed unsuccessful. In such a situation, the particular node(s) that could not be discovered are drawn with red labels and all nodes that are subsequently discovered successfully have orange labels. Only if auto-discovery as a whole was successful will ports have green labels.

- D) The right pane shows the expected nodes. The entries will change colour depending on the results of Auto-Discovery:
 - a. Green – A node has been discovered that matches this expected node.
 - b. Red – The expected node has not been discovered.

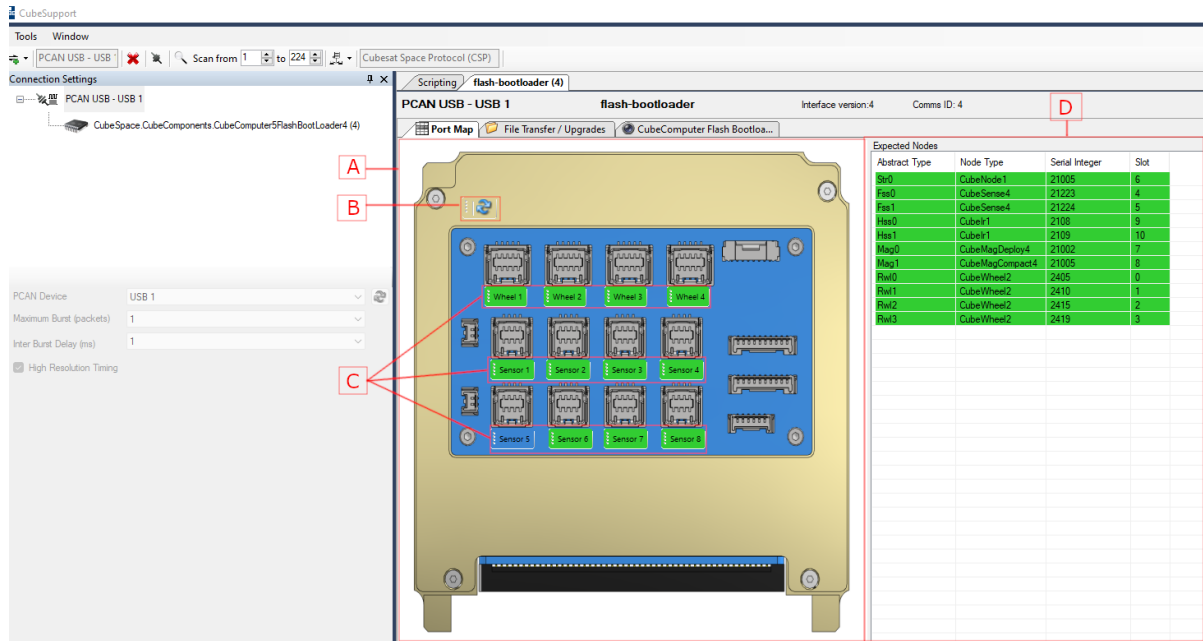


Figure 52: CubeSupport Port Map Successful Auto-Discovery

Figure 52 shows a successful Auto-Discovery, with all expected nodes present. Figure 53 shows the Auto-Discovery results for the same setup but after the sensor on port Sensor 3 was removed.

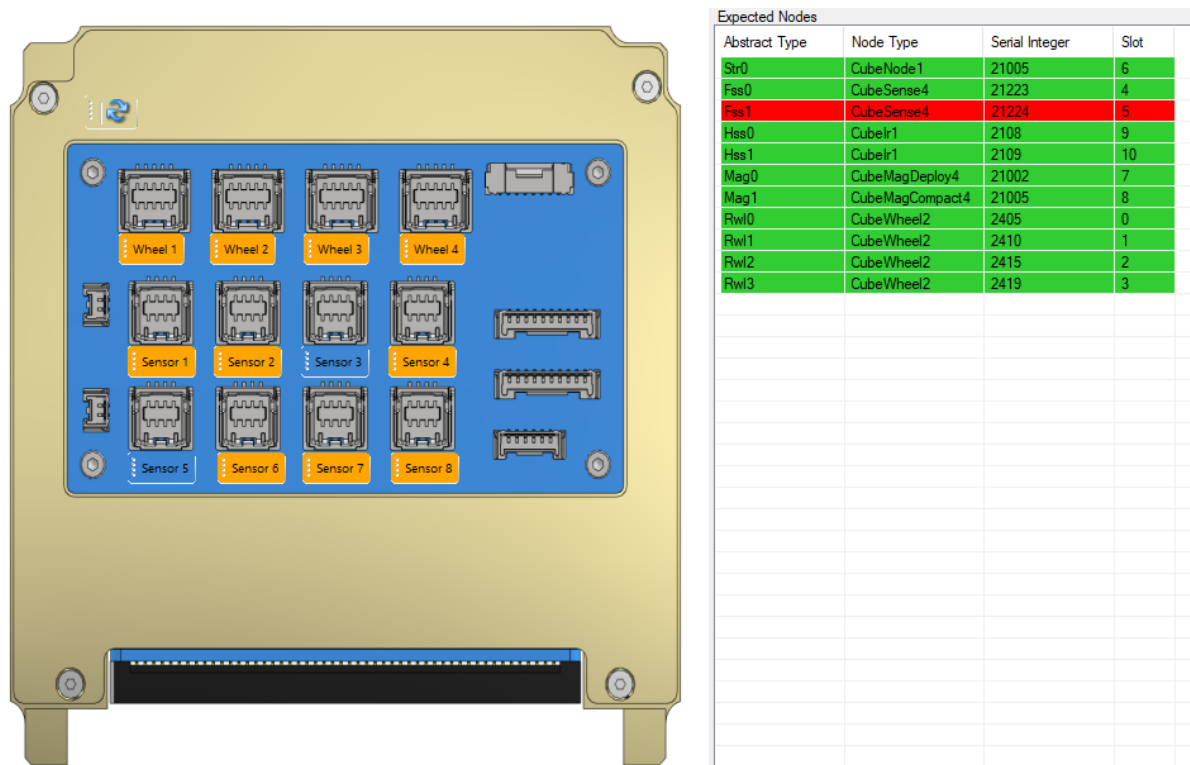


Figure 53: CubeSupport Unsuccessful Auto-Discovery.



6.5.2 CubeComputer tab

The CubeComputer tab contains command and telemetry UI elements for all of the common framework TCTLM. This section is useful for obtaining version information of the firmware programmed onto the CubeComputer, as well as its serial number. Status telemetry regarding communications interfaces and the boot process is also available from this tab.

The CubeComputer Unix time can be requested or adjusted from this tab, under the `common-framework - Time` heading in the same way as described in 6.1.4.1.

The CubeComputer tab displays the common-framework TCTLM UI elements (see 6.2.1), as well as cube-computer-common TCTLM. (see [12] for details on common-framework and cube-computer-common TCTLM).

The result of port auto-discovery and diagnostic errors can be requested under the `cube-computer-common - Port` heading.

6.5.2.1 Unsolicited telemetry and event log output

The UI elements under the `cube-computer-common - Logging` heading allows the user to enable unsolicited log outputs. For unsolicited telemetry logging, the `Unsolicited Telemetry Message Setup` control in Figure 54 is used.



Unsolicited Telemetry Message Setup

UART Config

interval2s

- ☐ Calibrated STR sensor telemetry
- ☐ Calibrated RWL sensor telemetry
- ☐ Main estimator telemetry
- ☐ Main estimator high-resolution telemetry
- ☐ Raw GNSS sensor telemetry
- ☐ Raw PST3S star tracker telemetry
- ☐ ACP execution telemetry
- ☐ CubeComputer Health
- ☐ Health telemetry for CubeSense Earth
- ☐ Health telemetry for Reaction Wheels

UART2 Config

interval200ms

- ☒ All
 - ☐ Health telemetry for CubeNode PST3S
 - ☐ Health telemetry for CubeMag magnetometer
 - ☐ Health telemetry for CubeSense Sun
 - ☐ Torquer Current measurements
 - ☐ Raw CubeSense Sun telemetry
 - ☐ Raw external sensor telemetry
 - ☐ Controller telemetry
 - ☐ Backup estimator telemetry
 - ☐ Models telemetry

CAN Config

interval200ms

- ☒ All
 - ☐ Health telemetry for CubeNode PST3S
 - ☐ Health telemetry for CubeMag magnetometer
 - ☐ Health telemetry for CubeSense Sun
 - ☐ Torquer Current measurements
 - ☐ Raw CubeSense Sun telemetry
 - ☐ Raw external sensor telemetry
 - ☐ Controller telemetry
 - ☐ Backup estimator telemetry
 - ☐ Models telemetry

Figure 54: Unsolicited telemetry log setup

Unsolicited telemetry output (See [6] for details on unsolicited telemetry logging) can be enabled for either UART, UART2 or CAN communications interfaces..

For each communication channel, the user must specify the log output interval from the drop-down and use the checkboxes to select which telemetry frames must be output. Clearing all checkboxes will turn off unsolicited telemetry for that interface.

When CubeSupport receives an unsolicited telemetry log message from the CubeComputer, it will create a new tab with the heading **Unsolicited Telemetry**, and each telemetry frame in the log setup will be display a set of TCTLM UI elements with telemetry field values as in Figure 55.

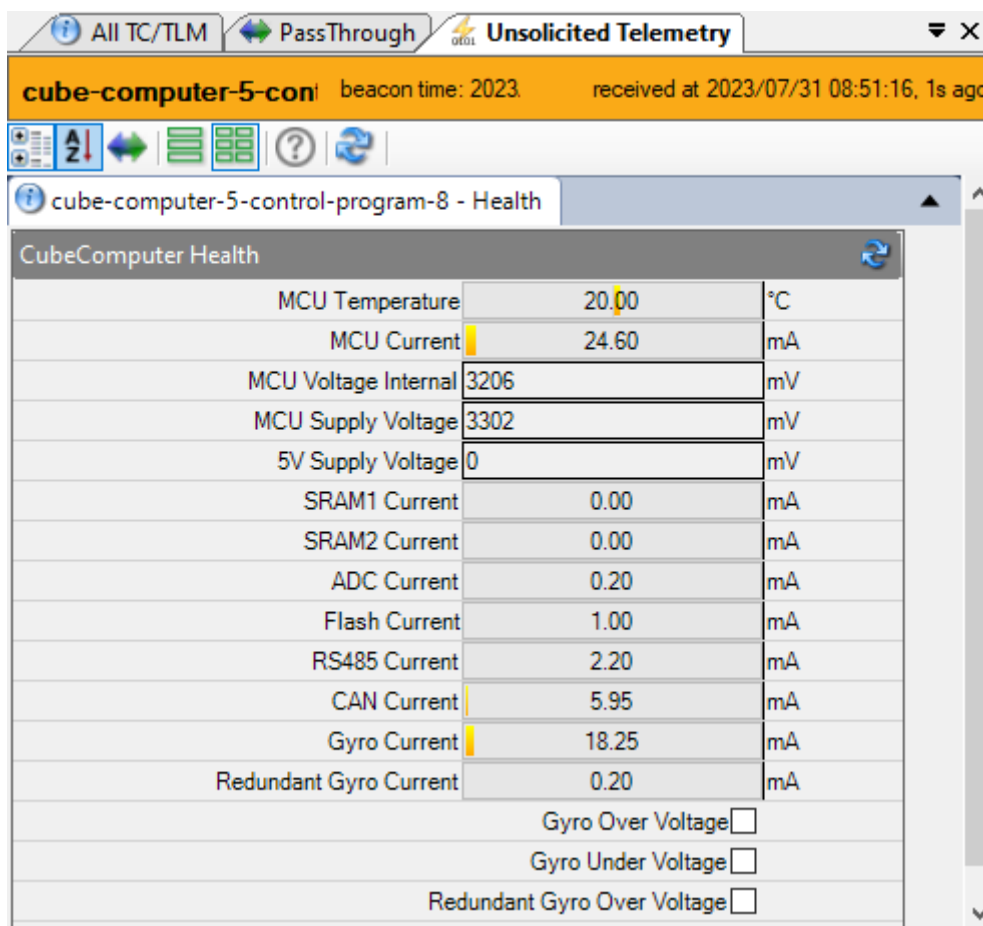


Figure 55: Example of unsolicited telemetry log window in CubeSupport

The received telemetry will also be displayed in the table in the [Telemetry Log](#) tab (see section 6.5.8) so that it can easily be exported to file.

Unsolicited event messages can also be enabled from this tab, by using the [Unsolicited Event Message Setup](#) TCTLM UI element in Figure 56.

Unsolicited Event Message Setup	
UART Info Events	☐
UART Minor Warning Events	☐
UART Major Warning Events	☐
UART Critical Events	☐
UART2 Info Events	☐
UART2 Minor Warning Events	☐
UART2 Major Warning Events	☐
UART2 Critical Events	☐
CAN Info Events	☐
CAN Minor Warning Events	☐
CAN Major Warning Events	☐
CAN Critical Events	☐

Figure 56: Unsolicited event output setup



The user can select which class of events must be output. For more detail on event messages, consult [6]. When CubeSupport receives an event message, it displays it under a newly created tab with the **Unsolicited Events** heading, as in Figure 57

Local Date/time	Remote date/tim...	Remote date/tim...	Uptime (s)	Class	Source	Details
2023/07/31 09:0...	2023/06/26 17:...	2023/06/26 15:...	600.243	Information	CubeComputer	Estimation Mode Cha
2023/07/31 09:0...	2023/06/26 17:...	2023/06/26 15:...	600.243	Information	CubeComputer	Estimation Mode Cha

Event name	Class	Counter
Estimation Mode Changed	Information	21945

Description	Source	Parameters								
Estimation Mode Changed : Estimator mode = EstGyro, Change Reason = EstChangeCmd, Is Main Mode = True	CubeComputer	<table border="1"><thead><tr><th>Parameter</th><th>Value</th></tr></thead><tbody><tr><td>Estimator mode</td><td>EstGyro</td></tr><tr><td>Change Reason</td><td>EstChangeCmd</td></tr><tr><td>Is Main Mode</td><td>True</td></tr></tbody></table>	Parameter	Value	Estimator mode	EstGyro	Change Reason	EstChangeCmd	Is Main Mode	True
Parameter	Value									
Estimator mode	EstGyro									
Change Reason	EstChangeCmd									
Is Main Mode	True									

Figure 57: Example of unsolicited events log output in CubeSupport

Event information is displayed in a table view. The user can select event entries to view the event details at the bottom of this region. The table can also be exported to a CSV file using the **Save** () button from the toolbar.

6.5.3 Power State

The power state of connected sensors and actuators can be changed by selecting the desired enumeration value for its power state in the **Power state** TCTLM UI element as in Figure 58.

The current power state of connected nodes can be requested using the **Node Initialization States** telemetry block.



Power state			
RWL0 power state	PowerOff	▼	
RWL1 power state	PowerOff	▼	
RWL2 power state	PowerOff	▼	
RWL3 power state	PowerOff	▼	
MAG0 power state	PowerOff	▼	
MAG1 power state	PowerOff	▼	
GYR0 power state	PowerOff	▼	
GYR1 power state	PowerOff	▼	
FSS0 power state	PowerOff	▼	
FSS1 power state	PowerOff	▼	
FSS2 power state	PowerOff	▼	
FSS3 power state	PowerOff	▼	
HSS0 power state	PowerOff	▼	
HSS1 power state	PowerOff	▼	
STR0 power state	PowerOff	▼	
STR1 power state	PowerOff	▼	
ExtSensor0 power state	PowerOff	▼	
ExtSensor1 power state	PowerOff	▼	

Figure 58: Power state TCTLM UI element

6.5.4 ADCS functionality

The ADCS functionality is grouped under [AdcsTelemetry](#), [AdcsTelemetrySensorRaw](#) and [Commands](#) TCTLM groups as shown in Figure 59.

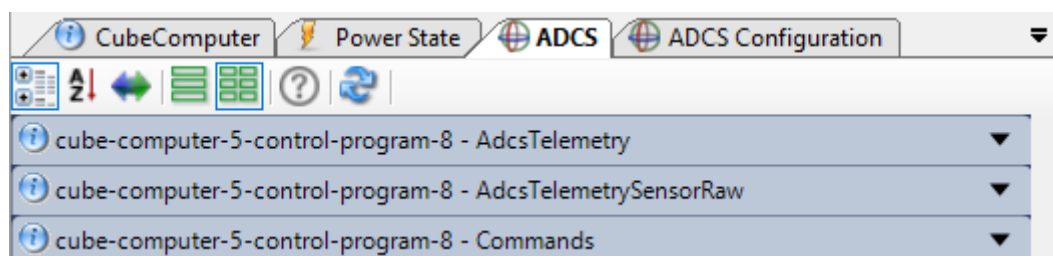


Figure 59: ADCS tab control group headings (collapsed)

The [AdcsTelemetry](#) group includes all calibrated sensor and actuator telemetry, as well as state telemetry, including estimated attitude state.

The [AdcsTelemetrySensorRaw](#) group includes raw sensor measurement telemetry.

The CubeADCS can be commanded into the various modes, and with desired reference values using the controls in the [Commands](#) group.

6.5.5 ADCS Configuration

The ADCS configuration settings are viewed and adjusted from the [ADCS Configuration](#) tab. This includes all configuration messages mentioned in [7].



6.5.6 CubeComputer and Node Health

The health telemetry (measured temperature, voltages and supply currents, as well as related status flags) for the CubeComputer itself, and all connected nodes can be requested and viewed from the **Health** tab.

6.5.7 File Transfers/Upgrades Tab

The “File Transfer / Upgrades” tab is shown in Figure 60. The functionality on this tab allows users to upgrade firmware of nodes (CubeProducts) connected to the CubeComputer. The CubeComputer control program itself cannot be upgraded from here. For the latter, the user has to reset the CubeComputer into its bootloader and perform the control program upgrade from there.

The tab contents are organised in a “Remote Files” region (top), and a “Local Files” region (bottom).

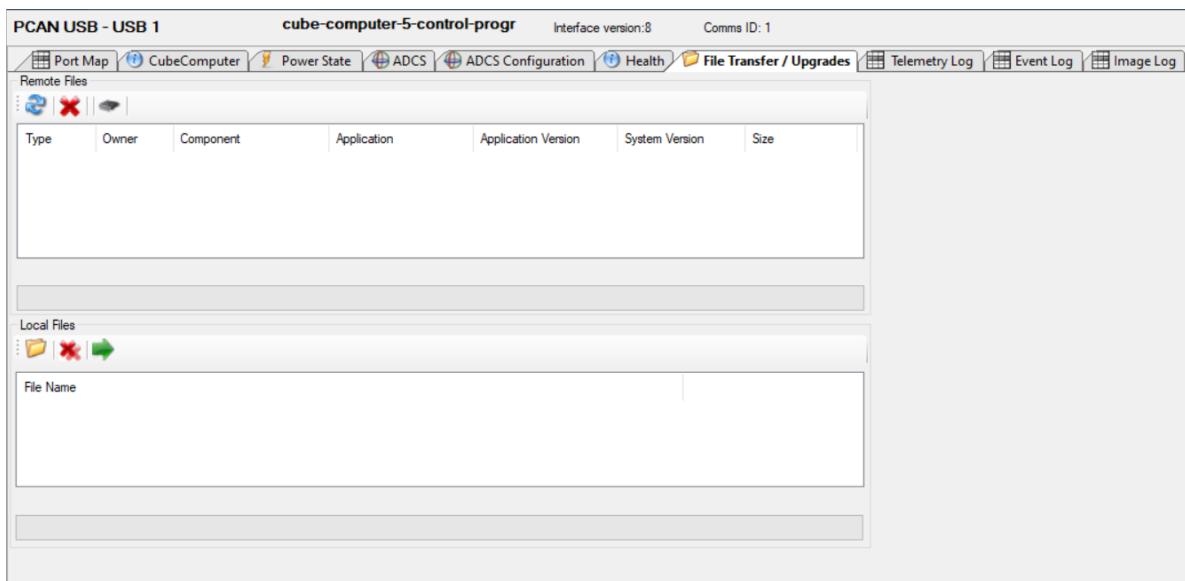


Figure 60: CubeComputer File Transfer / Upgrades Tab.

6.5.7.1 Local files

The local files group is used to load local CubeSpace (.cs) files and stage them for upload.

When the user clicks the “Load File” button (), a dialogue window appears, shown in Figure 61.

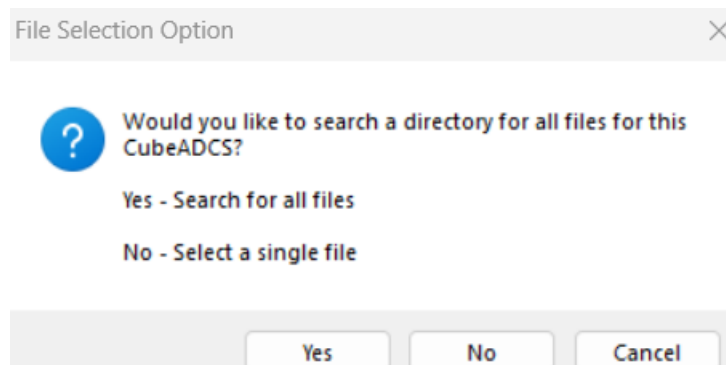


Figure 61: File Load Option



If the user selects the **Yes** option, CubeSupport searches for all the firmware and configuration files for the nodes that are part of the integrated CubeADCS, in the selected directory (the user is prompted to specify a directory after pressing the **Yes** button). The files that are located are loaded and staged for upload and displayed in the Local files list. This option is typically used when a new software bundle is provided, and the user wishes to upgrade the entire CubeADCS system.



Note that bootloader files are not included in the search as they are not required under normal operating conditions. If the user wishes to upload bootloader files, select the “No” option and search for the specific bootloader file

To specify a single specific file, press the **No** button in the dialog window in Figure 61, and choose the desired file from the file selection window that appears.

Once staged, the files that the user wants to upload can be selected and then uploaded by pressing the Upload button (→). The progress bar at the bottom of the region shows the progress of the upload of each individual file. When the batch of uploads is complete a message window appears indicating a successful upload of all files, or individual message window(s) for each failed upload.

6.5.7.2 Remote files

The remote files region is used to view files stored on the CubeComputer.

The refresh button can be used to request file information for all files. The file list is automatically refreshed after every batch of file uploads. Once the user has confirmed that the required files are present on the CubeComputer, the user can then select the configuration file(s) of the CubeProduct(s) to be upgraded, then click the upgrade button.

Note that although both the CubeProduct firmware binary and configuration are both upgraded, only the configuration file should be selected in order to perform an upgrade operation.



User discretion is required to ensure that the firmware binary for the selected configuration file is also present on the remote filesystem.

The progress bar at the bottom of the group shows the progress of each individual upgrade. When the batch of upgrades is complete a message window will appear indicating a successful upgrade of all CubeProducts, or individual message window(s) for each failed upgrade.

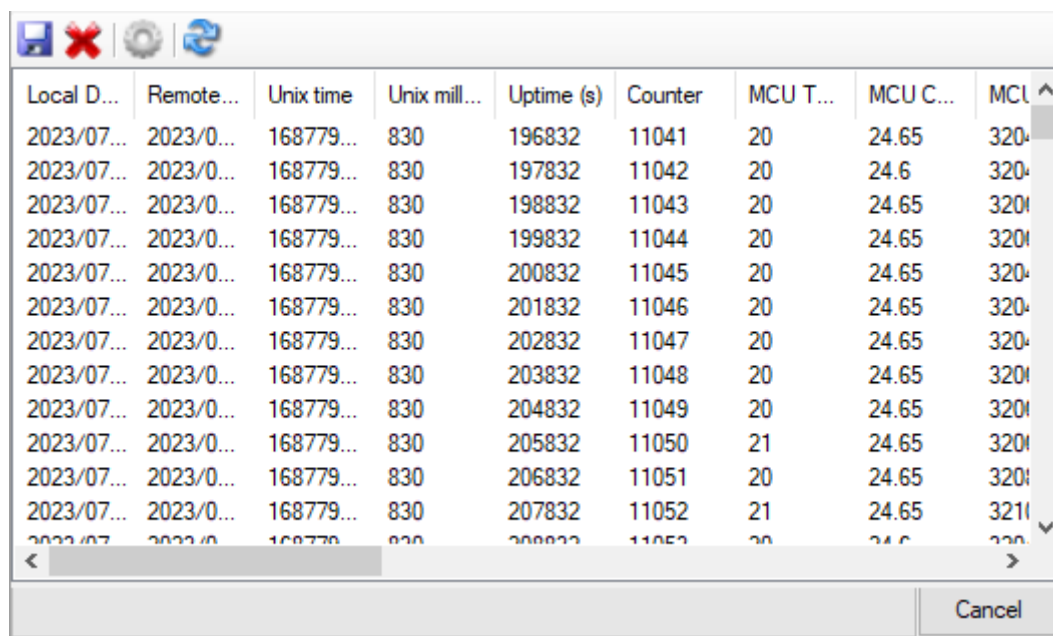


Note that upgrading the bootloader of a connected CubeProduct is not possible from the remote files group. (Under normal conditions, the bootloader should not be upgraded). If the bootloader firmware binary files have been uploaded, they will appear in the list of files, however, raw telecommands/telemetry must be used from the “CubeComputer” tab to perform the upgrade. Consult [4] for details.



6.5.8 Telemetry Log interface

The Telemetry Log interface displays a table for telemetry entries, with columns for each field of every telemetry frame in the log. Row entries exist for every time-stamped log entry, as in the example in Figure 62.



Local D...	Remote...	Unix time	Unix mill...	Uptime (s)	Counter	MCU T...	MCU C...	MCU
2023/07...	2023/0...	168779...	830	196832	11041	20	24.65	320
2023/07...	2023/0...	168779...	830	197832	11042	20	24.6	320
2023/07...	2023/0...	168779...	830	198832	11043	20	24.65	320
2023/07...	2023/0...	168779...	830	199832	11044	20	24.65	320
2023/07...	2023/0...	168779...	830	200832	11045	20	24.65	320
2023/07...	2023/0...	168779...	830	201832	11046	20	24.65	320
2023/07...	2023/0...	168779...	830	202832	11047	20	24.65	320
2023/07...	2023/0...	168779...	830	203832	11048	20	24.65	320
2023/07...	2023/0...	168779...	830	204832	11049	20	24.65	320
2023/07...	2023/0...	168779...	830	205832	11050	21	24.65	320
2023/07...	2023/0...	168779...	830	206832	11051	20	24.65	320
2023/07...	2023/0...	168779...	830	207832	11052	21	24.65	321

Figure 62: Telemetry Log display for CubeComputer

As mentioned in section 6.5.2.1, the log automatically displays unsolicited telemetry that CubeSupport receives. A log can explicitly be requested using the **Request Log** () toolbar button. The log request form that will open is shown in Figure 63.

The log that should be returned can be configured to have specific time range, or time range relative to the current time, as can be seen from the options in Figure 63.

The log can also be filtered to include only specific telemetry frames, using the checkboxes in **Filter Options** box. The interval between entries within the returned log can also be specified from the **Telemetry Interval** drop-down list.



Request Entry From

Create Request

Request Entry Options:

☒ Telemetries occurring in the last: 0 seconds

☐ Telemetries between two date time ranges:

From: 31 Jul 2023 08:59:00 UTC

To: 31 Jul 2023 09:39:59 UTC

☐ X telemetry entries following timestamp:

From: 31 Jul 2023 08:59:00 UTC

Number of Entries: 0

☐ First X telemetry entries:

☐ Last X telemetry entries:

Number of Entries: 1

☐ X telemetry entries following write counter:

Write Counter: 0

Number of Entries: 0

☐ All

Filter Options:

Telemetry interval: 200ms

☒ All

- ☒ Health telemetry for CubeNode PST3S
- ☒ Health telemetry for CubeMag magnetometer
- ☒ Health telemetry for CubeSense Sun
- ☒ Torquer Current measurements
- ☒ Raw CubeSense Sun telemetry
- ☒ Raw external sensor telemetry
- ☒ Controller telemetry
- ☒ Backup estimator telemetry
- ☒ Model telemetry

Request

Figure 63: Telemetry Log request form

After pressing the **Request** button, CubeSupport requests and downloads the log using the common transfer protocol (see [6] for detail).

The progress bar at the bottom of Figure 62 indicates the busy status. The transfer can be aborted using the **Cancel** button.

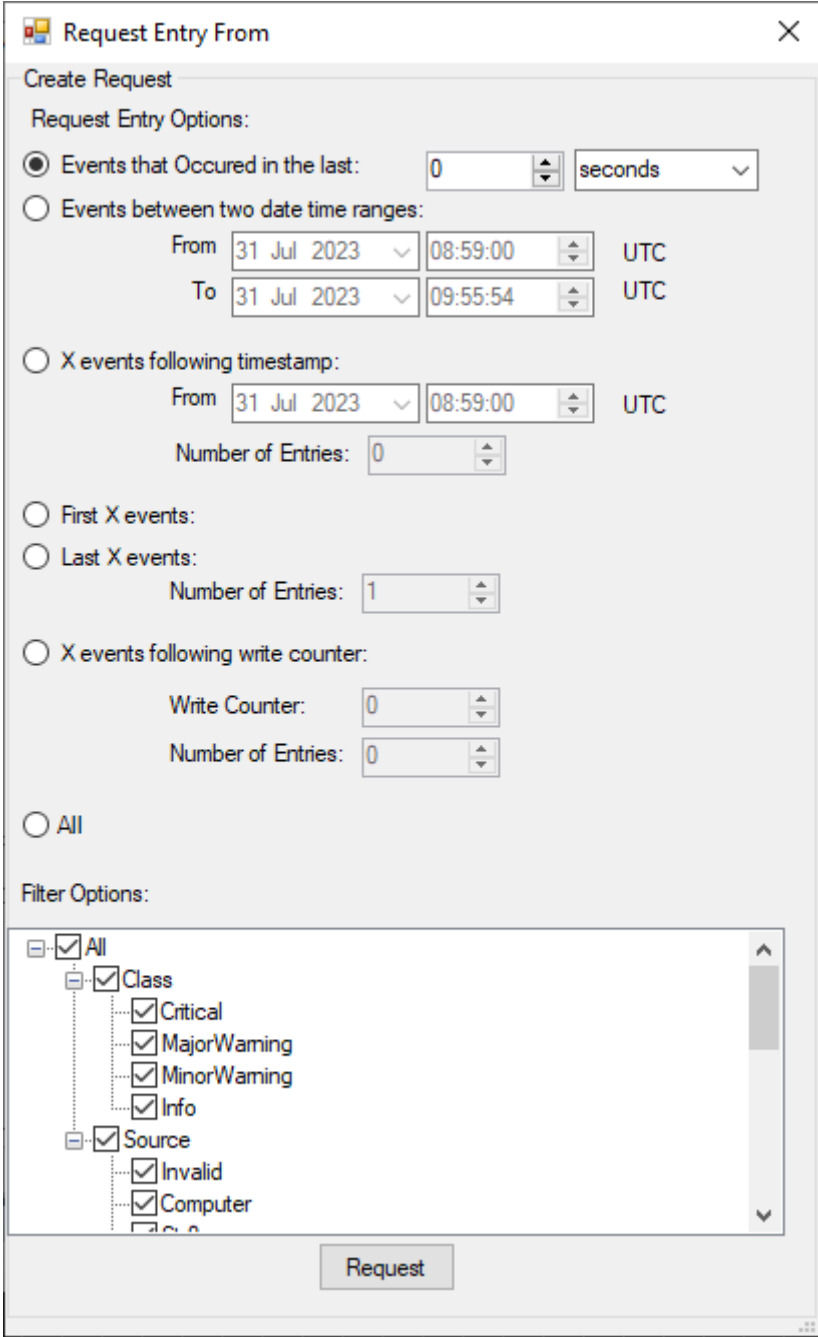
After the log has been downloaded, the table contents in Figure 62 is replaced by the downloaded and decoded data. The log can explicitly be cleared using the **Clear log** button (✖), and the log can be saved to a CSV file using the **Save** (💾) button. The export **Options** button (⚙️) displays a window where the delimiting character that is used for the exported file can be changed (by default a comma is used).



6.5.9 Event Log interface

The Event Log tab for the CubeComputer appears similar to the Unsolicited Event log tab, as in 6.5.2.1, but with the addition of a **Request log** () button in the toolbar.

The Request log button will display a window from where the event log download can be configured.



Request Entry From

Create Request

Request Entry Options:

☒ Events that Occured in the last: 0 seconds

☐ Events between two date time ranges:

From 31 Jul 2023 08:59:00 UTC

To 31 Jul 2023 09:55:54 UTC

☐ X events following timestamp:

From 31 Jul 2023 08:59:00 UTC

Number of Entries: 0

☐ First X events:

☐ Last X events:

Number of Entries: 1

☐ X events following write counter:

Write Counter: 0

Number of Entries: 0

☐ All

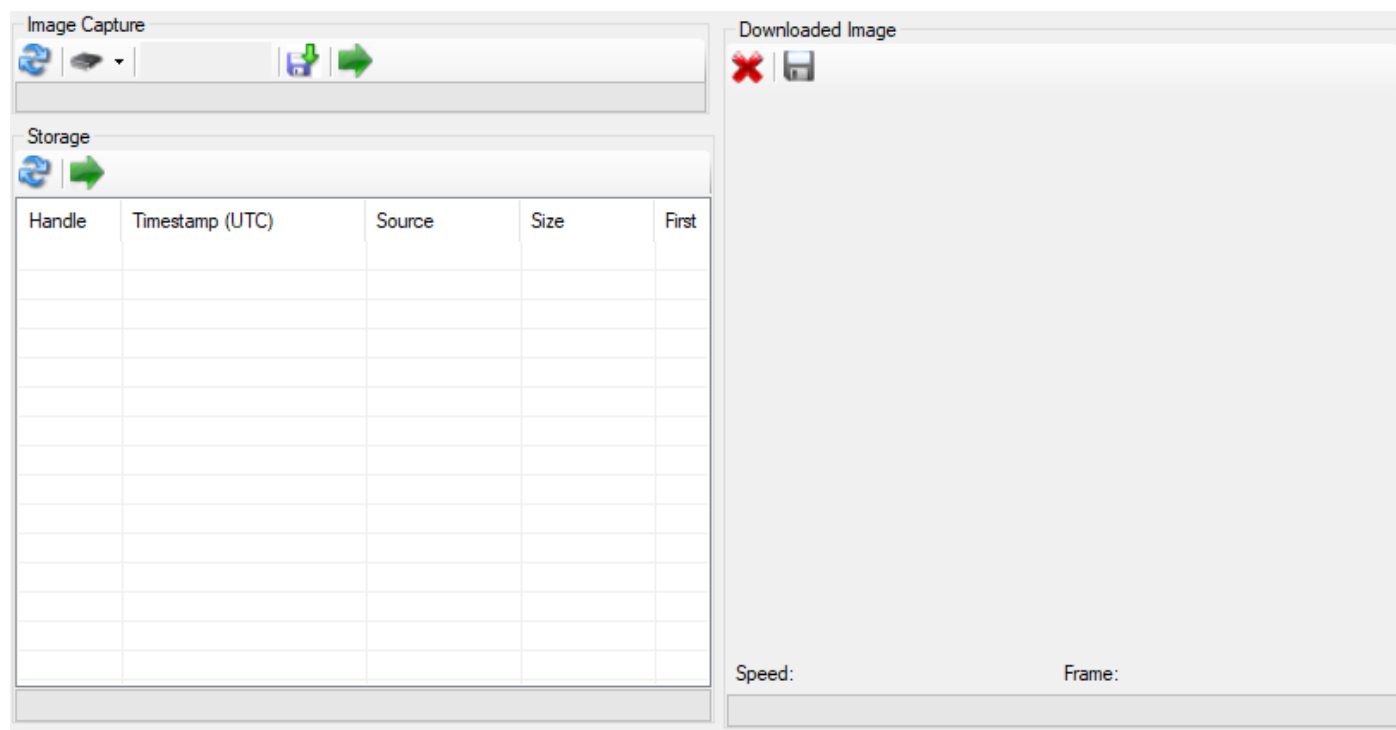
Filter Options:

- ☒ All
 - ☒ Class
 - ☒ Critical
 - ☒ MajorWaming
 - ☒ MinorWaming
 - ☒ Info
 - ☒ Source
 - ☒ Invalid
 - ☒ Computer

Request

Figure 64: Event Log request form

As with telemetry logs, events can be requested for a specific time range, or relative to current time, or by number of event entries. The log can also be filtered by event class, and event source. (See [6] for details).





Select the desired stored image and press the **Download** button () from the **Storage** region.

The downloaded image will be displayed in the **Download Image** region, from where it can be saved to a file on disk.

6.5.10.2 Transfer image directly from sensor

The captured sensor image can be streamed directly from the sensor (through the CubeComputer communications interface) by selecting the desired sensor from the **Node selection** dropdown button () in the **Image Capture** region, and then pressing the **Download** button () in the **Image Capture** region.

As before, the image will be displayed in the **Downloaded Image** region from where it can be saved to disk.

6.5.11 Pass-Through telecommands and telemetry

Pass-through refers to a communication pipe between the CubeComputer slave interface and the nodes connected in the system. See [6] for details on pass-through communications, including protocol details. This feature enables the full set of TCTLM of each node to be exposed via the CubeComputer slave interface. CubeSupport includes a convenient UI for this functionality on the **PassThrough** tab, as shown in Figure 66.



Figure 66: Pass-through interface

To make use of pass-through communication the target node must first be selected from the **Node selection** drop-down button, . This list will be populated with only the available nodes as per the expected nodes configuration of the CubeComputer. The list can be refreshed using the **Refresh** button, .

After selecting the desired target node, select the desired target program for that node – either the bootloader or the control program, using the **Program selection** dropdown button, .

Pass-through mode is entered when the user clicks the **Connect** button, . On successful pass-through mode initiation, CubeSupport will proceed to display all the relevant telecommands and telemetry UI elements for the target node in the remainder of the **PassThrough** tab region. The user can interact with these as if connected directly to the node.



Note that this feature in CubeSupport experiences occasional issues (in version 7.23) and that retrying the connection may be necessary in case of initial failure.



6.6 CubeMag Control Program

Once a successful connection to the CubeMag has been established, a single tab with label “Advanced” is displayed in the connected functions UI region as depicted in Figure 67.

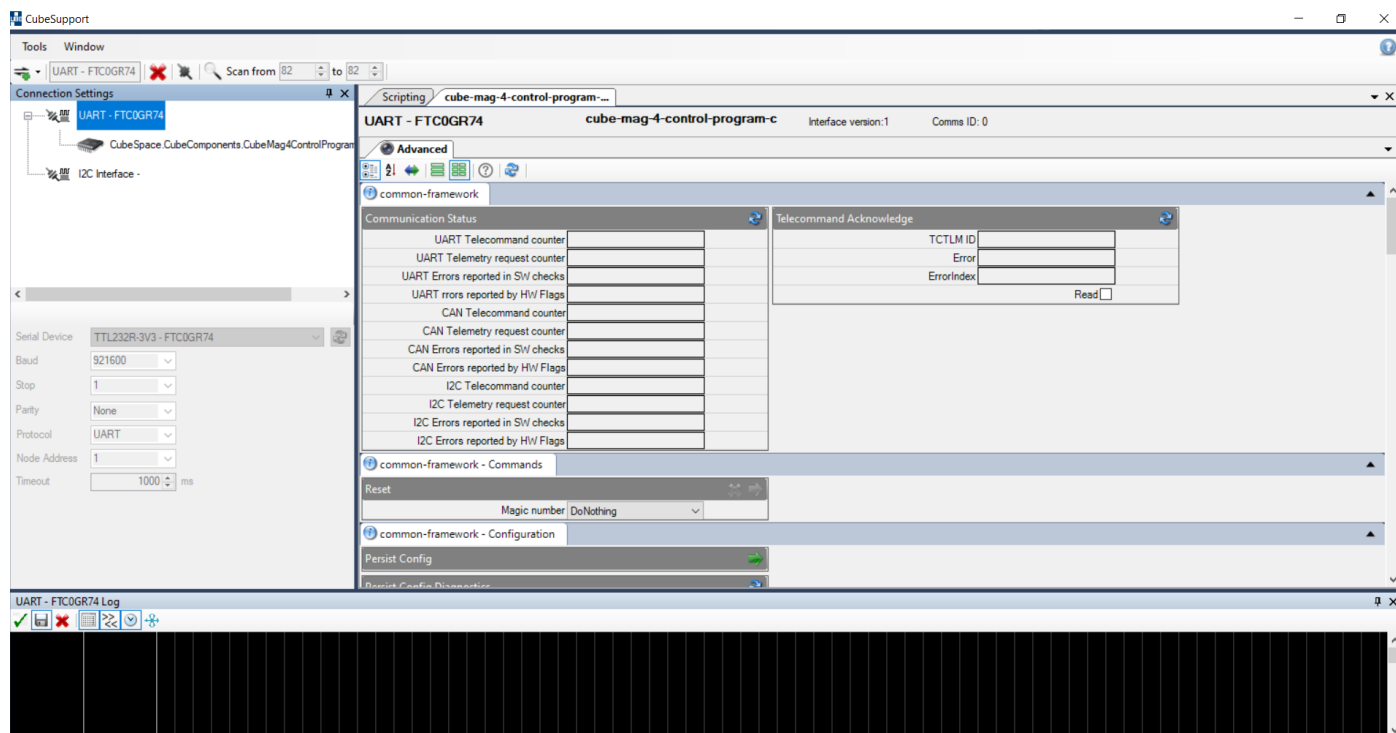


Figure 67: CubeMag Control Program Home Screen

The **Advanced** tab has multiple TCTLM UI elements split into two groups, **common framework** and **control program specific**.

The common framework allows the user to send or request TCTLM that are common to all CubeProducts (see 6.2.1).

The control program specific section displays everything that is relevant to the CubeMag control, output, and health. This extends to the status of the magnetometer and error flags. The control program specific TCTLM UI elements are elaborated further below.

6.6.1 CubeMag-Application

This group of TCTLM UI elements returns the measurement of the primary magnetometer raw and calibrated measurements. These fields are read-only telemetries and cannot be set.

6.6.2 CubeMag-Calibration

Configuration Parameters of the MMC sensor can be read back and modified from the current loaded values. These calibration parameters set the bias and the sensitivity of the CubeMag sensor.



CubeMag calibration values are determined during a calibration process at CubeSpace, and should not be changed without consulting CubeSpace



6.6.3 CubeMag-Configuration

This TCTLM group includes configuration parameters relevant to the configuration of the CubeMag MMC sensor.



CubeMag configuration settings should not be changed without consulting CubeSpace

6.6.4 CubeMag-Housekeeping

This TCTLM reports non-zero error codes if the primary sensing element of the CubeMag experiences an internal error. Please consult CubeSpace if any such errors are present.

6.6.5 CubeMag-Deployable

The next TCTLM groups are only relevant to the CubeMag deployable and will not appear if a CubeMag Compact is connected.

6.6.5.1 CubeMag-4-Control-Program-Deploy-1 - Application

The TCTLM in this group returns the measurement of the secondary magnetometer raw and calibrated measurements.

6.6.5.2 CubeMag-4Control-Program-Deploy-1 - Calibration

Configuration Parameters of the primary magnetic sensing element can be adjusted from the currently loaded values. These calibration parameters set the bias and the sensitivity of the primary magnetic sensor.



CubeMag calibration values are determined during a calibration process at CubeSpace, and should not be changed without consulting CubeSpace

6.6.5.3 PNI Magnetometer Config

This tab includes configuration parameters relevant to the configuration of the CubeMag PNI sensor.



CubeMag configuration settings should not be changed without consulting CubeSpace

6.6.5.4 Cube-mag-4-control-program-deploy-1 – Deployment

This TCTLM group provides the user access to the deployment commands needed to deploy the magnetometer boom.

The [Arm deploy](#) and the [Deploy](#) commands need to be sent in order for the boom to deploy. Before sending both commands, the user should prepare for deployment by mounting the CubeMag to a secure platform,



or by holding the CubeMag, by its harness, close to its base. The **Deployment Status** can be refreshed to obtain feedback from the deployment.

6.6.5.5 Cube-mag-4-control-program-deploy-1 – Housekeeping

This TCTLM reports non-zero error codes if the secondary sensing element of the CubeMag Deploy experiences an internal error. Please consult CubeSpace if any such errors are present.

6.6.5.6 Cube-mag-4-control-program-deploy-1- Housekeeping telemetry

This group contains telemetry that provide an overall health status of the unit. General MCU health telemetry can be received here along with the temperature measurements from the primary and secondary magnetometer. The deployment status telemetry is also included in this section.



6.7 CubeWheel Control Program

After following the procedure defined in chapter 5, the CubeWheel should be connected to the CubeSupport program.

The connection is **successful** when the control program tabs are populated, as in Figure 68.

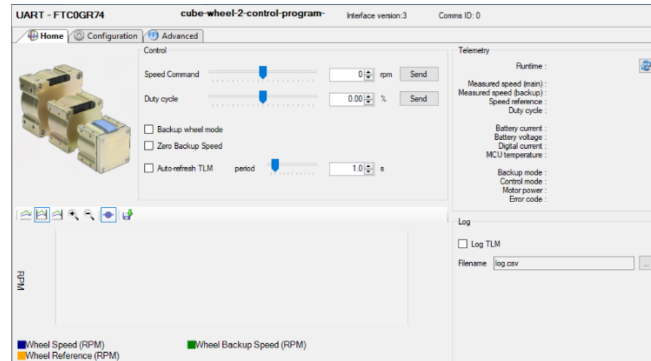


Figure 68: CubeSupport successful connection

The CubeWheel Control Program interface has three tabs: **Home**, **Configuration** and **Advanced**. Examples of each tab are shown in the images below.

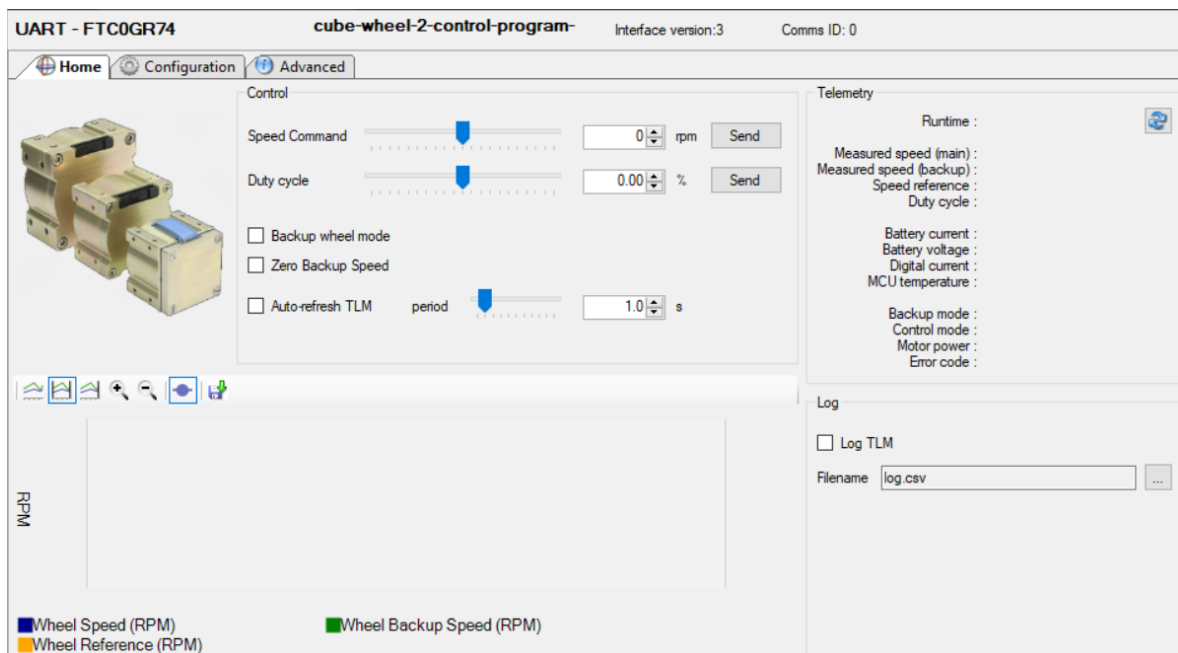


Figure 69: Home tab for CubeWheel

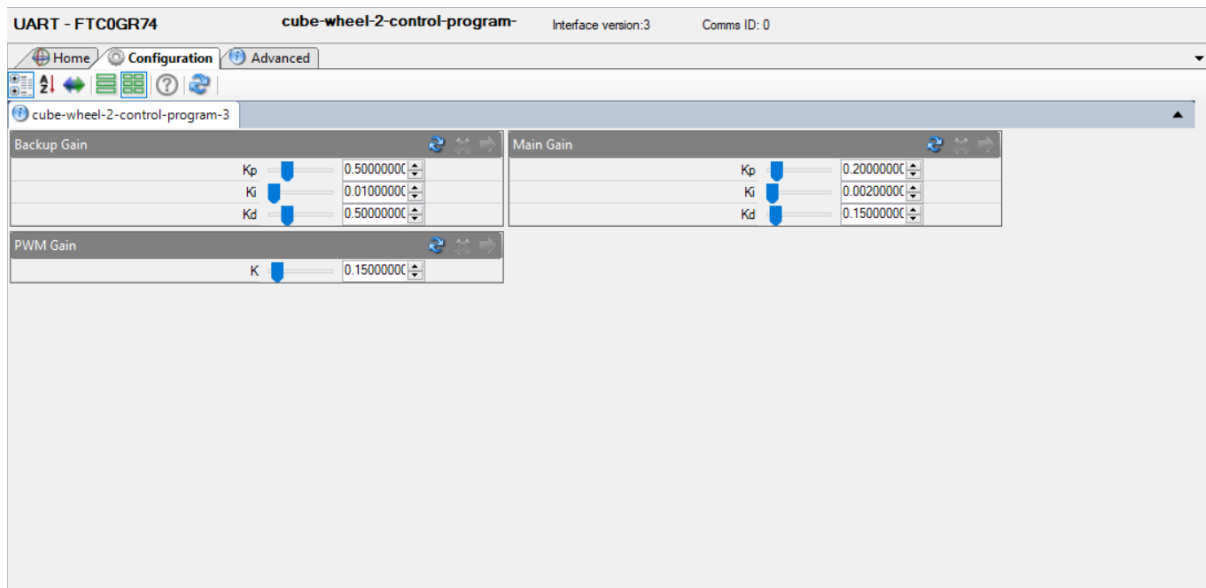


Figure 70: Configuration tab for CubeWheel

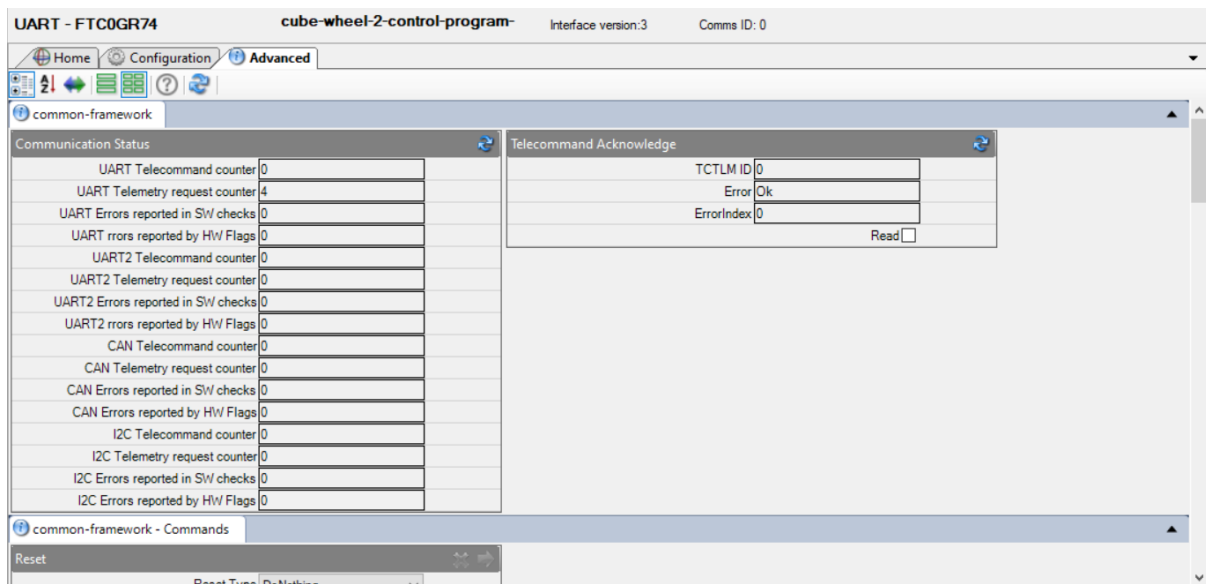


Figure 71: Advanced tab for CubeWheel

The **Home** tab contains fields for the following:

- wheel control commands
- wheel status telemetry

The **Configuration** tab, which is seldomly used, enables the user to change the following:

- Backup Gain
- PWM Gain
- Main Gain



Controller gains: The correct functioning of the CubeWheel is heavily dependent on the controller gains. Consult CubeSpace before changing the controller gains.

The **Advanced** tab contains fields for the following:

- Common Framework General
- Common Framework Commands
- Common Framework Configuration
- Common Framework Error Log
- Common Framework FDIR
- Common Framework Identification
- Common Framework Time
- *CubeWheel Control Program Specific*
 - *Backup Gain*
 - *Control Mode*
 - *Health Telemetry*
 - *PWM Gain*
 - *Status and Error Logs*
 - *Wheel Speed*
 - *Backup Wheel Mode*
 - *Main Gain*
 - *Motor Power*
 - *Wheel Commanded Duty Cycle*
 - *Wheel Data*
 - *Wheel Reference Speed*



6.7.1 Home tab

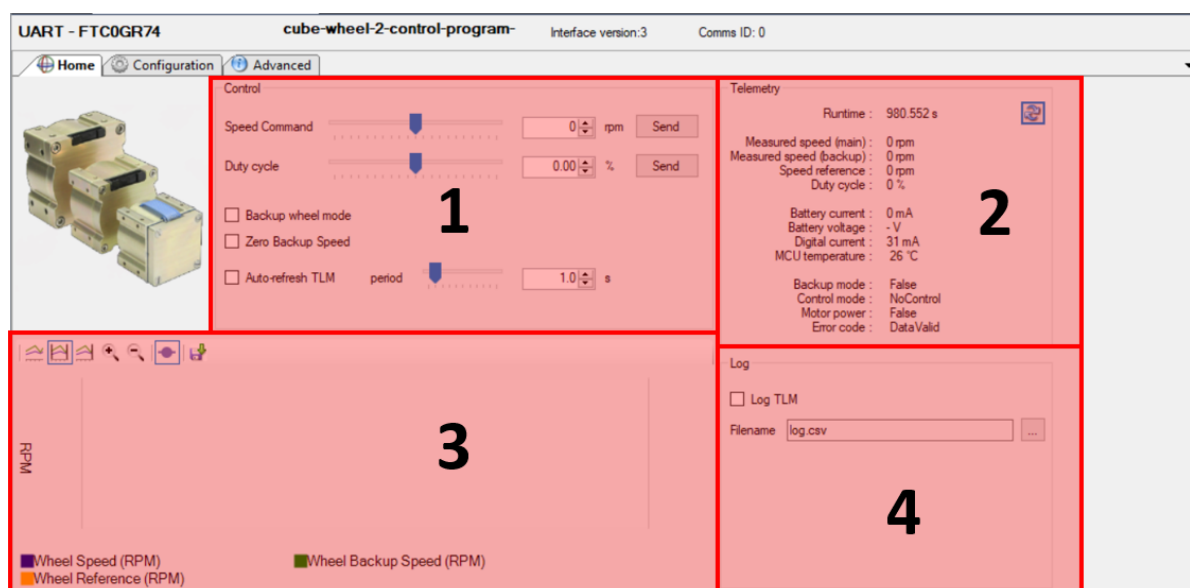


Figure 72: The UI regions of the Home tab

Figure 72 illustrates the various UI regions of the **Home** tab. Each region is briefly described below:

1. **Control** – The reference wheel speed and the commanded duty cycle can be set using the UI elements in this region. Backup wheel mode can also be enabled. It is further possible to automatically refresh data displayed on the Home tab by selecting the **Auto-refresh TLM** checkbox with desired interval.
2. **Wheel Data Telemetry** – The wheel speed reference and measurements, the wheel current, and the commanded duty cycle is displayed in this region. The current control mode, the status of the backup mode, the state of the various switches, and the error state (if any) is displayed in this region. The current control mode can be selected, and the backup mode and error codes are displayed in addition to the battery current, battery voltage, digital current and MCU temperature.
3. **Telemetry Graph** – A graph of the wheel reference speed and measurements is shown here.
4. **Log** – This region houses data logging settings

6.7.1.1 Creating a telemetry log

To create a telemetry log, simply check the **Log Data** checkbox in the **Log Settings** region under the **Home** Tab. Note that **Auto-Refresh TLM** must be enabled before telemetry can be logged.

CubeSupport generates the log as a .csv file in CSV format. The filename and location of the log must be set in the cubesupport program in the log region. The following telemetry data is logged:

RunTime	RunTime in seconds (including the millisecond telemetry)
WheelSpeed	Wheel speed measured by the primary sensor in rpm
WheelBackupSpeed	Wheel speed measured by secondary sensor in rpm
WheelSpeedReference	Reference wheel speed in rpm



DutyCycle	Duty cycle (including direction) of PWM control signal expressed as a percentage
WheelCurrent	Motor current (drawn directly from power supply) in mA
DigitalCurrent	MCU and 3V3 logic current in mA
MCUTemperature	MCU Temperature in degrees Celcius
BackupMode	Backup mode flag
ControlMode	Control mode
MotorSwitch	Motor switch state
WheelErrorState	Wheel data status

Figure 73 illustrates how the logs are created in the relevant folder. Note that while CubeSupport is busy logging telemetry, the size of the log will be given as 0 KB. **Do not open a log file while CubeSupport is busy logging to that file.** To view a log, first ensure that [Log Data](#) on the [Home](#) tab is not selected before attempting to open the file. If the log file name is not changed, the previous log file will be appended.

Name	Date modified	Type	Size
autogen	26/04/2023 08:12	File folder	
obj	26/04/2023 08:13	File folder	
output	26/04/2023 09:08	File folder	
cslib.dll	26/04/2023 08:13	Application extension	2 473 KB
cslib.pdb	26/04/2023 08:13	Program Debug Database	522 KB
cubesupport	26/04/2023 08:13	Application	95 KB
cubesupport.exe	26/04/2023 08:13	Configuration Source File	2 KB
FTD2XX_NET.dll	26/04/2023 08:13	Application extension	69 KB
FTD2XX_NET	26/04/2023 08:13	XML Source File	104 KB
libcsp32.dll	26/04/2023 08:13	Application extension	828 KB
libcsp64.dll	26/04/2023 08:13	Application extension	921 KB
log	26/04/2023 10:38	Microsoft Excel Comma...	20 KB
Newtonsoft.Json.dll	26/04/2023 08:13	Application extension	686 KB
Newtonsoft.Json	26/04/2023 08:13	XML Source File	694 KB
nodedef.dll	26/04/2023 08:13	Application extension	2 071 KB
nodedef.pdb	26/04/2023 08:13	Program Debug Database	797 KB
PCANBasic.dll	26/04/2023 08:13	Application extension	337 KB
WeifenLuo.WinFormsUI.Docking.dll	26/04/2023 08:13	Application extension	301 KB

Figure 73: CubeWheel telemetry logs created by CubeSupport

An example of a CubeWheel telemetry log file is shown in Figure 74.



RunTime	WheelSpeed	WheelBackupSpeed	WheelSpeedReference	DutyCycle	WheelCurrent	DigitalCur	MCUtemp	BackupMode	ControlMode	MotorSwitch	WheelErrorState
3262.505	0	0	0	0	0	0	31	27	FALSE	NoControl	FALSE
3262.631	0	0	0	0	0	0	31	28	FALSE	NoControl	FALSE
3262.76	0	0	0	0	0	0	31	27	FALSE	NoControl	FALSE
3262.886	0	0	0	0	0	0	31	28	FALSE	NoControl	FALSE
3263.017	0	0	0	0	0	0	31	28	FALSE	NoControl	FALSE
3263.145	0	0	0	0	0	0	31	27	FALSE	NoControl	FALSE
3263.274	0	0	0	0	0	0	31	28	FALSE	NoControl	FALSE
3263.402	0	0	0	0	0	0	31	28	FALSE	NoControl	FALSE
3263.531	0	0	0	0	0	0	31	28	FALSE	NoControl	FALSE
3263.656	0	0	0	0	0	0	31	28	FALSE	NoControl	FALSE
3263.784	0	0	0	0	0	0	31	28	FALSE	NoControl	FALSE
3263.919	0	0	0	0	0	0	31	27	FALSE	NoControl	FALSE
3264.057	0	0	0	0	0	0	31	28	FALSE	NoControl	FALSE
3264.183	0	0	0	0	0	0	31	27	FALSE	NoControl	FALSE
3264.329	0	0	0	0	0	0	31	28	FALSE	NoControl	FALSE
3264.455	0	0	0	0	0	0	31	28	FALSE	NoControl	FALSE
3264.583	0	0	0	0	0	0	31	28	FALSE	NoControl	FALSE
3264.711	0	0	0	0	0	0	31	28	FALSE	NoControl	FALSE

Figure 74: CubeWheel telemetry log example

6.7.2 Configuration tab

Figure 70 displays the **Configuration** tab. This tab houses three TCTLM UI elements for adjusting the CubeWheel configuration:

1. **Backup Gains** – The gains of the main speed controller at low speed can be read and configured using this TCTLM UI element.
2. **Main Controller Gain** – The gains of the main speed controller at high speed can be read and configured using this TCTLM UI element.
3. **General PWM Gain** – The gain of the PWM register that commands the motor driver can be read and configured using this TCTLM UI element.



Do not make changes to wheel configuration settings without consulting CubeSpace

6.7.3 Advanced tab

The advanced tab displays automatically generated TCTLM UI elements for the CubeWheel, based on its node definition (API). The available TCTLM UI elements include those of the common-framework (see section 6.2.1 for details) and TCTLM that are specific to the CubeWheel control program.

The control program specific group of TCTLM UI elements includes all possible telecommands and telemetry that may be sent/requested from the CubeWheel.



6.8 CubeSense Sun Control Program

On connecting to the CubeSense Sun, CubeSupport displays a **Home** and **Advanced** tab. The Home tab contents are shown in Figure 75.

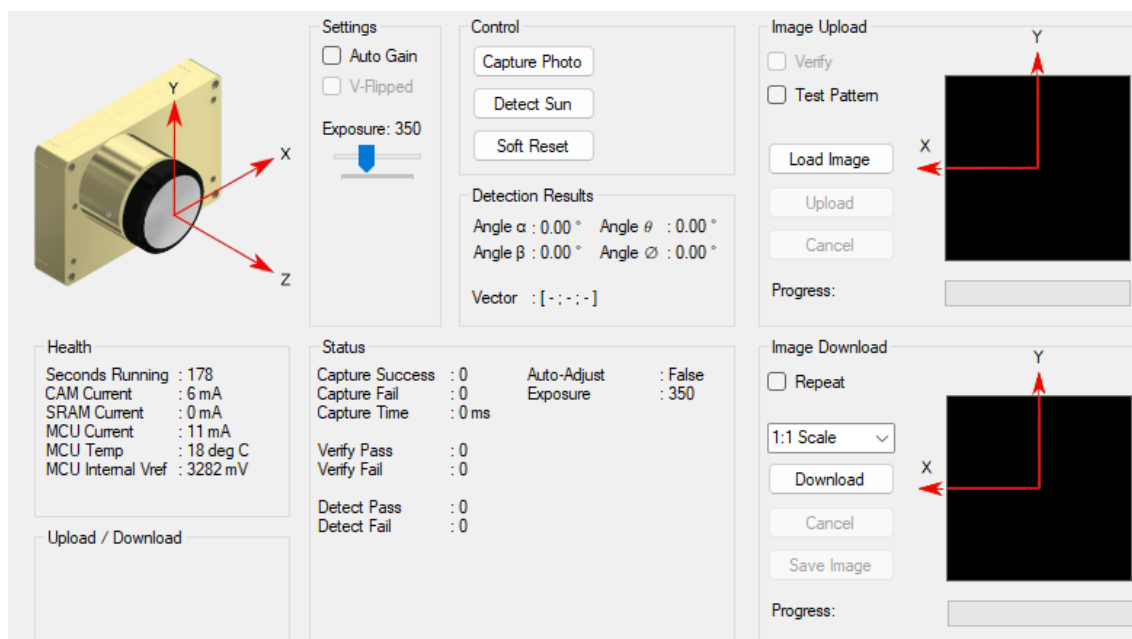


Figure 75: CubeSense Sun Home tab contents

6.8.1 CubeSense Sun Home tab functionality

The **Health** group-box displays measured current, voltage levels and temperature for the connected CubeSense Sun. The health telemetry displayed on the **Home** tab will continuously refresh at a rate of 1Hz once the CubeSense Sun is connected.

The **Capture Photo**, **Detect Sun** and **Soft Reset** buttons can be used to initiate the relevant processes. To perform a sun detection, press the **Capture Photo**, followed by the **Detect Sun** button. Following this, the **Detection Results** updates to show the angles to the measured sun direction, as well as the measured sun vector in the Sensor Coordinate Frame, and the **Status** region displays telemetry regarding detection status, and counters of successful vs. failed sun detection attempts.

The **Settings** group allows the user to change gain and exposure settings. This might be required for lab tests, where a higher exposure value is needed if the stimulus used to represent the sun is dimmer than what the sensor can recognise.

An image from the sensor can be downloaded by pressing the **Download** button in the **Image Download** group of controls. The scale drop-down list can be used to download a sub-sampled version of the image. This is useful if a shorter download time is needed, and a lower quality image will suffice. The downloaded image will be displayed but can also be saved to disk by pressing the **Save Image** button and selecting the desired file location and name.

The **Image Upload** group of controls is intended for CubeSpace internal use.



6.8.2 CubeSense Sun Advanced tab

The **Advanced** tab displays automatically generated TCTLM UI elements for all the telecommands that may be sent to, and telemetry that may be requested from the CubeSense Sun. This includes all common-framework TCTLM (see section 6.2.1 for details).

From this tab it is possible to change the configuration of the CubeSense Sun. Configuration settings that are adjustable from a user perspective include the gain and exposure, and the mask areas. The mask areas allow users to define rectangular areas in the image that should be ignored when detecting the sun. This is useful if the satellite has appendages or deployed elements that are visible in the field of view. Reflections from such parts can lead to false sun detections. For details on setting the mask areas, please consult the CubeSense Sun User Manual.

To make configuration settings permanent, send a **Persist Configuration** telecommand from this tab.



Configuration settings other than gain, exposure and mask areas should not be changed without consulting CubeSpace



6.9 CubeSense Earth Control Program

The CubeSense Earth sensor can be configured and tested through the CubeSupport application. When connecting to the CubeSense Earth node a **Home** tab and **Advanced** tab is populated in the connected functions UI region as shown in Figure 76.

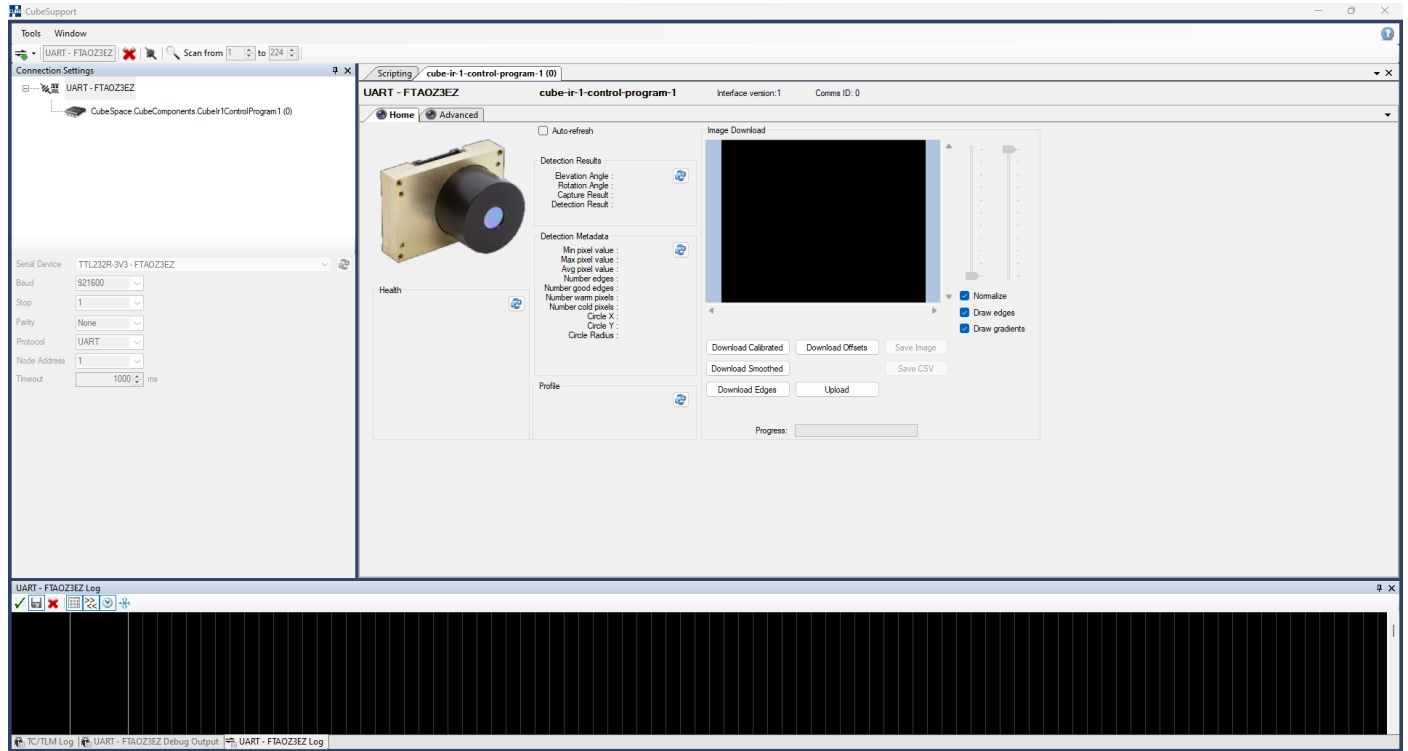


Figure 76: Home tab for CubeSense Earth Control Application

6.9.1 Horizon detection

To manually trigger a capture and detect operation, press the  button in the **Detection Results** region, see Figure 77. Once a capture and detect operation completes the values of the results are shown. The results include the elevation angle, rotation angle, whether the image capture operation was successfully completed and lastly if the horizon was detected.

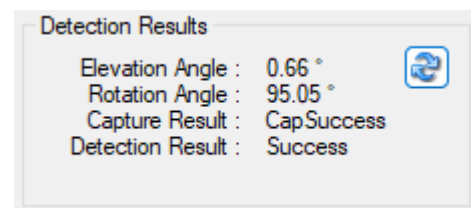
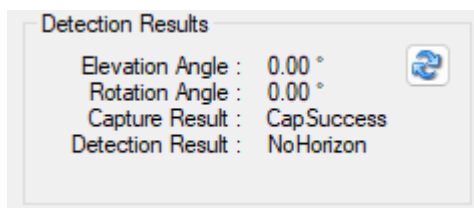


Figure 77: Detection Results Subsection Figure 78: Detection result - Successful Detection

For more information on the image that was captured click on the  button in the **Detection Metadata** section as shown in Figure 79

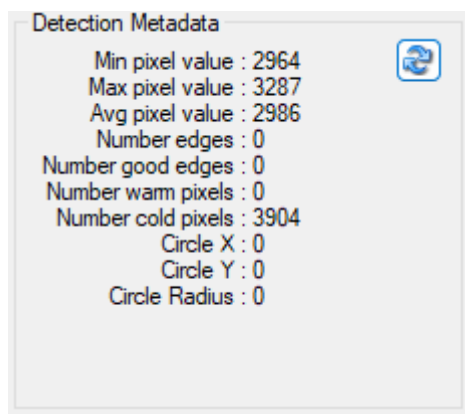


Figure 79: Detection Metadata - No Horizon in View

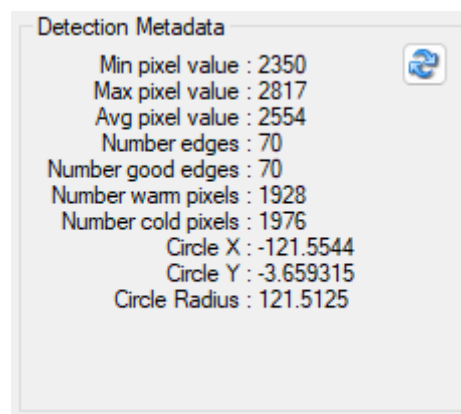


Figure 80: Detection Metadata - Horizon in View

The values returned are the minimum, maximum and average pixel values (generally values below 1200 indicate space temperature, earth temperatures are in the range of 1200 – 3200 and the Sun shows up in the ranges closer to 5000).

The next items are the number of edges detected and the number of good edges, if these values do not match it indicates that false edges were discarded to increase the accuracy.

The number of warm pixels and number of cold pixels gives an indication of how much of the Earth is visible in the picture.

The circle x, circle y and circle radius entries indicate the measurements of the circle extrapolated from the arc of the horizon. The circle radius entry is useful as an extra indication of how accurate the extrapolation is, the radius should be around 120 (which translates to the radius of the Earth)

The last function related to the capture and detection is the ☐ Auto-refresh checkbox located above the **Detection Results** region. Checking this box will refresh the **Detection Results** and **Detection Metadata** sections once a second.

6.9.2 Image downloads



Note: Do not use any of the download buttons in the “Image Download” section of the app, discussed in the following section, or trigger a manual operation with the  buttons while the ☒ Auto-refresh is active. This can lead to missed telemetry messages between the CubeSupport and the CubeSense Earth, which may cause undefined behaviour. Untick the box first then click any of the download buttons to download and display the last capture.

One can download and display the latest image capture using the **Image Download** section, see Figure 81. The **Download Calibrated** button downloads the calibrated image as used for detection. The **Download Smoothed** button is used to download a gaussian smoothed version of the image. To download the detected edges and overlay them over the displayed image the **Download Edges** button can then be used.

The images retrieved from the device can be saved to the local machine using the **Save Image** and **Save CSV** buttons.



The **Normalize**, **Draw edges** and **Draw gradients** checkboxes changes how the image is displayed in the preview window. The **Normalize** checkbox is checked by default and auto adjusts the contrast levels of the image. Unticking the **Normalize** checkbox allows for manual adjustment thereof using the sliders.

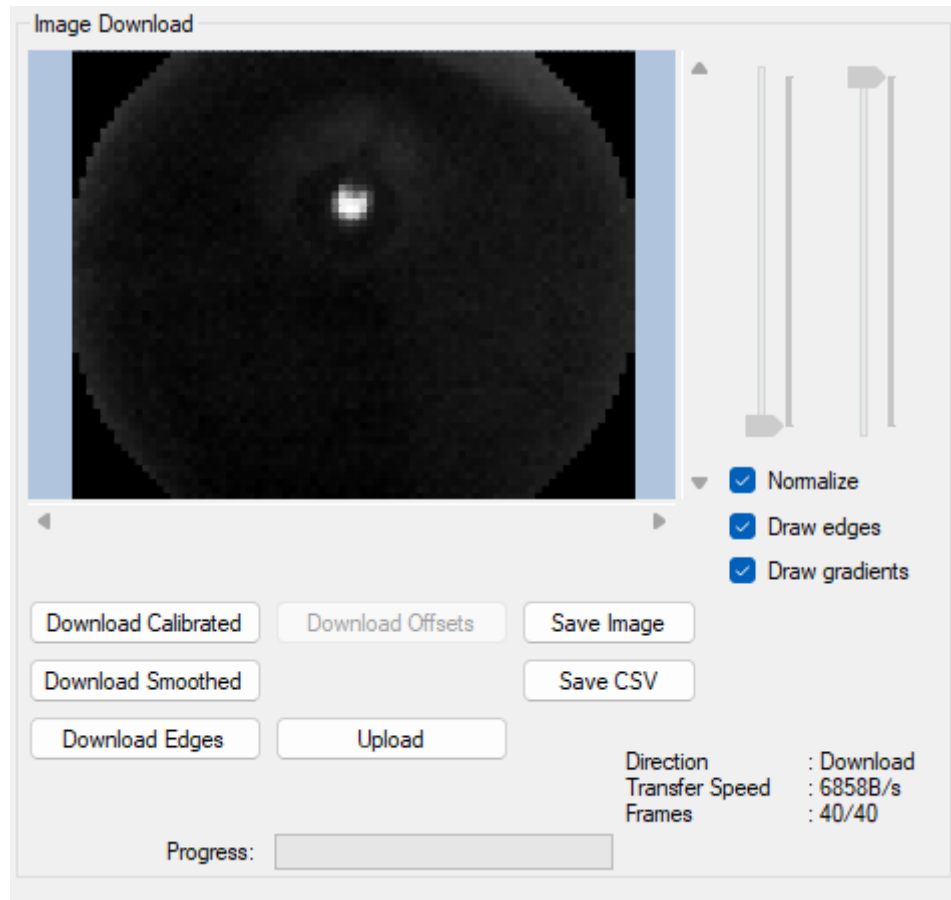


Figure 81: Downloading and displaying images

6.9.3 Sensor Health

The last regions of interest on the **Home** tab is the **Health** and **Profile** sections which provide a summary of the most crucial health indicators (Figure 82) as well as execution times of the internal processes to complete (Figure 83) respectively.

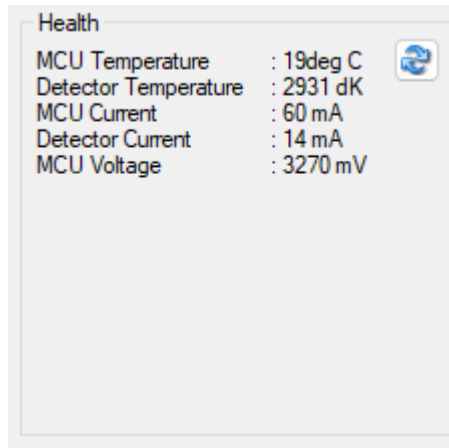


Figure 82: Home Tab - Health Overview

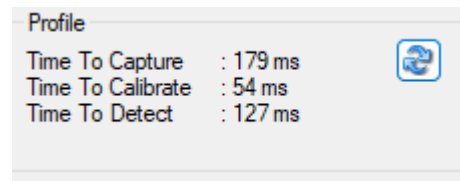


Figure 83: Home Tab - Profile

6.9.4 Dead-pixel masking

The infrared sensor used in the CubeSense Earth node ships with factory calibration of dead pixels but in some cases, additional dead pixels may appear. These pixels are detected during the CubeSpace calibration process and then masked. Should dead pixels appear throughout the lifetime of the satellite's orbit they can be added to the list of dead pixels using the functions accessible through the **Advanced** tab of the Control Program interface. A brief overview of these parameters are given below for the dead pixel settings (refer to Figure 85, Figure 84 and Figure 86).

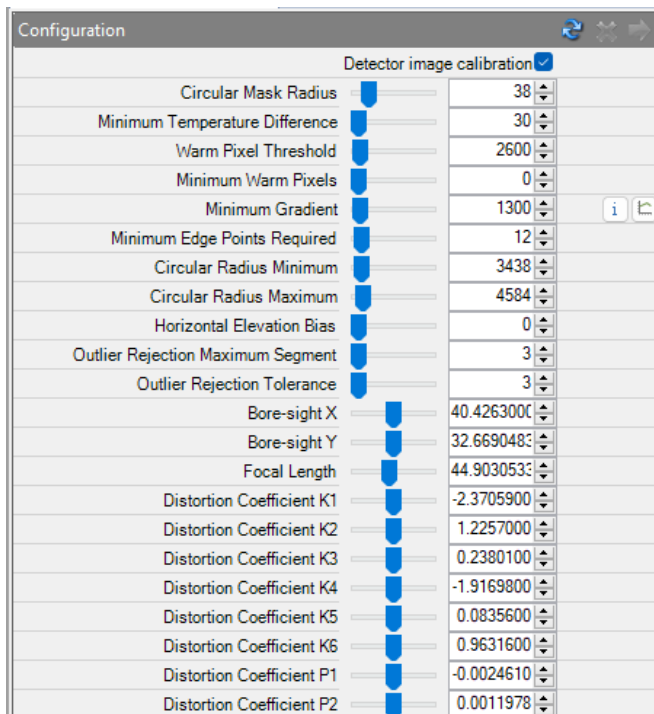


Figure 85: Advanced Tab - Configuration

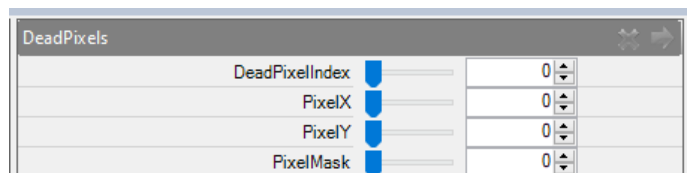


Figure 84: Advanced Tab - DeadPixels

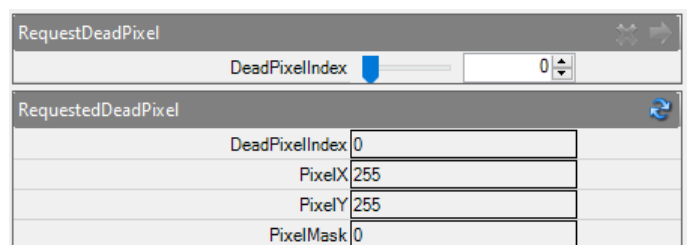


Figure 86: Advanced Tab - RequestDeadPixels

The process of masking out dead pixels are described in the following steps:



- 1 Request the current dead-pixel table entries using the `RequestDeadPixel` command, and `RequestedDeadPixel` telemetry request until an empty slot is found (PixelX = 0, PixelY = 0, PixelMask = 0).
- 2 Locate the X/Y coordinate of the dead pixel on the image. To do this the user can zoom in on the image preview until the grid appears (shown in Figure 89) and then hover the mouse cursor over the pixel of interest.
- 3 Evaluate the pixels surrounding the identified dead pixel, and compute a mask value to indicate which of the neighbouring pixel may be used to substitute for the dead pixel. The mask value is a bit-mask where each bit maps to a neighbouring pixel relative X/Y location, as shown in Figure 87.

	X-1	X	X+1
Y-1	1000 0000	0000 0001	0000 0010
Y	0100 0000	Dead Pixel	0000 0100
Y+1	0010 0000	0001 0000	0000 1000

Figure 87: Bit-mask Values For Surrounding Pixels

- 4 Add the entry to the dead-pixel table using the `DeadPixel` telecommand. Use the table index obtained in step 1 for the `DeadPixelIndex`.
- 5 Save the updated configuration to flash memory using the `Persist Config` telecommand.

Using Figure 88 as an example the two dead pixels have coordinates of (56, 63) and (57, 63). The masking pixels that were chosen in this example are shown in Figure 90 and Figure 91. The pixel that was selected for the first pixel is to the top left (PixelMask = 128) and the second dead pixel's mask was selected as the pixel above it (PixelMask = 1). The result of the masking is shown in Figure 89.

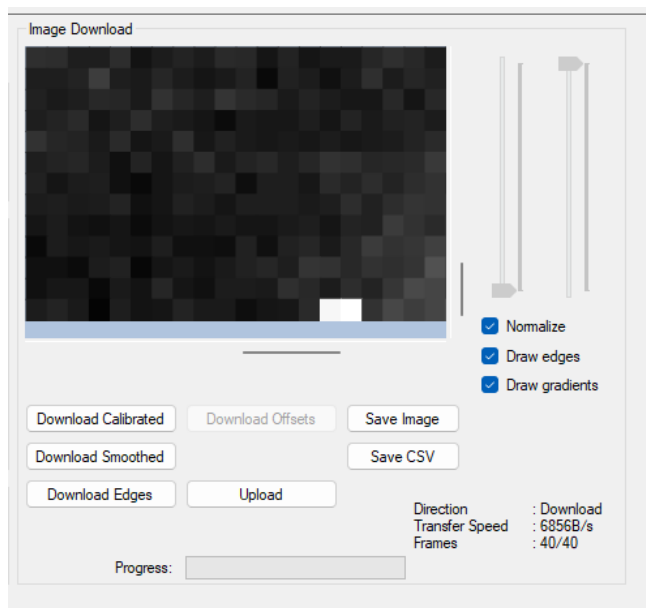


Figure 88: Dead Pixels Pre-Masking

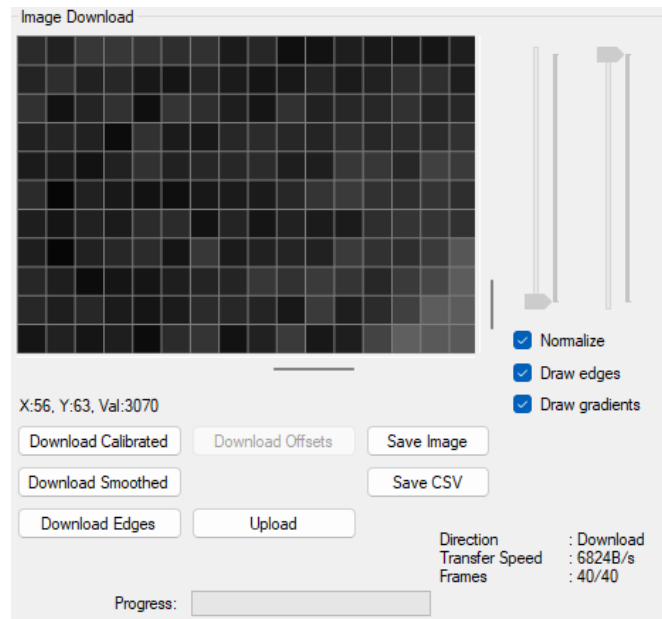


Figure 89: Dead Pixels After Masking

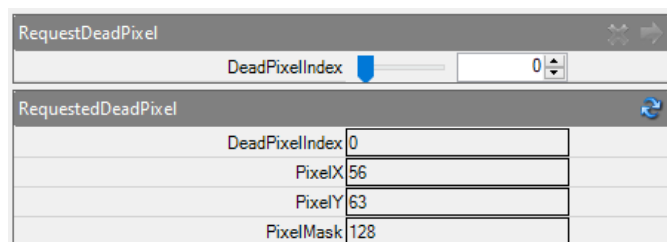


Figure 90: Mask Entry For First Dead Pixel

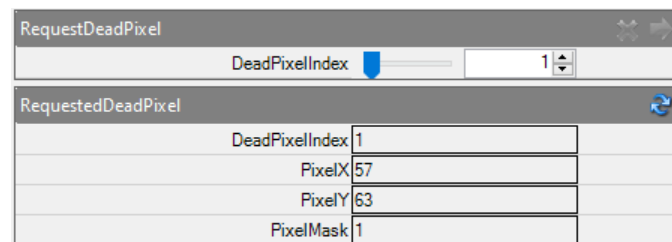


Figure 91: Mask Entry For Second Dead Pixel

6.9.5 Detection settings

The entries in the **Configuration** command of the **Advanced** tab consist of detection algorithm parameters as well as optical calibration parameters.



The optical calibration parameters are determined during CubeSpace calibration and should not be modified.

Other detection settings (see Table 10) may be adjusted to improve algorithm performance, but should be done in consultation with CubeSpace.

Table 10: Configuration Entries Related to the Detection algorithm

CONFIGURATION ENTRY	NOTE
Circular Mask Radius	Is used to set the circular mask, measured in pixels and located around the center of the image. This is used to reject false edges. Any identified edge point that falls outside of this radius will not be used in the detection algorithm



CONFIGURATION ENTRY	NOTE
Minimum Temperature Difference	Minimum temperature difference required to detect a valid gradient
Warm Pixel Threshold	The threshold at which the pixel is a warm pixel
Minimum Warm Pixels	Used to offset the Warm Pixel Threshold to tweak detection accuracy
Minimum Gradient	Minimum sharpness of a gradient to be considered an edge point
Minimum Edge Points Required	The minimum required edge points to extrapolate the radius of the arc
Circular Radius Minimum	The minimum radius value to be considered a valid horizon
Circular Radius Maximum	The maximum radius value to be considered a valid horizon
Horizontal Elevation Bias	Can be used to compensate for atmospheric effects
Outlier Rejection Maximum Segment	The maximum nr of continuous edge points that can be rejected when not within the Outlier Detection Tolerance
Outlier Rejection Tolerance	Sets the maximum distance that an edge point needs to be within the proximity of another edge point to be considered part of that arc



6.10 CubeStar Control Program

The user interface for the CubeStar is still in development. Future versions of this document will include the details of how to operate the CubeStar through CubeSupport.



7 Scripts

The scripting interface is available as a tab from the **User Controls** section from the main CubeSupport window. The scripting tab is shown in Figure 92.

Figure 92: Scripting window.

7.1 Built-in scripts

The **Script Selection** window lists four built-in scripts:

Table 11 Built-in scripts

SCRIPT NAME	SCRIPT FUNCTION
(Gen1) Firmware Deployment Script	Upload and upgrade to a new control program on the CubeComputer
(Gen1) CubeACP Basic Functional Tests	Perform a suite of functional tests for the CubeComputer
(Gen2) CubeACP Firmware Deployment Script	Upload and upgrade to a new control program on the CubeComputer
(Gen2) CubeACP Basic Functional Tests	Perform a suite of functional tests for the CubeComputer

These scripts are executed by pressing the **Run Script** button. Make sure the required hardware connections are made prior to attempting script execution.



7.2 Loading scripts

Some script files will be supplied by CubeSpace as .cs files. These scripts can be loaded and executed from the CubeSupport user interface. To achieve this, press the **open** button in the top left of this window. This enables a file browser from where a script file can be selected. Upon pressing the **OK** button, the script is compiled into executable code, and if there are no errors present, the selected script should be visible in the left-side list. Compiler errors are otherwise displayed in a message window, and must be corrected before attempting to reload the script. A number of scripts can be uploaded at this time.

Select the chosen script and then press the **Run Script** button. This will activate the script by which any output generated by the script will be visible in the **Output** section of the window.

7.3 Script Options

Some scripts may have settings which can be changed before pressing the **Run Script** button. These options, if any, are visible after selecting the script in the **Script Option** UI region.



8 Known Issues

This chapter lists known issues in the CubeSupport application.

ISSUE NUMBER	DETAILS	PRESENT IN VERSION(S)
1	Colours used in Gen2 CubeComputer port map and autodiscovery are not intuitive, difficult to distinguish for colour-blind people and confusing.	7.23
2	An error in CubeSupport for versions 7.23 and before will result in configuration files for the CubeIR not being loaded if the user selects the 'Yes' option (to find all relevant files) in the firmware upload tab.	7.23
3	The automatic UI instantiation for Gen2 CubeComputer behaviour has a known issue in CubeSupport for versions 7.23 and before – message windows with “Failed to exit bootloader” or “Yielded to main application” might be displayed in which case disconnecting and reconnecting to the CubeProduct will be necessary to restore functionality.	7.23
4	Downloading images from CubeSense Sun or CubeStar while the CubeComputer control program is running may experience occasional interruptions or error reporting. Doing it while the bootloader is running, is much less prone to error.	7.23
5	Pass-through UI feature sometimes fails on first attempt and requires retrying connection.	7.23
6	In general, CubeSupport has an outdated user interface and inconsistent user experience across CubeProducts. This will be addressed with a replacement application.	-